UE Gameplay 求职备战终局版 · RFINAL（文字终审替换稿）

> *本稿是 `rfinal.txt` 的完整可替换文本：题号、章节、问题单元和知识覆盖与原稿一一对应，只重写文字、句序和段落。技术内容不增不减；疑似有误之处没有在正文里擅自修改，统一列在 `claude_prose_review_v2.md` 的自检部分和 `claude_final_review_v3.md` 的技术冻结疑点中。*

写在前面：这本书怎么用

基线与规模。 技术基线是 UE5.5。主教材 182 题已经冻结，不再因为单篇面经临时加题。本版在主教材之后补了五部分：Gameplay 场景题 40 道、AS 项目追问 45 道、ISD 项目追问 30 道、绿洲/Lua 实习追问 25 道、生产工程与趋势 15 道。编写方式是 Claude 调研、GPT-5.6 Sol 红队复核；整理日期 2026-10-20。

这一版定下来的几件事。

- 主教材保持 182 题，正好落在约 180 题的目标范围内，不再无边界扩张。复核没有发现需要推倒主教材的技术硬伤，但“资料研究完”不等于“备战体系完成”，所以这一版把力气花在场景题和项目追问上，新技术只放在方向/趋势层。
- 公开面经只用来排复习顺序、判断某类题在什么条件下会被问到，不用来宣称“某题有多大概率被问”。
- 工具、验证、构建、测试在岗位 JD 里出现得多，在题库里被低估了，因此单独补了 15 道生产工程与趋势题。但 JD 里出现得多，不等于面试里问得多，两者不混用。
- AS、ISD、绿洲三个项目合计准备约 100 道追问。这些题只写“问题真正考什么、该怎么想、需要什么证据”，不写成可以背诵的标准答案——项目答案只能来自你自己的代码和运行结果。
- Iris 和 Mover 只做进阶认知。UE5.5 中它们的代码都在 Experimental 路径下，不能挤占传统复制和 CharacterMovement 的时间。
- Motion Matching 和 StateTree 属于方向性学习。做动画或 AI 方向的同学应该能比较它们、说清边界，但它们不是所有 Gameplay 岗位的首轮公共底座。

为什么不再加一本书。 腾娱互动 2026 年 UE5 游戏客户端实习岗位的要求依然集中在框架与功能设计、编辑器与检查机制、性能与测试、3C 与 AI、网络基础、构建发布。这些能力靠场景题和项目证据训练，比继续背 API 清单有效。主教材已经覆盖对象、生命周期、网络、GAS、动画、AI 以及 C++、操作系统、网络基础；再往里加长尾名词，收益比不上项目连续追问、手写代码和拿出运行证据。

四层学习体系

层	内容	怎么练	练到什么程度
底座层	C++、STL、算法入门、操作系统、TCP/UDP、设计模式	闭卷口述，加手写代码	60–90 秒内讲清结论、机制、边界和一个例子
UE 核心层	Framework、UObject 与 GC、反射、复制与 RPC、资源与构建、性能	画调用链，练故障定位	能从对象、状态、执行端、生命周期四个角度解释一次失败
简历防守层	AS 的 GAS 与网络，ISD 的物理与交互，绿洲的 Lua 与多端生产	每条简历描述连续追问五层	答案能落到代码、日志和运行证据，不夸大
方向层	Motion Matching、StateTree、Iris、Mover、开放世界专项	目标岗位需要时再加深	说得清它解决什么、成熟度如何、替代什么、出问题怎么查

复习顺序

1. **第一轮**：第五册 C++ 与算法 → 第九册高频基础 → 第一册玩法框架 → 第二册多人网络 → 第三册对象、资源与构建。目标是把公共底座和 UE 主链串起来。
2. **第二轮**：第六册 GAS → 第四册系统、数学与物理 → 第七册动画 → 第八册 AI → 第十册 Lua 与场景入口。深度按简历暴露面和目标岗位分配。
3. **第三轮**：本版新增的 40 道场景题。每题限定 8–10 分钟口述，必须讲到对象、状态、执行端、生命周期、失败条件和验证方法。
4. **第四轮**：AS、ISD、绿洲项目追问。答案如果指不出代码位置、拿不出运行证据，也没有明确说“尚未验证”，就不算过关。
5. **最后**才看趋势附录。Iris、Mover 这些内容排在传统复制、CMC、生命周期和 C++ 基础之后。

模拟面试自评尺

分数	表现	判定
0–3	只会背定义，一问“为什么”“什么时候会失效”就答不下去	未掌握
4–5	能解释机制，但对象、状态、执行端或生命周期讲不完整	可复习
6–7	能给出调用链、失败条件和替代方案	合格
8–9	能对应到自己的代码、日志、测试和权衡	强
10	连续追问五层仍然自洽，能承认未验证的边界并给出验证方案	面试可用

本书结构

紧接着是精编主教材 182 题，保持原结构和参考资料。主教材之后依次是：

- A. Gameplay 场景题 40 题

- B. AS 项目追问 45 题
- C. ISD 项目追问 30 题
- D. 绿洲/Lua 实习追问 25 题
- E. 生产工程与趋势 15 题
- F. 最终模拟面试清单

外部核验范围。 当前核验过的公开资料包括：腾娱互动“2026 实习生-软件开发-游戏客户端开发-光环工作室-3A 开放世界端游 UE5”官方岗位页，Epic 的 UE5.5 Iris API，以及 Epic 的 Mover、Pose Search、StateTree 文档。岗位 JD 只用来判断需要哪些生产能力，不用来声称某道题必考。

UE Gameplay 面试教材：题目与答案·精编终版·UE5.5

182 题：删减低频专项，补齐高频基础，保留 AS / ISD / 绿洲简历防守。

正文以 UE5.5 为技术基线；普通 C++、操作系统和网络部分以相应标准和官方资料为准。本版从原来的 160 题中删除、合并或移入附录 11 道低优先级题，保留 149 道主干，再补入 33 道高频缺口与简历防守题（C++、操作系统、网络、Lua、场景题）。AS、ISD、绿洲 UGC 只按能确认的简历与项目事实讲解，扩展方案不冒充项目实测。正文优先，低频专项不占首轮背诵时间。

目录

- **第一册　UE 玩法框架**：对象与状态 / 初始化、控制与交互 / 界面与脚本（20 题）
- **第二册　多人网络**：网络对象与数据传播 / 权威、预测与交互（19 题）
- **第三册　对象、资源与构建**：类型、构建与对象 / 垃圾回收与引用 / 回调、资源与构建（14 题）
- **第四册　系统、数学与物理**：系统与网络基础 / 玩法数学 / 碰撞、物理与 ISD / 性能与渲染常识（19 题）
- **第五册　C++ 与算法**：对象与所有权 / 类、继承与模板 / 容器与复杂度 / 并发与错误定位（34 题）
- **第六册　Gameplay Ability System（GAS）**：能力、属性与状态 / 预测、表现与角色重生（18 题）
- **第七册　角色动画**：从玩法状态到动画姿态 / 动作时序、瞄准与位移（13 题）
- **第八册　Gameplay AI**：决策、感知与任务 / 移动、攻击与故障定位（12 题）
- **第九册　高频基础补强**：C++ 高频入口 / 操作系统与并发 / 网络与游戏传输 / 设计模式 / 渲染常识（21 题）
- **第十册　Lua/UnLua 与 Gameplay 场景题**：Lua 与 UnLua / 背包与 Buff / 战斗链 / 索敌 / 异步与幂等 / 调试（12 题）
- 参考资料

第一册　UE 玩法框架

这一册解决一个问题：一段玩法逻辑该挂在哪个对象上、这个对象活多久、谁能改它。后面讲网络、GAS、动画时遇到的很多“为什么为空”“为什么显示旧值”，根子都在这里。

对象与状态

01-01　UObject、Actor、ActorComponent 应该怎样选择?

看它需不需要在世界里有自己的身份、宿主和生命周期。

UObject 是接入 UE 类型系统、序列化和垃圾回收的基础对象。Actor 是世界里的实体，可以独立生成、销毁，也可以参与网络复制。ActorComponent 是挂在某个 Actor 上的一项功能：需要空间变换和父子层级时用 SceneComponent，需要几何、碰撞或渲染时通常用 PrimitiveComponent 的派生类。

拿一扇门来说：门本身是 Actor，门上的通用交互能力做成组件，门禁规则可以是一个数据对象。一次伤害计算只是短暂的数据和运算，通常不必为它生成 Actor。顺带一提，“能不能 Tick”不是选择依据，Actor 和组件都可以 Tick。[1][2]

01-02　Actor、Pawn、Character 有什么区别?

三者是逐层加约定的关系。Actor 是世界实体；Pawn 在此之上约定“可以被 Controller 控制”；Character 再提供胶囊体、骨骼网格和 CharacterMovementComponent 这套人形移动设施。

Character 和它的移动组件是配套设计的，给任意 Pawn 加一个移动组件，并不能得到完全相同的行为。人形的行走、跳跃、网络移动通常适合 Character；固定炮台、特殊载具或自由运动的物体可以直接用 Pawn。

两个常见误解：Pawn 可以暂时没有 Controller，也不一定由真人控制；“需要移动”也不等于必须继承 Character。真正要看的是碰撞形状、运动方式和控制关系是否符合 Character 这套现成约定。[1]

01-03　Outer、Owner、Attachment、Possession 分别是什么关系?

这是四种互相独立的关系，要分别设置、分别检查，不能用一个“父对象”的概念一笔带过。

- **Outer** 是 UObject 的包含和命名上下文，会影响对象路径等机制，但不能理解成“父对象一定保活子对象”。
- **Owner** 是 Actor 逻辑和网络拥有链的一部分，网络系统沿这条链找到拥有连接。
- **Attachment** 是组件之间的空间父子关系。
- **Possession** 表示 Controller 当前控制哪个 Pawn。

最容易踩的坑：把武器挂到角色手部插槽，只建立了空间关系，武器并不会因此获得正确的网络拥有权。同样，把对象地址存进一个普通字段，也没有建立 GC 能追踪的强引用。[1][2][3]

01-04　PlayerController、PlayerState、Pawn 为什么要分开?

因为它们的寿命不同。PlayerController 负责玩家控制和连接相关的逻辑；Pawn 是当前被控制的身体；PlayerState 保存需要跨身体延续的玩家状态。玩家死亡后换一个 Pawn，身份、队伍和比分不应该跟着消失。

联网时还要考虑“谁有这份对象”。常规联网游戏里，服务器有所有玩家的 PlayerController，普通客户端主要只有自己的那一份，不会为每个远端玩家各建一份。PlayerState 通常会复制给相关客户端，适合放公开的玩家信息，但密码、私有库存这类数据不应该因此无条件公开。

所以决定一个字段放哪里，要同时看寿命、权威端和可见范围，光凭“这个数据属于玩家”判断不够。[1][3]

01-05　GameMode 与 GameState 怎样分工?

GameMode 定规则，GameState 发布现状。

GameMode 负责服务器端规则：什么时候开赛、怎样出生、何时判胜。GameState 负责把各端都需要知道的比赛现状发布出去，比如比赛阶段、比分和结束时间。服务器上也有 GameState；客户端通常拿不到服务器的 GameMode。

例如倒计时结束时，应该由服务器按规则切换阶段，再把新阶段和必要的时间基准放进复制状态；客户端根据这些状态显示 UI，而不是在自己的 Widget 里判定比赛结束。可以这样区分：一次动画或提示是事件；“比赛现在处于结算阶段”是需要保留、让后来加入的玩家也能恢复的状态。[1]

01-06　GameInstance、World、Level、Subsystem 的寿命有什么区别?

GameInstance 通常存在于整次游戏运行期间，可以跨地图；World 是当前运行的世界；Level 是世界中的一个关卡及其 Actor 集合。

Subsystem 的寿命跟着宿主走：GameInstanceSubsystem 跟随游戏实例，WorldSubsystem 跟随世界，LocalPlayerSubsystem 跟随本地玩家。所以世界里的泡沫调度、关卡交互注册表适合考虑 WorldSubsystem；本地输入上下文适合 LocalPlayerSubsystem；跨地图的选择和会话管理可以放在 GameInstance 这一层。

两点要记住。Subsystem 不是自动联网的全局单例，服务器和客户端各有自己的实例。选寿命时选“刚好够用”的那个，避免管理器还活着，却一直攥着已经离开世界的对象。[1]

初始化、控制与交互

01-07　BeginPlay 执行了，为什么 PlayerState、UI 或其他依赖仍可能为空?

BeginPlay 只说明当前对象开始参与玩法，不保证它依赖的对象、复制数据和异步资源已经按业务需要到齐。

典型情况是两端进度不同：服务器上的控制关系可能已经建立，客户端的 PlayerState 引用、资源或界面还在路上。解决思路是把初始化写成“依赖齐全才算完成”的过程：BeginPlay、控制关系变化和相关的 OnRep 都可以尝试接入，接入前先检查是否已经绑定过同一个数据源。依赖从旧 Pawn 换到新 Pawn 时，先解除旧关系，再重新初始化。

固定延迟一秒只是改变了出错的概率。可靠的判断条件是两条：依赖确实可用；重复调用不会重复订阅，也不会重复授予能力。[4][5]

01-08　SpawnActor、NewObject、CreateDefaultSubobject 有什么区别？动态组件为什么有时不工作？

三者的入口和协议不同。SpawnActor 通过 World 创建 Actor，并走完 Actor 的生成流程；NewObject 创建 UObject，但不会替它补上 Actor 的生成协议；CreateDefaultSubobject 用在构造函数里创建默认子对象。用普通 `new` 创建的对象不是合法的 UObject，反射和 GC 也不会接管它。

动态组件“不工作”，通常是只做了第一步。运行时 NewObject 出一个组件后，还要给它正确的宿主，并按用途完成注册、空间挂接和必要的初始化——对象存在于内存里，不代表它已经参与碰撞、渲染或 Tick。如果组件需要复制，还要满足 Actor、组件和子对象各自的复制条件；即使组件设成可复制，它引用的对象也不一定立刻在客户端可用。

排查时把“创建、注册、联网、依赖就绪”当成四步分别检查。[4][6]

01-09　Tick、Timer 和事件通知怎样选择？

按“什么时候需要更新”来选。逐帧连续的运动、瞄准更新，或必须跟着当前帧走的计算，用 Tick；有明确间隔的定时工作用 Timer；只在数据变化时才需要更新的逻辑，用事件或委托。比如血量没变就不必每帧刷新文字，但被持有物体的目标位置可能需要持续更新。

Timer 并不天然比 Tick 便宜。把每帧轮询换成大量极短间隔的 Timer，只是换了一种调度成本。

必须用 Tick 时，要能在闲置时关掉，必要时降低频率。对先后顺序有要求时，用合适的 TickGroup 或显式依赖，不要依赖某次观察到的 Actor 遍历顺序。[7]

01-10　增强输入系统怎样把按键变成玩法动作？

增强输入把“按了什么键”翻译成“玩家想做什么”。Input Action 表示开火、跳跃、移动这类玩法意图；Mapping Context 把设备输入映射到 Action；Modifier 处理输入值，比如死区、反向和灵敏度；Trigger 决定长按、连按、松开等触发条件。

Started、Triggered、Completed、Canceled 是不同的事件。Triggered 会不会逐帧触发取决于具体的 Trigger，不能默认它只执行一次。

Mapping Context 属于本地玩家的输入配置，输入回调再转交给当前角色或技能系统。重生后按键没反应时，按顺序查：本地玩家、当前 Pawn、Mapping Context 和绑定是否仍然正确；再查 UI 是否截获了输入、技能是否允许执行。不要因为一个按键没反应，就同时去改输入、网络和能力系统。[8]

01-11　ControlRotation、ActorRotation、Velocity 与 Mesh 朝向为什么可能不同？

因为它们描述的是四个不同的量。ControlRotation 通常表达控制或瞄准方向；ActorRotation 是角色实体的朝向；Velocity 是实际移动的结果；Mesh 还可能因为模型初始旋转和动画产生视觉偏移。它们彼此有关联，但不是同一个东西。

第三人称自由视角就是例子：相机转了，角色不一定跟着转；横向移动时，移动方向也不一定等于瞄准方向。所以要先决定角色是“朝移动方向转身”还是“跟随控制方向”，再去设置 CharacterMovement 和角色旋转相关的选项。

动画一般读取速度、移动模式和相对瞄准角，而不是用本地按键推测运动——远端角色根本没有本机玩家的按键信息。[1][9]

01-12　CameraComponent、PlayerCameraManager 与 ViewTarget 怎样配合?

CameraComponent 在某个 Actor 上提供一个摄像机视点；PlayerCameraManager 决定本地玩家最终看到什么，并处理混合、震动等效果；ViewTarget 是当前观察的目标。

相机可以来自当前 Pawn，也可以来自观战目标、过场相机或其他 Actor。所以死亡观战不一定要把 Controller 的控制权交给被观察的角色，只切换 ViewTarget 就可以。

和网络相关的一点：服务器的权威射击不能依赖“服务器恰好有一个本地相机”，专用服务器根本没有本地玩家视角。客户端发来的瞄准信息应该当作受约束的输入，由服务器结合角色和武器状态验证。[1][10]

01-13　重生后 HUD 显示旧血量，应该怎样修?

通常不是血量算错了，而是 HUD 还订阅着旧 Pawn、旧属性源或旧委托。旧对象即使还没被 GC，也已经不是界面应该观察的那具身体，所以只检查指针非空或 IsValid 不够。

正确做法是：控制关系或数据源变化时，解除旧订阅、绑定新数据源，并立刻读一次当前值。如果 ASC 常驻在 PlayerState 上，要分清“ASC 没变”和“Avatar 变了”：属性的观察源可能保留，但身体、动画和装备引用仍需要刷新。

还有两个细节。旧的异步回调到达时，只能更新它自己所属的旧请求，不能覆盖新 HUD。订阅要在关系变化时主动管理，而不是每帧反复 GetPawn 碰运气。[1][4]

01-14　怎样拆分庞大的 Character，又不把它变成互相调用的组件堆?

按“数据由谁持有、什么时候失效”来划分职责。角色负责身体和运动；交互组件负责检测和交互入口；装备组件负责当前装备关系；跨重生的状态放在合适的长寿命对象上；规则由权威端决定。

拆分的目标是让每项状态都有清楚的修改入口，而不是单纯减少文件行数。组件之间优先通过小接口、参数或事件协作：武器提交一次射击请求，不直接去改 HUD；血量变化通知表现层，但不反过来依赖具体的 Widget。

两个边界。非常简单、不会复用的逻辑留在角色里完全合理。反过来，如果每个组件都随手拿到别的组件并改它的内部字段，耦合只是换了个地方。[1]

01-15　一个交互系统怎样同时支持门、物品和密码面板?

把“找目标”和“目标自己的规则”分开。玩家这一侧统一负责寻找候选目标、显示提示、提交交互意图；目标那一侧通过接口回答能不能用、提示什么，以及具体怎么执行。门决定开关规则，物品决定能否拾取，密码面板决定怎样验证输入，玩家角色不需要认识每一种具体的类。

两次检查各有分工。显示提示时的可用性查询应该只读状态，不能在这时扣道具或开门。真正执行时要再检查一次，因为从看到提示到按下按键之间，目标可能已经被别人用掉或销毁了。

联网时，客户端的查询主要用于反馈，服务器仍然要检查目标、距离、权限和当前状态。统一接口解决的是代码耦合，并发交互和网络合法性要另外保证。[1][11]

01-16　ISD 这样的门禁、凭证和密码机制怎样组合，而不是写死在玩家身上?

思路是让门只管“能不能开”的最终决定，许可条件由别的对象提供。门对外只暴露“请求激活”和当前状态；读卡器、密码面板等对象各自提供不同的许可条件；凭证属于玩家或背包的数据，密码规则属于具体面板或关卡规则。交互入口查询这些条件后，再交给门唯一的状态修改入口决定是否打开。这样同一扇门可以被钥匙、密码或世界事件触发，Character 里不用不断加类型判断。

组合时语义要说清楚：钥匙是“验证通过就消耗”还是“开门成功才消耗”，门已经开着时会不会重复消耗，任务条件变化后是否允许关门。这些是玩法选择，应该由对应的规则对象明确表达，而不是藏在 UI 回调里。

这里讲的是设计方式，额外机制不代表 ISD 已经实现。[1][11]

界面与脚本

01-17　UMG、Slate、Widget Blueprint 分别是什么?

Slate 是 UE 的 C++ 界面框架，提供控件、布局和输入处理；UMG 建在 Slate 之上，提供 UObject 化的 Widget 和可视化设计器；Widget Blueprint 是 UMG 的可视化资产及其逻辑。

共享一个 Widget Blueprint 不等于共享界面运行状态，每个创建出来的 Widget 都有自己的状态和关联控件。

更重要的是数据方向：Gameplay 数据不应该以“某个 Widget 存在”为前提。血量、背包和比赛阶段由各自的状态源维护；界面打开时先读现状、再订阅变化，关闭时解除自己的订阅。这样重建界面、切换输入模式或晚加入游戏时，都不需要把曾经发生过的玩法事件重放一遍。[12][13]

01-18　为什么长列表要使用虚拟化？条目复用最容易错在哪里?

长列表没必要为每条数据都建一个完整控件。虚拟化只为可见区域和少量缓冲生成显示条目，滚动时复用它们，所以“数据项”和“显示实例”必须分开。

复用时最容易错在刷新不完整：不能只换名字，选中状态、图标、数量都要刷新，还要解除对旧数据的监听。典型 bug 是：某个条目开始异步加载物品 A 的图标，随后被复用来显示物品 B；A 的图标即使加载成功，也只能被丢弃。回调要核对当前数据身份和本次请求编号；条目离开列表后，也要取消或忽略旧结果。

抓住这一点：问题不在“异步不可靠”，而在显示对象的寿命和某一条数据的寿命不一样。[13][14]

01-19　Lua 的 table、闭包和垃圾回收，与 UObject 有什么关系?

Lua 的 table 既能当数组，也能当键值表。把它赋给另一个变量，通常只是两个变量共享同一张表，不是深拷贝。闭包通过上值保留外部变量：事件回调捕获了一张表，即使别处不再用它，这个回调仍可能让它保持可达。

Lua 的垃圾回收和 UE 的 UObject 回收是两套机制，中间怎么关联取决于具体的绑定层。所以有两个判断都不成立：Lua 变量不是 nil，不能说明底层 Actor 还能用；脚本局部变量离开作用域，也不会立刻销毁 Actor。

绿洲这类受限运行环境，还要按平台接口确认对象有效性和事件解绑方式，不能把某个普通 Lua 插件的行为直接套过去。[15][2]

01-20　Lua、蓝图、UMG 与 DataTable 怎样分工，才能避免状态重复和配置不一致?

一种清楚的分工是：DataTable 保存静态规则和参数；Lua 或 C++ 推进运行状态；蓝图组织资产和表现；UMG 显示状态并发出操作意图。具体边界由项目工具决定，但同一项运行状态不能在脚本、蓝图和 UI 里各自维护一份互不相干的“真值”。

例如“下一波”按钮只发请求；玩法逻辑检查当前阶段后安排刷怪；UI 跟着阶段变化更新。如果出现长时间不出怪，要区分是请求没被接受、结算计时器还没走完、资源没准备好，还是刷怪条件失败，而不是先把所有延迟都调短。

配置里的行名、路径和数值应该在读取时校验，让错误尽早暴露。编辑器预览成功，只说明编辑器里的数据没问题，服务端部署的数据要另外确认。[1][12][16][15]

第二册　多人网络

学网络先建立一个画面：服务器和每个客户端各有一份世界，网络上传的是对象身份、状态和请求，不是指针和调用栈。这一册的每道题，都可以回到“这段代码在哪一端运行、改的是哪一份对象”来想。

网络对象与数据传播

02-01　联网以后，同名 Actor 是同一个对象吗？Listen Server 与 Dedicated Server 有什么区别?

不是同一个对象。服务器和每个客户端都有自己的 World 和对象实例。客户端调用一个普通函数，只会执行本进程里的代码，不会进入服务器那一份对象。多人 PIE 即使跑在同一个进程里，不同 World 中的对象仍是各自独立的状态。

Listen Server 既是服务器，又带一个本地玩家。在主机上测试时，“只有本地玩家才有相机或 UI”这类依赖很容易被掩盖。Dedicated Server 没有本地玩家体验，适合集中运行权威规则。

无论哪种模式，都要区分本地表现和服务器裁决。主机上一切正常，不代表远端客户端的网络请求已经走通。[1][3]

02-02　Authority、Ownership、LocalControl 和网络 Role 分别说明什么?

它们回答的是四个不同的问题：

- **Authority**：当前这份 Actor 副本是不是权威的。
- **Ownership**：这个 Actor 属于哪个连接，决定 RPC 能不能从那个连接路由过来。
- **LocalControl**：当前机器是否直接控制这个 Pawn。
- **Role**：这份副本在网络中扮演什么角色。常规的、由服务器生成并复制的角色，在服务器上是 Authority，在拥有者客户端上通常是 Autonomous Proxy，在其他客户端上是 Simulated Proxy。

这些概念不能互相替代。服务器上的远端玩家角色有权威，但并不是服务器的本地玩家在控制；客户端本地生成的、不复制的 Actor，在本地也可能是 Authority。所以 HasAuthority 不能用来万能地判断“当前进程是不是服务器”。

记一个对应关系：判断运行端看 NetMode，判断网络角色看 Role，判断本地输入看本地控制和玩家关系。[3][9]

02-03　一个客户端的 Server RPC 怎样到达服务器？为什么常放在玩家拥有的对象上？

Server RPC 本质上是“把一次函数调用编码成网络消息”。声明为 Server RPC 的 UFUNCTION 会经过 UHT 生成相关代码。调用时，UE 判断调用发生在哪一端、走哪条网络连接，把函数参数和对象身份按协议编码后发出去；服务器在自己那一份对应对象上解码并执行实现函数。它不是跨机器共享调用栈，也不会传递原始指针地址。

客户端能发起 Server RPC 的前提，是调用对象满足复制和拥有连接等条件。一扇公共的门通常不归某个玩家所有，所以更常见的入口是玩家的 PlayerController、Pawn 或它们合适的组件：请求里带上门的网络引用，由服务器检查后再操作门。

两个误区：客户端自己调 SetOwner，不能给自己授予服务器认可的权限；为了交互临时把公共物件的 Owner 转来转去，也会增加多人竞争和路由问题。[3][17]

02-04　Reliable RPC 能保证什么？连续发送两个 RPC，能确定何时到达吗？

Reliable 提供确认、重传，以及规定范围内的执行顺序；它不保证到达时间。

顺序保证是有范围的：同一个 Actor 及其关联子对象上的可靠调用，可以依赖文档规定的顺序；不能推广成不同 Actor 之间的全局顺序。可靠调用和不可靠调用之间，也不能随意建立业务上的先后依赖。

时间方面，网络延迟、丢包、排队和接收端处理都会影响到达时刻，断线、对象销毁等情况还会直接打断流程。每帧发送大量 Reliable 会积压可靠队列，甚至引发连接问题。

所以，需要“一组数据一起成立”的业务，通常应该组织成同一条消息，或者组织成可以独立恢复的状态，不要指望两个调用恰好连续到达。[3][5]

02-05　RPC 与复制属性应该怎样选择？

RPC 表达一次请求或事件，比如“尝试拾取物品”；复制属性表达现状，比如“物品已被谁持有”。判断的关键问题是：错过当时那次事件的客户端，后来是否还需要知道结果？

门已经开了、玩家已经死了、比赛已经结算，这些都不能只靠过去发过的一次 Multicast 来保存。Multicast 可以用于当前相关客户端的一次性表现，但它不是历史事件存储，也不保证所有潜在客户端都能收到。

通常的做法是：服务器先改变权威状态，再由状态变化触发表现；只有不需要长期恢复的音效、短暂粒子等才用事件。即便如此，本地预测的表现和服务器发来的事件仍要避免重复播放。[3][17]

02-06　属性从服务器变化到客户端更新，经过哪些步骤？

以传统的 Actor 复制路径为例，大致是四步：

1. 服务器决定哪些 Actor 需要更新，以及对某个连接是否相关；
2. 按复制条件、脏标记或差异比较，决定这次发送哪些属性；
3. 数据经过网络序列化和对象引用映射，形成这个连接上的网络消息；

4. 客户端解码，写入自己那份对象，并按规则触发复制通知。

可见，“序列化”只是数据编码那一部分，不等于整个复制系统。复制还要决定发给谁、发什么、什么时候发、引用怎么对应，以及收到后怎样使用。

还要记住复制方向：普通复制属性从服务器传到客户端，客户端修改同名变量不会自动上传。Iris、Push Model 等会改变部分内部路径，所以不要把某一种后端的类名当成所有 UE 项目的固定调用链。[17][18]

02-07　OnRep 会在什么时候执行？为什么不能把所有玩法逻辑放进去？

OnRep 是客户端收到复制属性后，更新本地派生状态和表现的主要入口。但它不保证每个中间值都能被看到：服务器在很短时间内把数值从 1 改到 2 再改到 3，客户端可能只收到 3。通知策略还会影响“值没变时是否触发回调”。

两套机制要分清。在 C++ 里，服务器本地修改复制字段，通常不会自动执行对应的 OnRep；蓝图的 RepNotify 在赋值时的行为又不一样。

常见做法是把“在本地应用这个状态”提成一个普通函数：服务器改完状态后按需调用它，客户端在 OnRep 里调用它。奖励、伤害这类权威提交仍然在服务器的规则入口完成，不能让一次界面刷新或一次重复通知再产生一次副作用。[17]

02-08　网络对象已经出现，为什么引用或其他属性仍未准备好？

因为网络传的是对象身份，接收端要把身份解析成自己 World 里的对象，而不是直接拿到服务器的地址。对象、组件、引用和后续状态不一定同时到达；不同 Actor 的更新之间，也没有可供业务随意依赖的全局顺序。

把多个相关字段聚合进同一个结构，可以减少分散通知造成的观察不一致，但这并没有建立跨 Actor 的事务，依赖的对象仍可能暂时解析不到。

所以消费数据的函数要做三件事：检查需要的数据是否齐全，允许之后重试，避免重复绑定。如果数据和阶段、版本有关，就显式携带版本号或操作身份，不要凭 OnRep 碰巧的执行顺序去猜当前阶段。[5][17]

02-09　晚加入玩家怎样恢复当前世界？为什么会出现门的动画和碰撞不一致？

服务器要保存足以解释当前世界的状态。新客户端收到相关 Actor 和它们当前的属性后，再通过初始化或复制通知恢复表现。比如门要有能说明问题的开关状态，正在运动的门可能还需要开始时间或进度；比赛需要阶段和时间基准。

门“看起来关着，碰撞却已经打开”，常见原因是动画只由过去的事件触发，而碰撞由当前的复制字段设置——两者消费的不是同一个东西。解决办法是让它们消费同一个状态，而不是在玩家加入时重放整局的 RPC。

初始创建、重新相关和运行中变化，可以复用同一个“本地应用”函数。这个函数只负责恢复现状，奖励和钥匙消耗这类一次性副作用不在它的职责里。[17][18]

02-10　相关性、休眠和更新频率各在优化什么？

三个手段优化的是不同环节：相关性决定某个连接是否需要某个 Actor；休眠减少长期不变的对象的复制处理；NetUpdateFrequency 控制传统复制路径中“多久尝试更新一次”，并不保证每秒一定送达那么多次。它们不是同一个“少发包”开关。

休眠对象的状态需要改变时，要按所用的复制路径正确唤醒或刷新，再修改要传播的数据，否则可能漏掉更新。

优化完要验证三种情况仍能恢复：晚加入、重新相关、休眠期间发生的状态变化。带宽降下来了，却让客户端永远停在旧状态，不算有效优化。具体阈值由实际的对象规模和状态变化频率决定。[18][19]

权威、预测与交互

02-11　服务器校验客户端请求，主要在校验什么？

校验的是“这次操作是否被允许”，数据格式正确只是最低要求。服务器要确认：请求来自哪个玩家，当前身体和武器是否对得上，资源和冷却是否允许，目标是否有效，距离和视线是否合理；同时限制频率、时间范围和重复请求。客户端可以提供输入意图、瞄准方向或候选目标，但伤害值、奖励和重要状态要由服务器按规则计算。

校验也要考虑延迟。双方的观察时刻不同，所以不能拿服务器当前这一瞬间的视角，和客户端输入做零容差的逐位比较。可信的方案要定义允许的偏差和降级策略：既不能把正常延迟造成的偏差一律判成作弊，也不能为了掩盖协议问题而放宽所有检查。[3][20]

最后要清楚服务器校验的边界。一份输入通过了上面所有检查，也不能证明它是真人产生的。服务器校验管的是“这次操作合不合规则”，客户端完整性保护和行为检测属于其他层次。Gameplay 程序首先要保证的是：客户端不能直接指定最终伤害、奖励或任意位置，并且尽量少下发不必要的敏感状态。不要承诺“校验了方向就能彻底防住自瞄”。[3][24]

02-12　两个玩家同时拾取同一物品，怎样避免双重奖励？

客户端只提交拾取意图；服务器在一个清楚的状态修改入口里检查物品是否还能领取，先完成“可领取 → 已领取”的转换，再执行奖励和广播。

这里首先要防的不是多线程。常规 Gameplay 回调主要在游戏线程上执行，真正的风险是“检查”和“提交”之间的异步、重入和重复请求，所以不要不分情况地加线程锁。如果奖励走异步流程或外部服务，还要防止同一次操作被重复提交。重复到达的请求应该得到一致的结果，不能再发一份物品。

高延迟下第一次交互没反应时，先查请求资格、服务端拒绝原因和状态传播，不要让客户端直接重复销毁目标，或每帧重发可靠 RPC。[3][17]

02-13　状态同步、帧同步和混合方案怎样选择？晚加入、断线重连与权威校验分别怎么处理？

帧同步传播的是输入或命令，各端依靠确定性模拟算出同样的结果，适合规则离散、模拟可以严格控制的玩法。状态同步由服务器算出权威结果再传播必要的状态，裁决更容易统一，但要处理带宽、插值和预测。实际游戏常常按子系统混合使用，而不是整个项目只选一种。

两种方案的配套设施不同：帧同步需要输入序号、迟到处理、校验和，以及回滚或追帧；状态同步需要快照、增量、预测和收敛。

晚加入通常从一份可信快照开始；断线重连要恢复会话和当前状态。无论哪种方案，都不能因为“各端模拟结果一致”就放弃服务器对输入资格和作弊边界的检查。[21][22]

02-14　CharacterMovement 的本地预测、服务器校正和重演是怎样工作的?

主因果链是“先本地跑，服务器复算，出错就校正并重演”。拥有者客户端先用本地输入模拟移动，并保存还没被确认的移动记录。服务器收到移动数据后按权威规则重新模拟，然后接受结果或发回校正。客户端应用校正后的状态，再把尚未确认的输入重新模拟一遍——因为网络往返期间玩家已经继续操作了，这些输入必须保留下来。

这就解释了自定义冲刺为什么会被拉回：只改客户端速度不够，影响模拟的状态必须进入合适的 SavedMove、服务器模拟和移动合并规则。否则客户端以为自己在跑，服务器却按另一套速度计算，就会不断把客户端拉回去。

两个边界：记录输入是为了能恢复同一段模拟，不是让客户端最终决定位置；带副作用的代码不能随便放进重演路径，否则一次输入会多次发奖励或多次播放不可重复的效果。[9]

02-15　远端角色为什么通常需要插值？什么时候可以外推?

远端角色没有本机玩家的即时输入，客户端只收到一个个离散的网络快照。

插值的做法是让显示用的时间稍微落后于估计的服务器当前时间，这样总能在两个已知快照之间插值，吸收到达时间不均匀带来的抖动，代价是表现延迟更大。外推则根据已有的速度等信息预估未来，短时间内能保持运动连续，但遇到突变和遮挡，误差会被放大。

新状态到达后需要收敛。瞬移、重生或移动模式突变时要单独处理，不能按普通轨迹一路平滑过去。

定位抖动时，先看角色碰撞体和网络快照，再看 Mesh 平滑和动画姿态。如果实体位置稳定、只有骨骼姿态在抖，再怎么调网络更新频率也可能没用。[9][21]

02-16　Hitscan 与投射物有什么不同？服务器权威射击应怎样组织?

Hitscan 在某个查询时刻，沿射线或形状一次性判断命中；投射物有持续的飞行、重力和碰撞过程。散弹说的是“一发包含多条弹道”，和“即时射线还是实体投射物”不是同一个分类维度。

权威射击通常从一次合法的开火请求开始：服务器验证武器、资源、射速和瞄准信息，决定射线命中或生成权威投射物，最后经过统一入口结算伤害。本地可以先播放后坐力、枪口火光和预测弹体，但伤害只从统一入口提交一次。

还要分开检查摄像机射线和枪口路径，否则会出现“相机看到了目标，枪口却被近处的墙挡住”的穿墙射击。[20][23]

02-17　预测弹体与服务器权威弹体怎样对应？PredictionKey 与 ShotId 是一回事吗?

不是一回事。PredictionKey 服务于 GAS 中受支持的预测操作，用来关联确认与回滚；ShotId 是项目自己定义的“一次射击”的身份，可以用来匹配本地预测弹体和权威结果。一个技能可能开火多次，一发射击也可能包含多个弹丸，所以不经设计就把两种编号等同起来是错的。

AS 的简历描述是：用角色和 ShotId 对应权威弹体，权威弹体到达时清理预测对象。这个方案能给出即时反馈，但还需要定义清楚几件事：编号的作用域、失败或超时怎么处理、权威弹体还没生成就已经命中怎么办。

另外两点：服务器确认了技能，不代表对应的 Actor 已经在客户端可用；新旧弹体怎样切换、要不要平滑误差，属于表现策略，GAS 不会自动包办。[20][23]

02-18　重生、换武器后才到达的旧结果，应该怎样处理?

先判断这个结果属于哪次执行、哪具身体、哪把武器，而不是只看目标指针还活着。同一个玩家重生前后可能是两个不同的角色；同一把武器也可能已经开始了下一次操作。可以用实际的对象身份、射击序号和必要的代次信息来区分，具体用几个字段由协议决定。

旧结果可以结束它自己的等待、清理它自己的预测对象，但不能覆盖新身体的 HUD、取消新技能，或重新设置当前装备。

区分两种身份：网络对象身份解决“对应的是哪一份对象”，业务执行身份解决“这是不是当前这一次操作”。两者有联系，但谁也替代不了谁。[5][20]

02-19　服务器状态正确，但某个客户端表现错误，应该沿哪条链查?

沿着数据流动的方向，一节一节往下查，找到第一个不符合预期的位置就停：

1. 服务器的最终状态是否真的正确；
2. 这个连接是否需要这个 Actor，Actor 是否处于休眠；
3. 属性是否发送、是否接收，引用能否解析；
4. OnRep 或本地消费函数是否更新了当前的 UI、动画或 Mesh。

现象本身也能缩小范围：只有晚加入的玩家出错，优先怀疑依赖了过去的事件而没有恢复现状；只有拥有者出错，检查预测结果和权威结果是否重复播放、互相抵消或用了不同的数据源；只有重生后出错，检查旧绑定和当前 Avatar。

记录执行端、对象身份、属性值和请求编号，比在所有函数里无差别打印更容易还原问题。[17][18]

第三册　对象、资源与构建

这一册讲 UE 对象从“被声明”到“被回收”的整条线：反射信息怎么生成，对象怎么创建、怎么保活，资源怎么从磁盘进入内存，代码和资产怎么变成发布包。很多看起来随机的崩溃和丢资源，都能在这条线上找到具体的断点。

类型、构建与对象

03-01　UCLASS、UPROPERTY、UFUNCTION 怎样接入 UE 反射？它们是不是 C++ RTTI?

UHT 在普通 C++ 编译之前处理这些反射声明，生成类型注册、属性和函数相关的代码，然后和其他 C++ 代码一起编译。UCLASS 让类接入 UE 的类型系统；UPROPERTY 可以按类型和说明符参与 GC、编辑器、序列化或复制；UFUNCTION 可以提供反射调用、蓝图事件或 RPC 入口。

它不是 C++ 原生 RTTI 的简单替代。RTTI 主要支撑 typeid、dynamic_cast 这类运行时类型功能；UE 反射还要服务资产、编辑器和网络。也不要以为写了宏就自动复制、自动存盘或线程安全，这些功能各有自己的条件。

蓝图相关的三个说明符常被一起问：BlueprintCallable 表示蓝图可以调用；BlueprintImplementableEvent 由蓝图实现；BlueprintNativeEvent 允许 C++ 提供默认实现、再由蓝图覆盖，C++ 默认实现写在对应的 `_Implementation` 函数里。[2][25]

03-02　CDO 是什么？为什么构造函数不能直接操作当前玩家？

每个 UClass 都有一个类默认对象（CDO），用来提供默认属性等信息。原生构造函数既可能参与 CDO 的建立，也会参与普通实例的创建，所以构造函数执行时，当前游戏世界、玩家、控制器或网络连接都未必存在。

因此构造函数只适合设置默认值和创建默认子对象，不能假设当前玩家已经出生。运行时依赖要放到满足其条件的阶段再处理。

另一个常见困惑：改了 C++ 默认值，已有的蓝图或关卡实例却还是旧值。这不一定是编译没生效，而是它们保存了自己覆盖过的值。排查时分清三层——原生默认值、蓝图类默认值、具体实例值，不要靠反复重新编译来消除这种差别。[2]

03-03　UBT、UHT、C++ 编译器与链接器分别做什么？

UBT 根据 Target、Module 和依赖关系组织整个构建；UHT 处理 UE 反射声明并生成辅助代码；编译器对每个翻译单元做语义分析和代码生成；链接器解析跨文件的符号，把目标代码和库组合起来。

理解了分工，就知道 `#include` 只让当前翻译单元看见声明，不保证链接时一定有定义。遇到“未解析的外部符号”，应该检查：函数是否有定义、定义是否进入了当前模块、模块依赖和 API 导出是否正确，而不是继续乱加头文件。只有本机能构建时，还要检查目标配置、条件编译和遗留的旧产物。前向声明能减少包含依赖，但需要完整类型、访问成员或按值定义对象时，仍然要有完整定义。[25][26][27]

静态库和动态库的区别也常被顺带问到。静态库在链接阶段把需要的目标代码并入最终产物；动态库在构建或运行时通过导入表、符号和装载器建立依赖，可以单独更新，但会带来部署、版本和符号可见性的问题。动态加载成功也不代表接口 ABI 兼容，所以跨模块边界时，不要随意暴露不稳定的对象布局，以及“谁分配谁释放”的责任。[25][26]

03-04　Runtime Module、Editor Module 与 Plugin 应怎样划分？

Module 是 UE 构建和依赖组织的基本单位。Runtime 模块放运行时需要的代码；Editor 模块可以依赖编辑器设施；Plugin 可以包含多个模块和内容，用于功能组织和分发。它们按职责划分，不是按文件夹名字区分的同义词。

最常见的事故是：游戏运行时模块无条件依赖了只在编辑器目标里存在的类型，结果编辑器构建正常，一打包就失败。公开头文件暴露的依赖和只在 cpp 里用到的依赖也要分开，避免把不必要的依赖传播给其他模块。

做插件不是为了让代码都显得“高级模块化”，而是为了有清楚的功能、依赖和启停边界。[26]

03-05　Actor 的构造、BeginPlay、EndPlay、Destroy 和 GC 是什么关系？

把它们看成四件不同的事：创建、参与世界、退出玩法、释放存储。

构造建立对象的初始结构；Actor 经过生成或加载流程完成初始化，组件注册后参与世界中的各个系统；BeginPlay 标志它开始参与这一轮玩法。Destroy 请求结束这个 Actor 的玩法存在，但不会立刻执行 C++ 的 delete；世界切换、关卡卸载等路径也会触发 EndPlay。EndPlay 和最终的内存回收不是同一个时间点。

所以停止任务、解除外部订阅、清除世界关系这些工作，应该在关系结束时做，而不是等到析构或 GC。在流送等情况下，EndPlay 之后对象甚至可能在内存里再留一段时间，因此“指针还能读到字段”不能证明它仍可用于当前玩法。[4][6]

垃圾回收与引用

03-06　UE 的垃圾回收为什么不是引用计数？Outer 能保证子对象活着吗？

UObject 的 GC 主要看可达性：从根和引擎保活结构出发，沿着可追踪的引用图走，走得到的对象保留，走不到的进入回收流程。因此，一个和外界完全断开的强引用环，不会像普通 shared_ptr 的强引用环那样必然永远活着。

Outer 主要提供包含和命名上下文，不能当成通用的保活承诺。想让对象活着，就要让它处在 GC 能识别的引用路径上；对象内部记着自己的 Outer，并不能证明持有者那边引用了它。

两个补充：AddToRoot 可以强制保活，但必须配对解除，不能用来掩盖所有引用设计错误；Actor 和组件还有自己的销毁协议，外部强引用阻止不了合法的 Destroy 或 DestroyComponent。[2][28]

03-07　TObjectPtr、裸 UObject 指针、TWeakObjectPtr、TSoftObjectPtr 怎样选择？

按“要不要保活、要不要已加载”来选。

- 常见的 UObject 成员，用 UPROPERTY 标记的 TObjectPtr 形成可追踪的强引用。
- 裸 UObject 指针可以用来短暂借用，但没有登记的普通字段不会自动成为 GC 根。
- TWeakObjectPtr 用来观察对象，不延长它的寿命。
- TSoftObjectPtr 主要保存软路径，适合在资源还没加载时表达引用。

TObjectPtr 这个模板名本身并不保证保活：放在任意普通 C++ 对象里，它仍然要进入收集器能遍历的路径才算数。软路径也不是已加载资源的强引用，加载完成后要明确由谁让它驻留。

对象偶尔被 GC 掉时，要同时检查两件事：引用字段是否可追踪；持有者本身是否可达。只把星号换成另一种指针模板解决不了问题。[2][28][29]

03-08　TSharedPtr、TWeakPtr 与 UObject 引用为什么不能混用？UE 容器和 STL 又怎样选择？

因为它们管理的是两套不同的寿命。TSharedPtr 和 TWeakPtr 管理普通 C++ 对象的共享所有权，靠共享控制块配合释放；TWeakObjectPtr 观察的是 UObject，遵守 UObject 的回收和有效性规则。所以不能用 TSharedPtr 去删除 UObject，也不能指望 UObject 的 GC 去释放普通 C++ 资源。

容器的选择也按边界来：需要参与 UE 反射、蓝图、序列化或引擎 API 时，用受支持的 UE 容器和类型，比如合适的 TArray、TMap；纯 C++ 模块可以用标准库，但跨进 UObject 边界时，要重新明确引用怎样被追踪。

还要注意，TArray 扩容同样可能让元素地址失效，“换成 UE 容器”并不能消除容器本身的生命周期问题。[2][28][30]

03-09　IsValid 返回真，为什么仍可能用错对象?

IsValid 只回答“这个对象在对象系统里还能不能用”，不回答“它是不是当前玩家的身体、当前界面的数据源、当前请求的目标”。

旧 Pawn 还没销毁、新 Pawn 已经被控制时，两个都是有效的，但 HUD 只应该显示当前关系指向的那一个。所以回调里除了检查对象有效，还要按业务检查当前的 Pawn、World、数据项或操作编号。

这并不是要求每个函数都套一层复杂的“代次框架”，只要在可能换对象、可能重入、或异步完成的边界保留足够的身份信息就行。抓住这一点：对象活着，和它仍属于这次操作，是两个问题。[2][4]

回调、资源与构建

03-10　委托、Timer 和 Lambda 为什么容易留下生命周期错误?

它们都把调用推迟到未来，而被捕获的 this、对象引用或业务关系可能先结束了。按值捕获一个指针只复制了地址，不会延长目标的寿命；弱绑定能减少悬空访问，但保证不了回调仍然属于当前的身体或当前的操作。

所以注册时就要想清楚由谁负责解除。必要时保存委托句柄或 Timer 句柄，在 EndPlay、界面移除或目标更换时清理。普通 Lambda 和绑定到 UObject 成员函数的计时器，自动清理的行为不一样，不能一概而论。

回调入口仍然要做必要的对象和身份检查。清理时也只清自己登记的工作，避免旧任务把新任务的监听一起移除。[7][2][31]

03-11　异步请求取消了，为什么完成回调还可能到来?

因为“取消”通常只是通知外部工作不要继续，已经进入完成队列的回调并不会从时间线上抹掉。请求还可能在提交函数内部就同步完成，或者在完成通知里又触发新请求。所以状态登记、回调订阅和实际提交，要按接口约定的顺序组织。

一个简单的做法是：为当前加载保存请求编号和目标，结果到达时核对它是不是仍然是当前请求。旧结果只释放自己的句柄，不去更新新目标。如果允许新请求覆盖旧请求，就在覆盖的那一刻让旧身份失效。

只比对资源路径不够，因为连续两次加载同一路径，也可能属于不同的物品槽或不同的角色。[29]

03-12　Asset Registry、Asset Manager、StreamableManager 各负责什么?

三者对应三件不同的事：知道资产存在、决定资产怎么管理、完成一次加载。

Asset Registry 提供资产元数据和查询，通常不用先加载完整资产就能知道它的类型和标记。Asset Manager 组织主资产、加载规则和相关策略。StreamableManager 和加载句柄负责处理具体的软引用加载请求。

加载完成也不等于永远驻留。资源还在不在，取决于强引用、加载句柄和管理规则；只留一个软路径，保证不了已加载的对象不被回收。反过来，释放某个句柄也不意味着对象立刻卸载，因为其他引用和引擎机制可能还持有它。[29][32]

03-13　Build、Cook、Stage、Package、Load 有什么区别?

五个词对应五个阶段：Build 生成代码产物；Cook 把资产处理成目标平台可用的数据；Stage 收集部署需要的文件；Package 按平台方式生成分发产物；Load 是运行时真正解析并加载对象。用“打包”一个词概括全部过程，会让问题无从定位。

编辑器能找到某个资产，只说明它在开发环境里存在，不能证明它进了目标包。动态字符串拼出来的路径、没被规则收集的软引用、只在编辑器可用的内容，都可能在运行时缺失。

遇到问题先判断失败在哪个阶段，再看对应的日志和产物。不要把所有目录都强制打进包里，来掩盖说不清的引用。[26][33]

03-14　编辑器正常，打包版缺资源或加载失败，应该怎样检查?

先判断是哪一类失败：资产没被收集、文件没被部署、运行时路径不对，还是目标平台不支持相关内容。看 Cook 日志和包内文件，就能区分“资源根本没进包”和“资源在包里却没被正确解析”。

常见的检查项有：动态拼接的路径、大小写差异、插件挂载点、不同的构建配置。修复要针对实际原因，比如补上正确的资产管理规则、换成稳定的软引用，或消除对编辑器模块的依赖。

验证要在相同的目标配置下重新跑一遍相关路径，编辑器里再成功一次代替不了打包环境的验证。另外，资源加载失败时要有明确的返回值或占位表现，空对象要在入口处挡住，不让它流进玩法逻辑。[29][33]

第四册　系统、数学与物理

这一册把计算机基础、玩法数学和碰撞物理放在一起，因为面试官常把它们和具体玩法连起来问：扇形攻击怎么判定，子弹为什么穿墙，卡顿怎么证明修好了。回答时先把概念区分清楚，再落到一个玩法例子上。

系统与网络基础

04-01　进程、线程、任务有什么区别？为什么线程越多不一定越快?

进程提供地址空间和资源边界；线程是进程内的执行流，通常共享进程的内存；任务是对“一段工作”的描述，可以交给线程池调度，不需要一个任务对应一个新线程。进程隔离也不代表完全不能共享数据，共享内存等机制可以显式建立通信。

线程多了不一定快，原因有两层。并行只能加速可以并行的那部分；同时还要付出调度、同步、缓存和数据传递的成本。大量很小的任务，可能比一个顺序循环还慢。

放到 UE 里，玩法对象不能因为放进了任务就随意跨线程访问。常见做法是：在安全的阶段取出需要的数据，后台处理普通数据，再回到允许的线程上应用结果。[35][36]

04-02　虚拟内存、TLB Miss、Page Fault、Cache Miss 应怎样区分?

程序主要使用虚拟地址，由页表映射到物理内存，借此获得隔离、权限和共享映射等能力。栈和堆是虚拟地址空间里的组织方式，不是两块固定的物理内存；地址空间大，也不代表可以无限占用物理内存。

三种“未命中”发生在不同层次：

- **TLB Miss**：地址翻译的缓存没命中，需要继续查找映射；
- **Page Fault**：当前页表状态无法正常完成这次访问，可能触发按需分配、装入页面，或者是一次访问错误；
- **Cache Miss**：数据或指令不在当前这一级缓存里。

TLB Miss 不一定导致缺页，缺页也不一定要读磁盘。排查大规模内存访问慢时，要按层次分别定位，而不是统称为“CPU 缓存没命中”。[37][38]

04-03　TCP 与 UDP 的本质区别是什么？游戏为什么不必只选其中一种？

TCP 给应用提供可靠、有序的字节流，消息边界要应用自己划分；UDP 保留数据报边界，但本身不保证送达、有序或去重。

TCP 的有序性有代价：早期丢失的字节会卡住后面字节交付给应用，而实时状态的特点恰恰是“旧消息已经过期”，两者可能冲突。所以登录、资源下载和实时战斗可以用不同的协议。

选了 UDP，也仍然要按消息的语义处理可靠性、节流和拥塞，它不是“没有流量成本”或“永远更快”。把每种消息都强制可靠有序，可能让过期的移动状态挡住新状态；完全不可靠，又不适合关键的提交。另外，传输层怎么选，和玩法怎么预测、插值、裁决，是不同层次的问题。[41][42]

04-04　延迟、带宽、抖动、丢包与 Tick Rate 分别影响什么？

延迟是信息到达要花的时间；带宽限制单位时间能传多少；抖动是延迟和到达间隔的变化；丢包是数据没有到达。

它们的优化手段各有边界。压缩消息能降低带宽和排队压力，但消除不了物理传播时间；加缓冲能吸收抖动，却会增加显示延迟。服务器 Tick Rate 决定模拟推进的频率，它不是玩家的 RTT——即使服务器每秒更新很多次，一次输入仍然要经过采集、发送、网络传输、服务器处理和返回显示。

定位问题时，先分清是“每次操作都慢”、“偶尔突然一跳”，还是“数据持续积压”，再决定优化模拟、消息量、可靠队列还是客户端表现。不要指望调一个更新频率解决所有现象。[21][41]

玩法数学

04-05　点乘、叉乘怎样用于视野、扇形攻击和左右判断？

两个单位向量的点乘等于它们夹角的余弦。所以扇形判断可以这样写：先检查距离，再看前向向量与目标方向的点乘是否大于等于 cos(半角)。比如总角度 90 度的扇形，用 45 度半角，不需要对每个目标计算反余弦。目标和起点重合时，要先按玩法定义处理，避免对零向量做归一化。

叉乘能得到一个垂直方向；再拿它和选定的“上方向”做点乘，就能判断目标在左边还是右边。符号取决于坐标轴和运算顺序，最好拿一个明确的样例验证一下。

角度和距离只是候选筛选。目标有体积、有高度差，或者有墙体遮挡时，还要额外处理。二维地面扇形和三维圆锥也不是同一种区域，回答前先说明用的是哪种空间。[43]

04-06　位置、方向、法线为什么不能一律使用同一种变换?

三者对变换的反应不同：位置受平移、旋转和缩放影响；方向不应该被平移；法线还要保持和表面切线垂直，非均匀缩放时需要用逆转置等正确的变换。

坐标系转换也有固定规则。把世界坐标转到角色局部坐标，用的是该角色变换的逆；父子变换组合要遵守所用库的乘法约定，矩阵乘法不能交换顺序。从模型到屏幕的概念顺序是：模型空间 → 世界空间 → 观察空间 → 裁剪空间，然后是透视除法和视口变换。

Gameplay 里最常见的错误往往不是背不出这个顺序，而是把世界方向直接当成动画局部角度，或者把世界尺寸当成本地尺寸再乘一次缩放。另外，FTransform 不能精确表达任意带剪切的仿射变换，复杂的非均匀缩放层级要格外小心。[43][44]

04-07　欧拉角与四元数怎样选择?

欧拉角直观、便于编辑，但旋转顺序会影响结果，还有万向锁问题。四元数适合组合旋转和插值。注意 q 和 -q 表示同一个旋转，插值时要考虑走最短路径，并做归一化。

插值方式的选择：把三个角分别线性插值，不能保证在所有姿态下都得到自然的转向。Slerp 沿球面插值，在相应条件下角速度比较均匀；归一化线性插值计算更简单，小角度场景常常够用。

还有一个常被问的点：只有 Forward 向量不能唯一确定完整的旋转，因为绕 Forward 的滚转还是自由的，需要再给一个上方向之类的信息。

真正使用时，先确定旋转在哪个空间、转速是什么含义、是否允许跨越很大的角度，而不只是把变量类型改成四元数。[43][45]

碰撞、物理与 ISD

04-08　Trace、Sweep、Overlap，以及 AABB、OBB、包围球各有什么用途?

三种查询回答三个不同的问题：Trace 沿射线或线段查询；Sweep 让一个有体积的形状沿路径移动并查询；Overlap 检查某个姿态下的形状和哪些对象相交。形状、碰撞通道、是否使用复杂碰撞、是否忽略自身，都会改变结果。还要注意，Overlap 成立，不代表一定会产生阻挡或 Hit 事件。

三种包围体是精度和成本的取舍。AABB 与坐标轴对齐，测试便宜，但物体旋转后可能包得很松；OBB 跟着物体朝向，更贴合，但测试更复杂；包围球旋转不变，适合廉价的粗筛。常见做法是先粗筛候选，再做更精确的查询。

一个玩法例子：判断携带的物体能否沿某个方向移动，只检测终点有没有重叠，可能漏掉中途的薄墙，这时应该用 Sweep。[46]

04-09　高速子弹为什么会穿墙? CCD、Sweep 和物理子步是同一个东西吗?

如果只检测离散的位置，子弹一帧之内从墙前跳到了墙后，中间没有采样点，就会漏掉碰撞。

三种手段相关，但不是同一个开关：

- 沿上一位置到下一位置做 Trace 或 Sweep，可以检查整段路径；
- CCD 是物理系统处理高速碰撞的连续检测机制；

- 子步把较长的时间间隔拆小，提高轨迹和模拟的精度。

ProjectileMovement 通常用 Sweep 推进组件，所以不要机械照搬刚体 CCD 的配置经验。弯曲轨迹、初始穿透、移动目标和复杂碰撞仍要分别处理。提高 Tick 频率或放大碰撞体可能掩盖症状，但会增加成本或改变命中语义。

正确的顺序是：先确认是哪条移动路径、哪种碰撞体、哪个时间区间漏检了，再选方案。[46][23][47]

04-10 ProjectileMovementComponent 怎样更新投射物的位置和旋转?

它根据当前速度、加速度和时间步长计算运动，推动 UpdatedComponent，并按设置做碰撞查询。重力、追踪、反弹和停止条件会参与后续的速度和位置变化。开启“跟随速度旋转”时，朝向按速度方向更新，而不是由动画带动整个投射物。

要和物理刚体模拟区分开：如果组件被物理模拟接管了，就不能再假设这套手动位移仍然是唯一的控制源。子步能让重力或追踪下的弯曲轨迹更精细，但网络复制、预测匹配和伤害结算是另外的问题。

换句话说，一个投射物“在本地飞得对”，不代表它已经具备服务器权威、重生隔离或预测对账的能力。[23]

04-11 PhysicsHandle 拾取物体时，到底在控制什么?

PhysicsHandle 用类似物理约束的方式，把被抓住的刚体拉向目标位置或朝向；它不是每帧把整个 Actor 无条件传送到摄像机前面。目标的变化、刚体质量、阻尼、刚度和碰撞，都会影响跟随的手感。物体被抓住，也不代表世界里其他的物理约束消失了。

拾取前要先确定三件事：实际在模拟物理的是哪个组件、抓取点在哪、目标姿态是什么。携带过程中还要考虑物体和角色、墙体、其他物体的碰撞关系。

如果一边用 Handle 拉物体，一边又不断直接设置它的位置或速度，两套控制会互相打架。调手感时先理清控制源，再调参数，而不是一味把刚度加大到“看起来不抖”。[48]

04-12 为什么释放物体时，要把“已经放手”和“已经恢复碰撞”分开?

因为两件事的完成时间不同。放手意味着玩家不再持有物体，应该立刻解除抓取，并允许下一次拾取；恢复碰撞可能要等物体离开角色的范围。如果把两件事绑在同一个等待状态里，旧物体迟迟离不开时，玩家就无法继续交互。

ISD 的简历描述采用的正是这种分离：退出持有状态后，另外等待安全条件，再恢复原来的碰撞响应。

这个设计需要几处细节支撑。待恢复的记录要保存具体的组件、原始的碰撞响应和必要的持有身份；如果同一个物体又被拾起，旧的恢复操作不能把新持有关系下的碰撞提前打开；目标被销毁、角色更换或关卡结束时，也要结束对应的等待。

能确认的只是简历描述的主线，上面这些额外边界是否已经实现，以项目代码为准。[46][48]

04-13 Gloo 泡沫怎样决定附着、反弹或回收?

可以把一次碰撞命中当作输入，由材质和目标的接收规则决定结果。并不是每次命中都该附着：角色、特定表面、已有的泡沫，或者不支持附着的动态物体，都可能走不同的规则。这些规则最好集中在可查询的数据或接口里，不要在投射物里堆一长串具体 Actor 类型的判断。

附着时要明确用哪一份位置、法线和坐标空间，并且防止同一次碰撞的重复回调生成好几块泡沫。回收时要停止本次飞行、计时器和旧的命中状态，为下一次复用做好重置。

ISD 的简历说明了按材质和接收规则分类处理，但不能据此扩大成“已经支持任意动态刚体和所有复杂表面”。[46][23]

04-14　泡沫生长为什么不能只改显示缩放？

显示体积和碰撞体积必须按同一套规则生长。只放大 Mesh，看起来堵住了通道，实际却挡不住；不检查空间就放大碰撞体，又可能把角色或其他物体夹进去。

可行的做法是在每次生长前检查候选形状占用的空间，按规则停止、限制或拒绝生长。检查时要分清本地尺寸、世界缩放和最终碰撞尺寸，避免重复乘缩放；而且“形状变大”不能简单等同于“把一个固定形状从 A 点 Sweep 到 B 点”。多个泡沫同时生长时，还要明确计算和提交的顺序。

WorldSubsystem 可以统一管理本世界里的生长工作，但它解决的是调度和寿命组织，不会自动消除所有几何冲突。[46]

04-15　对象池什么时候值得使用？Actor 复用需要重置什么？

当频繁创建和销毁带来明显的分配、注册或初始化成本时，对象池可以复用对象或存储。但它不会消除 Tick、碰撞、渲染和网络的成本，还可能因为长期保留峰值容量而多占内存。所以要先确认热点在哪，再决定池化哪一层。

不同层次要重置的东西不同。普通 C++ 存储池要处理对齐、构造与析构、重复归还和容量不足；Actor 复用还要重置速度、碰撞、可见性、计时器、事件订阅、命中状态和业务身份。旧的回调不能作用到已经被重新租出去的对象上。

网络 Actor 要特别小心：“销毁后再生成”和“重复使用同一个网络身份”在语义上不一样，不能把单机的对象池原样搬进复制系统。[4][23][39]

性能与渲染常识

04-16　一帧卡顿应该怎样定位？怎样证明优化确实有效？

先看卡顿那一帧的时间线，分清是游戏线程、渲染线程、GPU、资源加载、GC 还是线程等待，再找出这一帧比正常帧多做了什么。Unreal Insights 能把 CPU 活动和事件对应起来；GPU 问题还需要相应的 GPU 计时或图形调试工具。平均 FPS 很高，也可能夹着明显的长帧。

证明优化有效，比做优化更容易出错。对比前后结果时，尽量固定平台、构建、地图、负载和资源的冷热状态，记录帧时间分布和玩家能看到的现象。某个函数变快了，却把工作挪到了另一帧；或者降低了更新频率，却让交互延迟明显变大——这些情况都要如实报告，不能只报一个漂亮的数字。

结论要写清楚测了什么、改了什么。没有运行数据时，只能说“预期收益”，不能写成“已经提升了多少”。[49]

04-17　CPU 到屏幕的渲染主线是什么?

主线是：游戏逻辑推进世界状态 → 渲染侧据此维护场景和绘制数据 → RHI 和图形 API 组织命令 → GPU 完成顶点处理、光栅化、着色和其他计算 → 最终提交显示。阴影、深度和后处理可能各自包含多个 Pass，所以一个模型不一定只画一次。

游戏线程、渲染线程、RHI 线程和 GPU 可以重叠工作，不同平台或配置的线程安排也不一定完全相同。因此总帧时间并不等于所有统计项之和。

对 Gameplay 程序来说，首先要知道自己增加的对象、材质、骨骼或查询，把负担放在了哪一层。关键是把瓶颈定位到真正耗时的阶段，而不是凭某个指标的变化推断整体性能。[50]

04-18　Draw Call 减少一半，为什么帧率没有提高?

因为瓶颈可能根本不在 Draw Call。减少 Draw Call 主要降低的是绘制组织、状态切换和提交的成本；如果瓶颈在像素着色、半透明过度绘制、带宽、阴影或其他 CPU 工作上，帧率就未必有明显变化。一个盖住大半个屏幕的复杂半透明效果，即使批次不多，也可能很贵。

所以要比较游戏线程、渲染侧和 GPU 的实际耗时，再看优化有没有作用在当前瓶颈上。合批和实例化本身也有代价，可能带来更粗的剔除粒度、额外的内存或资源管理上的限制。

一个诚实的结论可以是：“提交成本下降了，但当前仍然受 GPU 像素负载限制。”不要为了证明成功只挑某个局部数字。[50][49]

04-19　UI 失效缓存、合批与 Retainer 分别优化什么?

三者针对不同的成本：失效缓存减少不变的控件反复布局、反复生成绘制数据的 CPU 工作；合批减少兼容绘制状态的提交次数，但受材质、纹理、裁剪和层级等条件限制；Retainer 把一棵子树绘制到纹理上，可以控制刷新频率，代价是额外的纹理，还可能增加延迟。它们不是可以互相替代的开关。

评价效果要看对应的指标。布局耗时降了、批次数没变，也可能是成功的优化；反过来，把频繁变化的 UI 强行缓存起来，可能不停失效，收益有限。

顺序上，先用事件更新替代无意义的轮询，避免反复重建整棵控件树，再根据实际瓶颈选择缓存、列表虚拟化或批次优化。[13]

第五册　C++ 与算法

C++ 题表面上问语法，很多其实在问同一件事：对象由谁拥有、能活多久、什么操作会让它或它的地址失效。带着这个问题读这一册，很多看似零散的知识点会连成一张网。

对象、资源与所有权

05-01　指针和引用有什么区别？非空指针为什么仍可能不能访问?

语法上的区别是：指针可以为空、可以重新指向别的对象；引用初始化后不能重新绑定，常用来表达“调用方必须提供”的借用。但两者都不管理被指向对象的寿命，引用同样可能悬空——比如返回局部对象的引用，或者引用的容器元素已经被销毁。

三种“坏指针”要分清：空指针明确表示没有目标；没有正确初始化的指针常被叫作野指针；悬空指针曾经指向合法对象，但对象的寿命或地址稳定性已经结束了。

所以检查 `p != nullptr` 只能排除空指针。释放对象后把某一个变量置空，也修复不了其他仍保存着旧地址的别名。真正要说清的是：对象由谁拥有，借用能持续多久，期间哪些操作会让地址失效。[27][31]

05-02　const T、*T* const、const T* const 分别限制什么?

读法是看 const 修饰谁：`const T*` 限制通过这个指针修改目标，但指针本身可以改指向；`T* const` 限制指针改指向，但目标仍可修改；`const T* const` 两者都限制。

const 成员函数限制通过 this 修改非 mutable 成员，但不会让成员指针所指向的外部对象也变成只读。

const 表达的是访问约束，不等于对象绝对不变，更不等于线程安全：一个 const 接口如果读取的是其他线程正在无同步修改的数据，照样有数据竞争。const_cast 只能改变访问类型，不能让本来就定义为 const 的对象被合法修改。

设计参数时，要把“是否拥有、是否可空、是否允许修改”放在一起考虑，而不是机械地加 const。[51]

05-03　RAII 到底解决什么问题？没有异常也有用吗?

RAII 把“释放资源”的责任绑到对象的寿命上：资源管理对象离开作用域时执行析构，于是正常返回、提前返回和异常展开三种路径的清理被统一了。这里的资源不只是堆内存，还包括锁、文件、句柄，甚至临时状态的恢复。

没有启用异常时 RAII 同样有用，因为提前返回和多分支依然存在。例如作用域锁能避免某个 return 忘记解锁；状态守卫可以在查询结束时恢复临时修改的碰撞配置。

它的边界也要清楚。RAII 不会自动撤销已经发出的消息、已经提交的奖励或已经写入的磁盘数据，业务回滚要另外设计。析构函数也不适合承担可能失败、又需要向调用方反馈结果的主要业务提交。[31]

05-04　new/delete、operator new 与 malloc/free 有什么区别?

普通 new 表达式通常先取得存储、再构造对象；delete 表达式先执行析构、再按分配协议释放存储。operator new 只是其中的分配入口，本身不等于完整的 new 表达式。malloc/free 只管理原始字节，不会替类执行构造和析构。

分配和释放必须配对：new[] 得到的数组要用 delete[] 释放，new 创建的对象不能交给 free。失败时的表现也不同：普通 new 分配失败会抛异常，nothrow 版本和 malloc 则各自以返回值表示失败。

实际工程中，优先让容器和智能指针管理这些配对。UE 的 UObject 另有自己的创建和回收协议，不能用普通的 new/delete 管理。[52]

05-05　unique_ptr 怎样表达所有权？函数参数应该怎样写?

unique_ptr 表达独占所有权：不能复制，但可以移动。

参数写法跟着语义走。按值接收 unique_ptr，通常表示“我要接管所有权”；如果只是临时使用对象，传 `T&` 或允许为空的 `T*` 更直接，不要把借用接口写成拥有接口。返回 unique_ptr 能清楚地表示“我创建了对象并把它交给你”，一般不用担心因此复制整个资源。自定义删除器还能封装特殊的释放协议。

两个容易混的操作：release 返回裸指针并放弃管理，不会删除目标；reset 更换或清空当前管理的对象，并按协议释放旧目标。不要为了让编译通过就随手 release，否则资源责任又会变得模糊。[31][53]

05-06　shared_ptr 的控制块、weak_ptr 与循环引用是什么关系？

shared_ptr 的控制块里记录强引用数、弱引用数和删除方式等信息。最后一个强引用离开时，目标被销毁；只要还有弱引用，控制块就继续保留，用来判断目标是否已经结束。weak_ptr 不拥有目标，使用时通过 lock 尝试拿到一个临时的强引用。

循环引用就出在这里：两个对象用 shared_ptr 互相拥有，外部引用都离开后，它们的强计数仍然不为零，于是泄漏。解决办法是先明确拥有方向，把观察关系或回指关系改成弱引用，而不是随便挑一边改类型。

make_shared 常把控制块和对象放在一次分配里，减少分配次数；代价是只要还有弱引用，这块合并的存储就要延后释放。所以它不是对所有对象都毫无代价的最优选择。[54]

05-07　shared_ptr 是线程安全的吗？

要拆成三层来答。

- **控制块**：多个 shared_ptr 实例共同管理一个控制块时，引用计数这类共享管理行为有标准规定的并发保证。
- **同一个 shared_ptr 变量**：多个线程任意读写同一个变量，没有保证。对它的发布和替换需要同步。
- **被管理的对象**：它的字段不会自动受到保护。

例如两个线程各拿一份 shared_ptr，目标不会因为其中一份被释放而立刻消失；但如果两个线程同时修改目标里的 vector，仍然会发生数据竞争。被管理对象的内部状态需要自己的同步策略。

weak_ptr::lock 解决的是“尝试拿到所有权”，它不负责判断对象当前的业务状态是否允许修改。[54][36]

05-08　深拷贝与浅拷贝有什么区别？拷贝、移动和析构应该怎样一起设计？

浅拷贝只复制成员本身的表示；如果成员是裸资源句柄或指针，两个对象就会指向同一份资源，析构时重复释放。深拷贝为新对象建立一份独立的资源副本，但它的成本和语义必须由这个类型自己明确规定。

几个特殊成员各有分工：拷贝构造用同类型对象初始化一个新对象；拷贝赋值替换一个已经存在的对象；移动构造和移动赋值把可转移的资源交给目标，并让源对象保持可析构、且满足其契约的状态。

设计上优先遵循零法则：让 string、vector、unique_ptr 这类成员去管理资源，自定义类就可以直接用编译器生成的、正确的特殊成员函数。如果类直接拥有裸资源，就要把析构、拷贝构造、拷贝赋值、移动构造、移动赋值五个放在一起考虑，并明确它是深拷贝、共享还是禁止复制。

还有一个细节：用户声明析构函数等特殊成员，会影响隐式移动操作的生成，所以“只加一个空析构”也可能改变它在容器里的行为。[31][55]

05-09　std::move 是不是把对象移动走了？

不是。std::move 只是改变表达式的值类别，让它能参与移动重载的选择；它本身不搬运资源，也不销毁对象。到底发生什么，由被调用的那个构造函数或赋值运算符决定。

所以有几种“写了 move 却没移动”的情况：类型没有可用的移动操作时，调用仍可能走复制；对 const 对象用 std::move，通常绑定不到常见的 `T&&` 移动构造（那会丢掉 const），于是可能调用接受 `const T&` 的复制构造。

移动之后源对象依然存在。标准库通常要求它处于“有效但值未指定”的状态，自定义类型则要遵守自己的契约。所以不能一律断言被移动的 string 一定为空，也不能因为写了 move 就认为旧对象已经析构了。[56]

类、继承与模板

05-10　虚函数怎样实现运行时多态？虚表是 C++ 标准规定的吗？

语言保证的是行为：通过合适的基类指针或引用进行虚调用时，会选中动态类型对应的最终覆盖函数。主流 ABI 通常用“对象里的虚指针 + 类对应的虚表”实现这个间接调用，但表的结构、指针数目和布局并不是语言标准统一规定的。

对比来看，非虚函数的调用一般按静态类型和名字查找等规则确定；而且一旦把对象按值复制成基类对象，派生部分的状态就丢了。

虚函数的性能要放到实际调用模式里讨论，间接跳转、内联机会和缓存都可能有关，但不能仅凭“有虚表”就断定整个系统性能差。接口清楚、扩展有限时，普通组合或模板也可能更合适。[57]

05-11　基类为什么常需要虚析构？构造和析构中能调用派生虚函数吗？

如果允许通过基类指针删除派生对象，通常就需要虚析构，这样删除才能走完从派生类到基类的整条销毁链。如果设计上就禁止这种删除方式，可以用受保护的非虚析构来表达，不必要求所有类都无条件加虚函数。

构造和析构期间的虚调用是另一回事。构造基类时，派生部分还没建好；析构基类时，派生部分已经结束了。这时的虚调用按当前正在构造或析构的那一层处理，不会像完整的派生对象那样派发。所以需要派生类多态行为的初始化，应该等对象完整建立后再显式进行。构造函数本身不能是虚函数；也不要在构造或析构中依赖纯虚行为完成关键业务。[57][58]

顺带记住构造顺序：完整对象通常按“虚基类 → 直接基类 → 成员（按声明顺序）→ 当前类构造函数体”构造，析构顺序相反。成员初始化列表的书写顺序改变不了成员的实际初始化顺序，所以应该按声明顺序书写，免得读者误判依赖关系。[57][58]

05-12　重载、覆盖、隐藏和对象切片有什么区别？

- **重载**：在同一个名字查找范围里，根据参数选择不同的函数。
- **覆盖**：派生类的函数匹配并替代基类虚函数的实现。override 能让编译器检查是否真的覆盖成功。
- **隐藏**：派生类的同名声明遮住了基类的整组同名函数。`using Base::Func` 可以把被隐藏的基类重载重新引入。
- **对象切片**：派生对象按值复制或转换成基类对象时，结果只剩基类部分。

切片的典型例子是 `vector<Base>` 接收一个 Derived 值，它并不会保存完整的派生对象；需要多态集合时，要用合适的指针或拥有型智能指针。同理，把参数从 `Base&` 改成 `Base`，不只改变了复制和寿命，也可能直接改变多态语义。[57][55]

05-13　sizeof 为什么不等于各成员大小相加?

对象的大小包括非静态成员、基类子对象、对齐填充，以及实现需要的额外状态（比如多态带来的部分）。静态成员不属于每个实例。成员的排列顺序、对齐要求、是否有多态，都会影响结果。数组里的每个元素都要按类型大小保持正确对齐，所以尾部填充也有意义。

几个特殊情况：完整的空对象大小不是零，因为它通常也需要一个可区分的地址，但空基类有专门的布局优化；多继承可能带来多个基类子对象和指针调整；虚继承是为了共享同一个虚基类子对象，和“让函数变成虚函数”不是一回事。

具体数值要在给定的平台、ABI、编译器和类型定义下计算，不能把某个 64 位平台的指针大小推广到所有系统。为了减少填充调整成员顺序可能有用，但还要兼顾接口兼容、访问局部性和同步，不能只追求 sizeof 最小。[59]

布局规则再补充几句：结构体成员按各自的对齐要求放置，成员之间和对象尾部都可能有填充，以保证数组中的下一个元素仍然对齐。union 的非静态成员共享同一段存储，大小通常由最大成员和整体对齐决定。`#pragma pack` 会降低实现使用的打包上限，`alignas` 可以提高或明确对齐；它们都会影响布局、ABI 和访问成本，不要为了省几个字节随意跨模块使用。[59]

05-14　static_cast、dynamic_cast、const_cast、reinterpret_cast 怎样区分?

- **static_cast**：用于规则允许的显式转换，但大多数向下转换不会替你做运行时类型检查。
- **dynamic_cast**：在适用的多态类型关系里做运行时检查；指针转换失败返回空，引用转换失败会抛出异常。
- **const_cast**：修改 cv 限定，也就是访问类型。
- **reinterpret_cast**：做较低层的重新解释，不能让任意一段存储合法地变成另一种对象。

四者都修复不了已经结束的对象寿命。

在 UE 里，UObject 体系通常用 `Cast<T>`、IsA 这类基于反射的类型功能；不能未经确认就假定 UE 模块开启了全部 C++ RTTI 和异常配置。

选转换方式之前，先问“这次转换为什么合理”，而不是一个个试到不报错为止。[60][2]

05-15　static 在函数、命名空间和类中分别意味着什么?

同一个关键字，三个位置三种意思：

- **函数内的 static 变量**：具有静态存储期，通常在第一次执行到它的初始化声明时初始化。
- **命名空间中的 static**：赋予内部链接，让名字只在当前翻译单元可见。
- **类的 static 成员**：属于类本身，而不是每个对象各存一份；static 成员函数没有 this。

C++11 起，局部静态变量的初始化有线程安全保证，但初始化完成之后的读写并不会自动同步。

把可变状态放进局部 static，也会延长它的寿命、制造共享状态。在多个 PIE World 或多场测试中，不要把普通的进程级静态变量误当成“每局一份”的游戏状态。[61]

05-16　Lambda 按值捕获为什么仍可能出现悬空或泄漏?

按值捕获只是复制被捕获表达式的值：复制一个裸指针，复制的仍然只是地址；捕获 this，也不会延长对象的寿命。按引用捕获则要求外部变量在回调执行时还活着。一旦把 Lambda 存进异步任务、Timer 或事件列表，这些前提很容易失效。

泄漏的方向正好相反：捕获 shared_ptr 能延长目标寿命，但如果目标又保存着这个回调，就形成了强引用环。观察关系可以改用弱引用，在回调执行时再尝试取得临时访问权。对 UObject，要使用符合它 GC 协议的弱引用。

即使安全地访问到了对象，也还要确认回调仍属于当前请求——内存安全不等于业务结果正确。[63][54]

05-17　模板为什么常把定义放在头文件？模板和运行时多态有什么区别?

隐式实例化模板时，使用具体模板实参的那个翻译单元必须能看到足够的定义，所以模板实现通常放在头文件里。如果用到的类型集合是固定的，也可以在 cpp 里定义并显式实例化，并不是绝对不能分开。

模板和虚函数是两种多态：模板根据编译期的类型生成对应的代码；虚函数在运行时根据对象的动态类型选择行为。模板可能带来内联和静态检查的优势，也可能增加编译时间和代码体积；虚接口方便在运行时替换实现，但要考虑对象寿命和间接调用的成本。

两者可以结合使用。“模板快、虚函数慢”不能当成不看场景的设计结论。[64]

05-18　完美转发解决什么问题？T&& 都是转发引用吗?

完美转发解决的是“把参数原本是左值还是右值的性质，原样传给下一层”。

在适用的函数模板推导中，`T&&` 可以成为转发引用，实参是左值还是右值会体现在推导结果里。但具名参数在函数体内一律是左值表达式，所以要用 `std::forward<T>` 按原始的推导结果转交。std::move 则是无条件地把表达式当作可移动对象，两者语义不同。

不是所有 `T&&` 都是转发引用：固定类型的 `Widget&&` 不是，`const T&&` 也不走同样的推导规则。引用折叠中，只要组合里出现左值引用，通常折叠成左值引用；只有右值引用和右值引用组合，才仍然是右值引用。

完美转发只负责传递参数性质：它不获得所有权，不延长临时对象的寿命，也不能让保存外部引用的代码自动变安全。[56]

容器与复杂度

05-19　vector、list、deque 应该怎样选择?

vector 连续存储，随机访问快、局部性好，通常是默认的顺序容器。list 用独立节点存储，已知节点位置时插入删除方便，其他节点的地址通常保持稳定，但查找、分配和指针跳转都有成本。deque 分段存储，兼顾随机访问和两端增删，但没有 vector 那种整段的连续性。

注意“list 插入是 O(1)”这句话不包括先找到位置的成本，也说明不了 CPU 缓存表现。需要稳定句柄时，可以用间接索引或独立拥有的对象，不一定要把所有数据换成链表。

选择时同时看访问顺序、增删位置、地址稳定性需求和数据规模。元素很大时，也可以让 vector 存拥有型指针，而不是直接搬动大块内容。[39][65]

05-20　size、capacity、reserve、resize 分别是什么?

size 是当前已经存在的元素个数；capacity 是不重新分配内存时最多能容纳的元素个数。reserve 只预留容量，不增加有效元素；resize 改变元素个数，可能构造或销毁元素。所以 `reserve(100)` 之后 size 仍然是零，这时直接写 `v[0]` 不是合法的添加元素方式。

扩容可能会取得新存储并搬迁元素，因此旧的指针、引用和迭代器都可能失效。clear 销毁元素，但通常保留容量；shrink_to_fit 也不保证立刻收回全部容量。已知规模时预留容量可以减少扩容，但不能用 reserve 掩盖“在任意修改中长期保存元素地址”这种设计错误。[39]

几个相关操作也常一起问。push_back 接收一个已经构造好的元素，再复制或移动进容器；emplace_back 把参数转发给元素的构造函数，直接在容器尾部构造，但并不保证在所有调用中都比 push_back 更快或更清楚。中间插入会移动或复制后续元素，并让插入位置及之后的迭代器、引用失效。扩容时，如果元素的移动操作可能抛异常而复制可用，容器可能选择复制来维持异常保证；正确标注 noexcept 的移动操作有助于走移动路径。[39][55]

05-21　map、unordered_map 与红黑树、哈希表是什么关系?

map 按比较关系保持键有序，常见实现是红黑树之类的平衡搜索树，查找和常见插入是对数复杂度。标准规定的是行为和复杂度，并不强制所有实现用同一种树。红黑树用颜色约束限制路径长度，修改之后通过重新着色和旋转维持平衡。

unordered_map 根据哈希值和相等关系把元素组织进桶里，查找平均接近常数时间，但最坏情况可能退化成线性。自定义键必须保证“相等的键得到相同的哈希”，比较关系也要满足容器的要求。负载因子、分配、桶数量和访问局部性都会影响性能。哈希冲突可以用链式组织或开放寻址等方式处理，具体容器的实现要另外查证。

怎么选：需要范围查询、有序遍历或稳定的最坏复杂度时，有序容器可能更合适；大量平均查找的场景可以考虑哈希表。不要因为听说是 O(1)，就忽略哈希冲突和 rehash 的成本。[66][67]

05-22　迭代器、指针和引用什么时候失效?

规则取决于容器和具体操作，没有一句话的通用答案。

- **vector**：重新分配会让所有元素地址和迭代器失效；即使不重新分配，插入或删除也可能让相关位置及之后的引用和迭代器失效。
- **list**：删除一个节点，通常只让和这个节点相关的引用失效。
- **unordered_map**：rehash 会让迭代器失效，但不会因此让元素的引用和指针一律失效；删除元素则另当别论。

不要把“迭代器失效”等同于“所有对象都被销毁了”。遍历时删除元素，要使用 erase 返回的新迭代器。

遇到扩容后崩溃，优先找出缓存了元素地址的地方，以及第一次继续使用旧地址的位置，而不是在崩溃点加一个非空检查。[39][66][65]

05-23　时间复杂度、均摊复杂度和实际运行时间有什么区别?

时间复杂度描述的是输入规模增长时运算量的增长趋势，不直接等于毫秒数。均摊复杂度把一串操作的总成本平摊到每次操作上：vector 尾部插入偶尔要扩容，但一系列追加的平均摊销成本可以是常数——这并不代表某一次追加不会很慢。

比较算法时，还要看空间、缓存、分配和输入分布。哈希的平均常数查找和树的对数查找各有前提；“先排序再查询”和“每次逐个扫描”哪个好，还取决于查询次数和数据更新频率。

最坏情况、平均情况和均摊分析不是一回事，回答前先说清楚在讨论哪一种。[39][68][69]

05-24　快速排序、归并排序、堆排序各有什么特点?

- **快速排序**：经典实现通过划分递归处理区间，平均 O(n log n)，划分极端时会退化到 O(n²)；选主元的策略能降低常见的退化风险。
- **归并排序**：合并已排序的区间，通常是 O(n log n)，容易做到稳定，但数组版本通常需要额外的缓冲区。
- **堆排序**：最坏情况也是 O(n log n)，额外空间小，但局部性和常数因子不一定好。

稳定性指的是相等键之间原有的相对顺序是否保留。

std::sort 不能简单等同于最朴素的快速排序：标准只规定它的复杂度等要求，库实现可以用混合算法。写代码时优先正确使用库算法，并保证比较器满足严格弱序——把比较器写成“小于等于”可能破坏算法的前提。[68][69]

05-25　二分查找怎样避免边界错误?

先定义区间和不变量，而不是背一个 mid 的模板。下面这段返回第一个不小于 target 的位置，使用左闭右开区间 `[left, right)`，结果允许等于数组长度。如果还要判断目标是否存在，再检查返回位置是否越界、该位置的元素是否等于 target。

```
#include <cstddef>
#include <vector>

std::size_t LowerBound(const std::vector<int>& a, int target) {
    std::size_t left = 0, right = a.size();
    while (left < right) {
        const auto mid = left + (right - left) / 2;
        if (a[mid] < target) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

为什么它是对的：循环每一轮都至少排除一个不可能的位置，空数组也能正确返回零。前提是数组按相应关系有序。修改边界规则时，要同步修改不变量，不要把闭区间和半开区间的写法混在一起。[68][69]

05-26　BFS、DFS 与最短路有什么关系?

BFS 按层扩展。在每条边代价相同的无权图里，第一次到达某个节点时的层数，就是到它的最少边数。通常在入队时就标记已访问，避免多个前驱把同一个节点反复入队。

DFS 沿一条路径往深处走，适合遍历、判断连通性或回溯，但它第一次找到的路径不保证最短。

普通 BFS 只适用于边代价相同的图；带权图里，边权非负时可以考虑 Dijkstra，用优先队列维护候选距离。图里有环时要有访问规则或距离更新规则；在树上递归也要考虑深度过大造成的栈风险。说复杂度是 O(V+E) 时，要讲清楚前提是用邻接表，以及每个节点、每条边各被访问几次。[68][69]

05-27　堆、Top-K 与最小堆为什么经常一起出现?

堆只保证根是当前的最值，不保证整个容器有序。

维护最大的 K 个元素时，可以保留一个大小不超过 K 的最小堆：堆顶是已保留元素里最小的那个，新元素比它大时才替换。总时间是 O(n log K)，空间是 O(K)。

但不要见到“前几个”就上堆。只找一个最近的目标，线性扫描往往更简单；所有结果都要有序时，完整排序可能更合适。目标还带“可见”之类的条件时，按距离取 K 个只是一种候选策略，不能自动保证拿到最近的那个可见目标。

数据结构解决的是“怎样维护候选”，结果对不对，还取决于筛选条件和什么时候停止。[68][69]

05-28　为什么含负数的“和为 K 的子数组”不能随意用滑动窗口?

滑动窗口依赖一个前提：窗口边界移动时，和的变化方向是可以推理的。数组里有负数时，扩大窗口可能让和变小，缩小窗口也可能让和变大，所以“和大了就左移”这条规则不能保证不漏解。

换个思路用前缀和。设 prefix 是到当前位置的前缀和，“某一段子数组的和为 K”就等价于“之前出现过前缀和 prefix − K”。用哈希表记录每个前缀和出现的次数，每一步先查询、再记录当前前缀；一开始先记录一次零，才能覆盖从数组开头开始的子数组。下面的代码假定输入规模和数值使得前缀和、prefix − k 以及答案都能用 long long 表示。

```
#include <unordered_map>
#include <vector>

long long CountSubarrays(const std::vector<int>& a, long long k) {
    std::unordered_map<long long, long long> count{{0, 1}};
    long long prefix = 0, answer = 0;
    for (int value : a) {
        prefix += value;
        const auto it = count.find(prefix - k);
        if (it != count.end()) answer += it->second;
        ++count[prefix];
    }
    return answer;
}
```

[66][69]

05-29　动态规划和回溯应该先想什么，而不是先写递归？

动态规划先想清楚“一个状态代表什么”，再说明它依赖哪些更小的状态、初始值是什么、按什么顺序计算。比如 `dp[i]` 表示到达位置 i 的方案数，那么转移就必须完整覆盖“最后一步”的所有合法来源，并避免重复计数。压缩空间之前，要确认下一次转移需要的数据仍然保留着。

回溯是逐步做选择、再撤销选择，枚举搜索树中的候选解。剪枝必须能证明被跳过的分支不可能产生需要的答案，不能因为某个样例上“好像没用”就把它剪掉。

两者都要处理空输入、不可达状态、重复元素和数值溢出。能讲清状态的含义和转移为什么正确，比记住二维数组或 DFS 的代码外形重要得多。[68][69]

并发与错误定位

05-30　mutex 应该保护什么？死锁的四个必要条件是什么，怎样预防、避免、检测和恢复？

死锁的四个经典必要条件是：互斥、占有且等待、不可抢占、循环等待，四者同时成立才会死锁。

对应的四种对策：

- **预防**：在设计上破坏其中一个条件，比如统一加锁顺序来破坏循环等待；
- **避免**：运行时只做能保持安全状态的资源分配；
- **检测**：建立等待关系图并找环；
- **恢复**：取消工作、回滚、抢占资源或终止进程。

游戏运行时通常更应该在预防上下功夫：统一锁顺序、缩小临界区、不在持锁时调用未知的回调、使用 scoped_lock 等工具。超时只能暴露问题或提供降级路径，不能自动恢复数据一致性。[70][73]

回到 mutex 应该保护什么：锁保护的是“需要保持一致的一组状态和操作”，而不是机械地给每个变量配一把锁。例如“检查库存还剩一个，然后扣掉一个”这两步，必须遵守同一个同步协议。临界区里尽量避免慢 I/O、未知回调和不受控的再次加锁。统一锁顺序、缩小持锁范围、使用 scoped_lock，通常比换成递归锁更有意义。[31][70]

05-31　数据竞争、业务竞态与 atomic 是什么关系？volatile 能同步吗？

数据竞争是 C++ 内存模型里的概念：不同线程对同一内存位置有冲突访问、至少一方在写，又缺少必要的同步，就会导致未定义行为。业务竞态的范围更大：即使每个字段都用了 atomic，多个步骤交错执行，仍然可能破坏“只能领取一次”这样的规则。

atomic 提供原子操作和内存顺序工具，但它不会自动把多个变量变成一个事务，也不保证在所有实现上都是无锁的。用一个原子标志发布数据时，还要考虑其他数据的可见性和同步关系。volatile 不是线程同步机制，替代不了锁或原子协议。

举个发布数据的例子：生产者先写普通数据，再对原子变量 ready 做 release 存储；消费者用 acquire 读到这次发布的值之后，再去读数据。这一对 release/acquire 建立的同步关系，保证了之前写入的数据对消费者可见。这个例子仍然要求没有其他未经同步的修改者；把 acquire/release 随便加在不相关的标志上，并不会建立需要的先行发生关系。[36][71]

05-32　join 与 detach 有什么区别？为什么线程已经结束，thread 仍可能 joinable?

join 等待关联的线程结束，并完成相应的同步；detach 解除 std::thread 对象和执行线程之间的关联，后台的执行可以继续存在。

线程函数已经返回，并不会让 std::thread 对象自动变成不可 join，它仍然需要被 join 或 detach。一个仍可 join 的 std::thread 被直接析构，会调用终止处理。

detach 有两个风险：它不会延长线程所引用的外部对象的寿命；也让关闭顺序更难管理。比较清楚的退出流程是：先停止接收新工作，通知等待中的线程退出，再等待工作线程结束，最后销毁它们用到的数据。较新的标准设施能简化部分生命周期管理，但取消请求仍然是合作式的协议，不会强行、安全地中断任意正在执行的代码。[72]

05-33　条件变量为什么必须围绕谓词等待?

因为通知不是持久的状态：通知可能发生在开始等待之前，也可能出现虚假唤醒。

所以真正的条件要保存在受互斥量保护的共享状态里，等待方每次被唤醒后都重新检查谓词，比如“队列非空，或者系统正在关闭”。生产者在同步协议内先修改状态、再发通知；消费者只在条件确实满足时才取任务。关闭队列时，要唤醒所有可能在等待的线程，并规定剩下的任务是处理完还是丢弃。

如果只写一次 wait，醒来就假定队列里一定有元素，那么虚假唤醒、或者元素被其他消费者抢先取走，都会变成越界访问或永久等待。[73]

05-34　未定义行为、内存泄漏和释放后使用应该怎样定位?

越界、释放后使用、数据竞争、有符号整数溢出等都可能触发未定义行为。某次 Debug 构建没有崩溃，不能证明代码正确：优化、内存布局或线程调度一变，问题可能在完全不同的地方暴露出来。最终崩溃的位置，也常常只是更早一次内存破坏的受害者。

不同问题查的方向不同：释放后使用，要查第一次释放的地方和最后仍保留借用的地方；泄漏，要查资源的拥有链为什么没有结束；内存持续增长，还要区分是缓存还是真正的泄漏。可用的工具包括符号、崩溃转储、数据断点，以及在适用平台上使用 AddressSanitizer 等。

只在 Shipping 版本出现的问题，要保留对应的构建和符号，尽量在接近的优化条件下复现，不能根据调试版的结果直接排除语言层面的错误。[27][36][74]

第六册　Gameplay Ability System（GAS）

GAS 的概念很多，但主线只有一条：谁拥有能力（ASC），能力怎样开始和结束（Ability 与 Task），数值怎样被改变（Effect 与 Attribute），状态怎样被表达（Tag），表现怎样跟上（Cue）。预测和重生是在这条主线上叠加的两个难点。

能力、属性与状态

06-01　GAS 解决什么问题？什么情况下不必使用 GAS?

GAS 把能力执行、属性变化、状态标签、效果持续时间和一部分网络预测，组织进同一套协议里。技能越多、效果越容易叠加，又需要联机、打断、消耗和冷却时，这种统一的价值就越大。

它不只是一个“技能基类”，也不会替项目决定所有战斗规则。引入 GAS 意味着要理解 ASC、Ability、Effect、AttributeSet 之间的关系，并承担初始化和网络配置的成本。所以只有几个简单动作的单机原型，完全可以先用组件和明确的状态逻辑来做。

选不选 GAS，理由应该来自玩法的复杂度和后续的变化，而不是“用了 GAS 就一定比普通组件高级”。[75]

06-02　ASC、OwnerActor、AvatarActor 与 AttributeSet 是什么关系?

ASC（能力系统组件）持有已授予的能力、正在生效的效果和标签等运行状态，并和 AttributeSet 配合处理属性。OwnerActor 是能力系统的逻辑拥有者；AvatarActor 是当前执行能力、表现动作的那具身体。两者可以是同一个 Actor，也可以分开。

AS 的简历做法是：PlayerState 持有 ASC 和 AttributeSet，Character 作为 Avatar，这样能力系统可以在更换身体后继续存在。这是项目的选择，不是 GAS 唯一的标准架构。

这种架构的代价在身体更换时出现：要刷新 ActorInfo，并重新连接依赖当前 Character 的动画、装备和事件。仅仅让 ASC 活得更久，并不会让旧引用自动失效。[75][76]

06-03　Ability 类、AbilitySpec 与 Ability 实例有什么区别？授予等于激活吗?

三者是三个层次。Ability 类定义能力的逻辑和配置。授予到 ASC 上的 AbilitySpec 记录这项能力在这个拥有者身上的等级、来源、输入和运行关联信息，并通过句柄来识别。Ability 实例是否创建、怎样复用，由实例化策略决定。所以类默认对象、Spec 和某一次执行，不能当成同一个对象。

授予也不等于激活。授予表示“有资格使用”；激活还要经过标签、冷却、消耗、网络策略等检查。

实例复用会带来状态残留问题：按拥有者复用的实例，不能把上一次执行的临时目标或委托留到下一次；不实例化的能力，更不能把每个玩家的可变状态写进共享对象。移除能力时，也要考虑它是不是还在执行中。[77]

06-04　CanActivateAbility、ActivateAbility、CommitAbility 与 EndAbility 各负责什么?

四个函数对应一次能力执行的四个节点：CanActivateAbility 判断现在有没有资格开始；ActivateAbility 建立本次执行并启动任务；CommitAbility 尝试提交配置好的消耗和冷却；EndAbility 结束这次执行。

几个容易犯的错：以为检查通过之后会一直满足条件；Commit 已经失败，却仍按“已经支付”继续往下走。

什么时候 Commit 是玩法决定的：起手就扣蓝，还是成功发射才扣蓝，结果不一样。关键是让一次业务提交只有一个清楚的入口，避免输入、动画通知和任务回调各扣一次。

另外，EndAbility 不会替项目代码恢复所有副作用。自己注册的外部委托、临时状态和资源，仍要按各自的生命周期清理。[77]

06-05　GameplayTag 怎样表达状态？死亡、眩晕与技能互斥应该怎样组织？

GameplayTag 是层次化的标识，可以描述状态、能力类别、事件和匹配条件。匹配时可能包含父子关系，所以精确匹配和层次匹配不能混用。多个来源同时提供同一个状态时，还要理解标签的计数和来源：随便移除一次，不代表这个状态已经完全结束。

死亡或眩晕可以阻止新能力激活，并按规则取消正在执行的能力。但“加一个死亡标签”不会自动实现整套死亡流程：伤害、死亡判定、能力取消、停止移动和安排重生，仍然要由服务器决定。

还要把“持久拥有的状态”和“当前身体的临时状态”分开，避免旧身体上的限制被带到新身体上。[75][77]

06-06　BaseValue 与 CurrentValue 有什么区别？属性复制为什么不能只刷新 UI？

BaseValue 是属性的基础值；CurrentValue 是叠加了当前所有有效修饰之后的结果。例如基础速度 600，一个持续的减速效果让当前速度变成 420；效果移除后，聚合结果重新计算。不要把两者机械地理解成“初始值”和“最新值”，基础值本身也会被合法修改。

属性复制不只是为了显示数字，还可能影响 GAS 的聚合和预测清理。使用 FGameplayAttributeData 的常见复制写法，会在 RepNotify 里通过 `GAMEPLAYATTRIBUTE_REPNOTIFY` 通知能力系统，并按预测的需要配置通知条件。

如果只给血条赋值、绕过了 GAS 的通知链，界面可能看起来是对的，内部的预测或属性依赖却没有正确更新。[76][20]

06-07　Instant、Duration、Infinite 效果怎样区别？周期效果又是什么？

按持续时间分三类：Instant 效果执行一次属性修改，正常的权威应用之后，不会作为持续效果留在活动容器里；Duration 在限定时间内有效；Infinite 一直有效，直到被移除。后两类可以携带持续的修饰、状态标签等信息。

效果定义主要是配置；GameplayEffectSpec 才包含这一次应用的等级、上下文和动态参数。

周期是另一个维度：持续效果或无限效果可以每隔一段时间执行一次修改，例如每秒扣血。所以不能说所有 Duration 效果都只改变 CurrentValue——周期执行可以修改基础值。另外，预测路径对瞬时效果有专门的临时处理，也不能用“Instant 从不进入任何活动记录”概括所有客户端内部行为。[78]

06-08　Modifier、MMC 与 ExecutionCalculation 应怎样选择？

按计算的职责来选：

- 简单的加减乘除等属性变化，用 Modifier；
- 需要根据捕获的属性、标签或参数算出一个修饰幅度时，用 MMC（修饰幅度计算）；
- 需要一次综合运算、输出多个修饰结果时，用 ExecutionCalculation，比如根据攻击、护甲和命中类型计算伤害。

复杂的计算类不一定更好，选择依据是计算职责和预测要求。GAS 的常规客户端预测支持属性修饰，但不能把服务器上的 ExecutionCalculation 原样当成已经支持预测的路径。

做伤害计算时，还要把输入、计算结果和最终的生命值提交分开，避免表现层的通知再修改同一份属性。[78][79]

06-09　属性捕获的 Snapshot 在什么时候固定？为什么来源和目标不能一概而论？

一条捕获定义同时说明三件事：捕获哪个属性、来自 Source 还是 Target、是否使用快照。在常见的 GAS 流程里，来源的快照值在 EffectSpec 创建时捕获，目标的快照值在 Spec 应用到目标时捕获。所以“所有快照都在 Spec 创建时固定”这种说法不准确。

用投射物来理解：射手开火之后才获得增伤，会不会影响已经飞出去的那一发，要看 Spec 什么时候创建、来源属性什么时候捕获；目标在被命中之前获得了护甲，则取决于应用时目标的状态。

不使用快照的捕获，会保留和相关属性聚合的联系，在真正计算时才求值，但这并不等于为任意计算自动建立了每帧更新。最终采用哪个时间点，仍然是玩法规则，要在计算设计里写清楚。[79][80]

06-10　SetByCaller、EffectContext 与伤害参数怎样分工？

SetByCaller 给这一次的 EffectSpec 提供动态数值，比如已经算好的伤害幅度；EffectContext 携带施加者、来源或命中上下文等信息。前者不是替代所有属性捕获的万能字典，后者也不是可以无限塞进各种可变对象的全局状态。同一个参数要有统一的键和单位，并处理值缺失的情况。

信任边界要守住：客户端发来“这次应该造成 500 点伤害”，服务器不能直接写进 SetByCaller，而应该根据合法的武器、等级、命中和效果规则生成或约束这个数值。

需要跨网络传递的自定义上下文还涉及序列化。不能因为在 C++ 里多加了一个字段，就假定另一端已经收到了。[78][79]

06-11　AbilityTask 为什么比普通延迟回调更适合能力？取消时仍要处理什么？

AbilityTask 把等待输入、等待动画、等待事件这类异步工作接进了能力的生命周期，并通过委托返回结果。能力结束时，任务体系能参与清理。

但“参与清理”有范围：任意的外部订阅、手动创建的对象、自定义的后台任务，不会因此自动结束；任务自己的销毁逻辑也必须正确解除监听。

要特别分清“动画播完”和“能力结束”是两件事。成功、失败、取消、打断、Avatar 更换，每一种情况都要走到正确的终止路径，并且避免重复调用 EndAbility。旧回调即使最后到达了，也要先确认它仍属于当前这次执行。如果只在“正常播放完成”里做清理，被打断的技能就会一直占着状态，甚至阻止下一次激活。[81]

06-12　TargetData 传输成功，为什么仍不能直接相信其中的命中？

TargetData 是传递目标信息的结构，可以包含命中结果、位置或 Actor 等内容。它解决的是“一次能力执行怎样携带和传递目标数据”，既不保证数据符合玩法规则，也证明不了数据来自真实的玩家输入。

所以服务器仍要核对适用的条件：执行身份、距离、遮挡、阵营、资源和时间等。客户端为了反馈及时，可以先做瞄准或命中表现；但权威伤害只采用服务器认可的结果。收到之后，还要防止同一份数据被重复消费，并限制数量和大小。

验证方案要和武器的语义一致：即时射线、飞行弹体和范围技能，不能共用一个“目标不为空就通过”的检查。[82][3]

预测、表现与角色重生

06-13　LocalPredicted Ability 与 PredictionKey 能自动预测一切吗?

不能。LocalPredicted 允许拥有者客户端先执行受支持的能力路径，减少等服务器带来的操作延迟；服务器仍然独立检查资格，并产生权威结果。PredictionKey 把这一次预测和它受支持的副作用关联起来，让系统可以确认、拒绝，或者在权威状态到达后清理预测结果。

它不是通用的事务编号，不会自动回滚任意的 Actor 生成、物理运动、背包修改或自定义伤害。预测窗口也有作用域：经过计时器或等待之后，最初的那个键可能已经不能再用了。

所以 AS 的预测弹体需要自己的身份匹配和清理逻辑，不能因为能力配置成了 LocalPredicted，就省掉这一层。[20]

06-14　GameplayCue 怎样与 Ability、GameplayEffect 分工？四种回调是什么意思?

三者分工是：Ability 组织动作的过程，GameplayEffect 表达属性或状态的变化，GameplayCue 主要负责声音、粒子和材质等表现。Cue 不应该成为伤害或比分的唯一真值，否则相关性、预测和晚加入的差异，都可能影响业务结果。

四种回调：

- **OnExecute**：一次执行型的表现；
- **OnActive**：激活那一刻的表现；
- **WhileActive**：建立持续生效期间的表现，它不是“每帧 Tick”；
- **OnRemove**：移除时的表现。

晚加入或重新相关的客户端，不一定经历过最初激活的那一瞬间，所以持续效果不能只靠起手特效来建立。预测和服务器确认同时存在时，要用对应的 Cue 路径和预测身份避免重复，并保证重复建立或移除时不会留下多份特效。[78][83]

06-15　输入已经触发，能力却没有反应，应该从哪条链排查?

沿着激活链一步一步往下查：

1. 输入是否送到了当前 Pawn 和正确的 ASC；
2. 目标能力是否已经授予，对应的 Spec 能否找到，输入标识是否匹配；
3. ActorInfo、拥有连接、网络执行策略是否正确，以及是否因为标签、消耗或冷却导致激活失败；
4. 只有确认 ActivateAbility 已经进入，才把注意力转到任务或 Montage 上。

重生后才出现的失败，尤其要核对新 Avatar 的初始化和输入重绑，不能因为看到 PlayerState 上的 ASC 还在，就认定所有依赖都正常。

记录这次失败停在哪一步，比到处调用 TryActivate 有效得多。把所有检查都绕开，也许能让动作播出来，但同时也绕过了资源和服务器资格的校验。[77][8]

06-16　ASC 跨重生保留时，哪些状态应保留，哪些必须重新建立？

保留还是清除，由状态的含义和持有者的寿命决定。角色成长、长期能力可以跨身体保留；当前的攻击任务、旧 Mesh 上的动画、身体专属的效果、装备对象的引用和身体事件的订阅，通常要结束或重新连接。长期减益在死亡后是否还存在，也是玩法规则，GAS 不会给出统一答案。

新身体接入后，更新 Owner/Avatar 信息，重新绑定身体相关的依赖，按需要初始化属性。两个方向都要防：不要无条件再授予一遍同一批能力，造成重复的 Spec；也不要直接清空全部能力和效果，误删本该保留的状态。

AS 的 PlayerState ASC 架构，最需要讲清楚的就是这些保留、清理和重建的边界。[76][77]

06-17　预测技能看起来扣了两次蓝，怎样区分正常预测与真正重复消耗？

先搞清楚“两次”发生在哪里。拥有者客户端先预测一次消耗，服务器再执行一次权威消耗，这两次分别发生在两端，并不意味着同一份权威蓝量被扣了两次。

排查时分别查看服务器属性、客户端预测属性和 UI 事件，确认问题出在真实数值上，还是只是表现重复。真正的问题可能是：同一端重复 Commit、任务回调和手动扣除叠加、同一次执行触发了两次，或者客户端没有通过预测键和属性通知正确清理本地副作用。对齐一次激活的身份、提交次数和属性变化，再检查 `GAMEPLAYATTRIBUTE_REPNOTIFY` 等接入。

如果只因为看到两端都打印了“扣蓝”，就删掉客户端预测或服务器提交，会把原本合理的流程破坏掉。[20][76]

06-18　可预测的蓄力技能怎样处理开始、释放和取消？

按下时，客户端可以立刻开始一段可撤销的蓄力表现，同时请求服务器认可这次激活；释放时，提交本次执行的目标信息和必要的蓄力参数。服务器限制时长、射击频率和资源，自己计算合法的结果，不能直接相信客户端发来的最终倍率。

计时要有明确的时间基准，两台机器的本地时钟不能直接相减。松开、取消、死亡和切换装备，都要结束相关的任务和表现。如果释放阶段还要继续预测，就使用一个合适的新预测窗口，而不是假定激活时的窗口一直开着。

至于权威弹体和本地预测弹体怎样对应、确认迟迟不到时怎样超时清理，属于项目自己的弹体协议，蓄力计时解决不了。[20][82]

第七册　角色动画

动画系统的基本方向是：玩法状态在前，动画姿态在后。动画读取玩法给出的事实（速度、朝向、动作状态），把它变成骨骼姿态；它可以提供时间点，但不应该反过来成为生命值、弹药这类状态的真值。

从玩法状态到动画姿态

07-01　AnimBlueprint、AnimInstance 与 SkeletalMeshComponent 分别是什么?

AnimBlueprint 定义动画逻辑，编译后生成一个动画实例类；SkeletalMeshComponent 持有网格，并用对应的 AnimInstance 驱动姿态。两个角色用同一份动画蓝图，并不意味着共享同一个可以随意修改的动画实例。动画资产、动画类和运行时实例要分开理解。

更换 Mesh 或动画类时，实例可能被重新创建。外部缓存的 AnimInstance 指针、Montage 监听，以及和旧实例绑定的状态，都不能继续无条件使用。

职责方向上，玩法组件通常提供速度、姿态或动作状态，动画实例负责消费这些信息；不要让动画蓝图反过来成为生命值、装备和权限的唯一来源。[84]

07-02　动画更新与姿态求值有什么区别？动画里能随便访问玩法对象吗?

更新阶段推进动画的状态、时间和混合参数；求值阶段根据当前状态生成骨骼姿态，再经过后续处理得到最终结果。它们并不总是按最直观的“游戏 Tick 之后立刻逐节点执行”的方式运行，并行动画和更新优化都会改变具体的执行安排。

所以动画里不能随便访问玩法对象。可能在工作线程上执行的逻辑，不能任意读写 Gameplay 的 UObject，也不能依赖“访问时机恰好安全”。通常的做法是在允许的阶段取出角色状态，把需要的数据交给动画侧，并使用官方支持的线程安全访问方式。

把大量世界查询、目标搜索或伤害提交放进姿态求值里，既混淆了职责，也增加了线程和性能风险。[84][85]

07-03　移动动画为什么通常读取速度，而不是直接读取按键?

按键表示操作意图，速度才是角色实际运动的结果。角色可能被墙挡住、被击退、站在移动平台上；远端角色更是根本没有本机按键可读。只看前进键，就会出现撞墙时仍在播放奔跑，或者远端角色明明在动却一直站着。

具体的动画参数还要再细分：世界速度和角色局部速度、地面和空中、速度大小和运动方向。对网络角色，要确认读到的是适合表现的移动事实，不要把网络胶囊的校正和 Mesh 平滑又混合成同一个输入。

输入意图也不是完全没用，它仍可以服务于起步、身体倾斜这类辅助表现，只是不能替代所有实际运动状态。[84][9]

07-04　状态机、BlendSpace 与 Montage 分别解决什么问题?

三者解决三类不同的问题：状态机按条件组织站立、移动、空中这些持续状态之间的转换；BlendSpace 根据速度、方向等参数，在样本动画之间插值；Montage 用于比较离散的动作播放、分段和通知，比如攻击或换弹。

它们通常配合使用，不是三选一的架构。例如移动状态输出一个 BlendSpace，攻击 Montage 再通过 Slot 进入动画图，覆盖部分骨骼。

容易出问题的是“动作状态由谁决定”。玩法系统和动画状态机之间要有明确的单向关系。如果两边都各自判断“已经进入攻击”，却没有同步结束和打断，就会出现动画已经结束、角色却还不能移动的情况。[86][87]

07-05　Montage、Slot 与骨骼分层怎样配合？为什么开枪后下半身不动？

Montage 通过匹配的 Slot 节点进入动画图。但 Slot 的输出最终能不能体现在姿态上，还取决于图中的连接和混合权重。

想边移动边开枪，可以保留移动姿态作为基础，再用按骨骼分层的混合覆盖上半身，而不是无条件用全身动作替换整个姿态。

下半身不动时，按这几项查：Slot 放在图中的哪个位置；分层混合从哪根骨骼开始、覆盖多深；基础姿态是否还在更新；这个 Montage 本身是不是全身动作。只调混合时长，修不好错误的骨骼范围。角色转向和武器瞄准也可能需要不同的骨骼层，不要把所有上半身动作堆在同一层，再指望权重碰巧合适。[87][88]

07-06　Montage 返回播放成功，但角色看不到动作，应该检查什么？

“播放成功”只说明调用被接受了，按这个顺序查后面的环节：

1. 调用是否落在当前角色、当前 Mesh 和当前的 AnimInstance 上，而不是旧身体，或者只在另一端执行了；
2. 资产和骨架是否兼容，动画图里有没有正确的 Slot，Slot 是否连到了最终输出；
3. 是否被其他状态、其他 Montage 或低权重混合盖住了。

网络场景还要区分“服务器触发了能力”和“每个客户端都已经有对应的动画表现”。普通的本地播放调用不会自动传到所有机器上，应该由项目采用的能力或状态同步方案来驱动。

如果角色位置移动正常、只有姿态不对，再去查动画输入和实例，不要同时盲改网络平滑和动画图。[87][84]

动作时序、瞄准与位移

07-07　AnimNotify 与 NotifyState 应怎样使用？伤害和换弹能只靠通知提交吗？

AnimNotify 标记时间线上的一个点；NotifyState 表示一段区间。它们适合脚步声、拖尾、碰撞窗口或动作时点这类与动画时间对齐的事情。

但通知的执行受播放端、动画更新、权重和打断等条件影响，“动画里有这个通知”不等于服务器一定会、而且只会收到一次合法的事件。所以伤害、弹药和技能提交要由权威的玩法规则来保护：通知可以触发请求或开启一个窗口，但处理时要核对当前动作，以及这件事是不是已经提交过。需要强同步时，要选择合适的通知类型和播放策略。

换句话说，屏幕上看到换弹动画完整播完了，不能证明服务器那边的背包已经可靠更新。[89]

07-08　连击、换弹和动作打断怎样避免旧回调修改新动作？

每个动作都要有明确的开始、可接续窗口、提交点和结束条件。正常完成、被替换、主动取消、死亡、换装备，每一种结束方式都必须释放这个动作持有的限制和监听。

对可能晚到的回调，要核对它是否仍属于当前这次执行，而不只是检查角色指针有效。例如旧的换弹被打断，新的攻击已经开始，这时旧回调不能再无条件地把角色状态设回空闲，也不能再补一次弹。区分手段可以是本次动作的标识、任务句柄或能力的执行关系；简单的单动作场景，不必为此搭一个庞大的通用框架。

抓住这一点：旧动作只能结束自己负责的状态，不能清掉后来那个动作的结果。[87][89]

07-09　Additive、AimOffset 与坐标空间有什么关系？

Additive 动画表达的是相对于某个参考姿态的增量，所以必须使用正确的参考姿态，以及正确的局部空间或网格空间设置。AimOffset 根据瞄准角度混合这些增量姿态，让基础的站立或移动姿态继续保留，上半身朝向目标。

瞄准输入不能直接把世界 yaw 当成角色的局部偏角。通常先算出视角相对于角色朝向的角度，处理好角度环绕和合理范围，再送进动画。角色整体转向和上半身瞄准之间，同一份旋转只能施加一次。

手臂反扭、或者跨过正负 180 度时突然跳变，往往应该先检查空间和角度的定义，而不是盲目调平滑。[90]

07-10　IK 解决什么问题？脚底贴地或扶枪为什么会抖动？

反向运动学（IK）根据末端的目标位置，反过来求出各关节的姿态，常用于脚底贴地、扶枪或接触类动作。它修正的是姿态，不会决定角色的胶囊应该移到哪里。目标位置和法线要先从世界空间转换到求解器要求的空间，还要和它在动画图中所处的阶段一致。

抖动的常见来源有四个：目标检测本身有噪声；空间混用；骨骼更新前后的时机不同；把 IK 之后的结果又当成下一次的目标，形成了反馈。可以适当平滑并设置接触状态，但坐标错了要修坐标，平滑掩盖不了。

网络角色还要区分权威的移动和本地的姿态修正，避免脚部表现反过来驱动物理位置。[91]

07-11　Root Motion 怎样影响角色移动？为什么客户端播放后又被拉回？

Root Motion 从动画里提取根部的运动，在配置允许时交给角色的移动链使用；它不是让 Mesh 的根骨绕过碰撞把角色拖走。对 Character 来说，这份位移是由移动组件、胶囊碰撞、动画提取模式和当前移动模式共同处理的。

被拉回的原因通常是两端不一致：网络下只在客户端播放了带根运动的动作，服务器并不知道这个动作或对应的运动规则，模拟就会分歧，随之产生位置校正。所以要使用支持网络的根运动路径，并确认两端对动作身份、播放和取消的认知一致。

还要避免双重移动：一边让根运动提供位移，一边又按同样的轨迹手动 SetActorLocation。[92][9]

07-12　Motion Warping 适合什么场景？目标更新后为什么动作会偏？

Motion Warping 在指定的时间窗口内调整适用的根运动，让动作更贴近目标位置或朝向，适合翻越、处决、贴近交互点这类场景。它依赖正确的根运动、窗口配置和目标；目标是否合法、碰撞、导航和多人裁决，它都不负责。

动作偏的原因，多半在目标上：目标过期了、空间搞错了，或者在动作期间随意跳变。所以要规定目标什么时候锁定、什么时候允许更新、失效时怎样取消。

它适合在已经明确的动作范围内修正轨迹，不适合把任意距离的动画强行拉到目标上，更不能借它绕过服务器对动作条件的验证。[93]

07-13　大量角色动画开销高时，怎样优化而不破坏玩法?

先把开销拆开：动画更新、姿态求值、骨骼数量、蒙皮、场景查询。再根据可见性、距离和重要程度，选择降低更新频率、简化动画图、减少骨骼，或者使用合适的共享和预算机制。不是每种优化都同时降低 CPU 和 GPU 的成本。

优化不能破坏玩法，关键在于：被降频或跳过更新的角色仍可能参与战斗，所以关键的命中、冷却和死亡不能只依赖每帧的动画通知。角色重新可见、或者恢复高频更新时，还要正确接回当前的玩法状态。

验收看最坏帧和行为一致性，而不只是平均帧率升了多少。[85]

第八册　Gameplay AI

AI 的执行链可以简化成三段：感知收集事实，决策（行为树或 StateTree）选择行为，任务驱动移动和攻击。常见的 AI bug，往往是某一段停住了，或者旧的任务还在跑。按段排查，比同时改三处有效。

决策、感知与任务

08-01　AIController、Pawn、BrainComponent 和行为资产怎样分工?

Pawn 提供身体、移动和表现；AIController 组织控制和目标决策；BrainComponent 这一类运行组件执行项目采用的决策逻辑。行为树或 StateTree 资产描述行为的组织方式，但这不代表每个 AI 的全部可变运行数据都该写进共享资产。

生成了 Pawn，不代表 AI 已经被控制、已经启动决策、或者有了可用的导航。初始化时要确认控制关系、决策组件和身体能力都已经接上。

多人游戏里，关键的 AI 决策通常由服务器负责，客户端只消费移动和表现；不要让每个客户端各自裁决同一个敌人的目标和伤害。[94][95]

08-02　行为树中的 Selector、Sequence、Decorator、Service 与 Observer Aborts 怎样执行? StateTree 又有什么不同?

先记住两个组合节点：Sequence 从左到右执行子节点，遇到失败就失败，全部成功才成功；Selector 从左到右尝试，遇到成功就成功，全部失败才失败。

另外三个机制：Decorator 约束一个节点能否进入或继续；Service 在分支活动期间周期性地更新信息；Observer Aborts 决定条件变化时，是否中止当前分支或更低优先级的分支，让 AI 能及时从巡逻切到追击。行为树并不是每帧把所有子节点无条件执行一遍。

中止有一个常被忽略的边界：它只结束树里的任务状态，任务自己发起的 MoveTo、Montage、Timer 和外部委托，仍然要正确取消。共享节点和实例内存也要按节点的策略保存运行数据。[95][97]

StateTree 更显式地用层次状态、转换和任务组合，表达“现在处于什么状态、在什么事件下转移”。两者都要处理异步任务结束、状态失效和外部工作清理。不能因为 StateTree 比较新，就无条件重写一棵已经稳定的行为树。[95][96]

08-03　Blackboard 为什么不是全局变量仓库？里面适合保存什么？

Blackboard 存放的是行为决策的工作数据，比如当前目标、最后已知位置、巡逻点和状态条件。每个键都应该有明确的类型、写入者和失效条件；多个任务随意修改同一个键，行为切换就会依赖偶然的执行顺序。

一个关键区分：目标 Actor 仍然有效，不代表现在还看得见、仍是敌人、或仍属于这次追击。所以“目标对象”和“最后感知到的事实”通常要分开存。

清空目标后，依赖它的任务要能被中止或重新评估，不能只清了 Blackboard，却让旧的移动请求继续执行。持久的战斗规则、所有玩家共享的数据，也不应该为了取用方便全塞进 Blackboard。[95]

08-04　异步行为任务怎样处理成功、失败和中止？

发起移动或等待动画之后，任务通常处于“进行中”状态，等结果回来再结束，而不是一启动就报告成功。

中止时，任务必须取消或停止自己发起的工作、解除回调，并规定晚到的结果要不要忽略。结果里要保留足以区分“是哪一次请求”的身份。某些行为树 C++ 节点使用共享模板，运行状态要按节点的实例化策略或节点内存保存，不能默认一个节点对象只服务一个 AI。旧的追击回调到达时，只能处理它自己那次请求，后来开始的新任务不受影响。

简单的实现用请求 ID 加当前任务检查就够了，不必引入和现有玩法规模无关的通用“租约”系统。[95][97]

08-05　AI 感知到目标，就应该永远追踪目标当前坐标吗？

不应该。感知提供的是带时间和来源的事实，例如看见了、听见了、丢失了刺激；行为系统根据这些事实形成当前判断。看到过目标，不代表失去视线后还能合法地读取它的最新位置，否则“记忆”就退化成了全知追踪。

通常要区分三样东西：当前能感知到的目标、最后已知位置、记忆的失效时间。丢失目标后，可以去最后位置搜索，或者保持短暂的警觉，再按规则回到巡逻。刺激过期、Actor 销毁、阵营改变、目标死亡，都要触发判断更新。

给不给 AI 额外信息是玩法设计问题，但要明确设计出来，而不是因为手里还攥着一个指针就意外地“知道”。[98]

08-06　EQS 的 Generator、Context、Test 和 Score 怎样协作？查询结果晚到或需要限制楼层时怎么办？

Generator 产生候选的位置或 Actor；Context 提供查询的参照，比如查询者、目标和已知位置；Test 对候选进行过滤或打分，最后按配置选出结果。EQS 给出的是“候选的好坏”，至于要不要攻击或移动，由行为树决定。

限制楼层有两类做法：在生成阶段只产生对应导航区域的候选；或者用高度、路径可达性、区域标记和楼层 ID 来过滤。昂贵的 Test 要放在便宜的粗筛之后。

异步结果到达时，还要核对目标和任务是否仍然有效，过期的查询结果直接丢弃。[99][100]

移动、攻击与故障定位

08-07　A*、Dijkstra、NavMesh、PathFollowing 和 MoveTo 分别负责什么？寻路成功为什么仍可能走不到？

这几个名词分属三层：算法、导航数据、实际运动。

- **算法**：Dijkstra 按当前已知的最小代价扩展，适用于非负边权；A* 在此基础上用一个不高估剩余代价的启发函数引导搜索。
- **导航数据**：NavMesh 把可行走区域表示成可查询的导航结构，路径规划在上面寻找走廊或路径。
- **运动**：PathFollowing 让 Pawn 沿结果前进，并处理到达、阻塞和重新规划；MoveTo 是发起这一整套流程的高层请求。

路径规划成功，只说明在查询条件下找到了一条候选路线，并不保证动态障碍、拥挤、角色碰撞、移动模式和目标变化都不会挡住执行。[94][100]

走不到时还要检查：导航代理的尺寸、终点的投射、到达半径，以及 Pawn 原点和脚底位置的差别。只放大接受半径，可能让任务提前报告成功，但角色其实仍然进不了交互距离。[94][100]

08-08　目标移动或新障碍出现后，为什么不能每帧无条件重发 MoveTo？

因为每个移动请求都有自己的路径和完成状态。每帧重新发起，会反复中止旧请求、重复寻路，让回调和行为状态变得混乱。

更好的做法是区分三种情况：目标位置只是小幅变化、路径已经失效、真的需要重新规划，并使用所选移动接口支持的目标跟踪和路径更新机制。目标已经不可达时也不要无限重试，可以退回搜索、换一个替代位置，或者冷却一段时间再试。

停止请求时，确认停的是当前任务负责的那次移动，不能让旧任务的清理把新任务刚发起的移动一起取消。记录请求身份和失败原因，比单看角色有没有迈腿更能定位问题。[94]

08-09　AI 攻击前需要检查什么？Focus 设置正确为什么仍然打空？

攻击资格通常包括：目标存在且可以被攻击、阵营、距离、视线、资源、冷却和当前的动作状态。

Focus 主要影响朝向控制。它不会自动把目标拉进射程，也不保证武器的射线不被遮挡。控制器朝向、身体朝向、动画瞄准和武器实际的发射方向，仍然可能各不相同。近战可以先满足攻击位置和朝向，再进入动作窗口；远程攻击要按武器模型计算瞄准和发射。

从开始动作到提交命中之间，关键条件可能已经变了，要按玩法规定重新验证其中哪些条件。不能因为目标进了攻击半径就持续扣血，也不能把“动画看上去朝着敌人”当成弹道正确的证明。[94]

08-10　AI 死亡、眩晕或切换目标时，哪些工作必须结束？

要结束的不只是行为树里的某个节点，还包括当前的路径跟随、攻击能力、计时器、动画等待和相关的外部订阅。停止决策不等于停止了已经发出的移动；清空目标，也不等于取消了已经开始的攻击。

结束的原则有三条：按每项工作的拥有关系去停止；允许它们多次收到终止信号，而不会破坏之后的状态；回调到达时核对当前执行——死亡前的旧攻击结果，必须拦在重生后的身体之外。

同时也要控制范围：不要顺手销毁和这个 Pawn 无关的共享系统，或清空其他任务的新状态。[95][97]

08-11　AI 站着不动，应该怎样按执行链缩小问题？

沿执行链从上往下查：

1. 服务器上是否有正确的 Controller 和被控制的 Pawn，决策系统是否启动了；
2. 当前选中了哪条行为分支、为什么选它，目标和 Blackboard 里的值是否符合条件；
3. 移动请求是否真的发出，导航有没有找到路径，路径跟随处于什么状态；
4. 移动组件有没有被碰撞、移动模式或速度限制住。

现象本身能帮你定位：角色位置在变、模型却站着，是动画输入的问题；行为显示正在追击却没有移动请求，是任务逻辑的问题；请求发出但失败了，就继续查导航和终点。

按这些边界观察，就不需要同时重写行为树、导航参数和动画图。使用行为树调试、AI 调试信息和日志时，注意确认看的是当前这个 AI，以及正确的执行端。[94][95]

08-12　大量 NPC 应怎样分配更新成本？怎样避免所有工作集中到同一帧？

先把成本拆成决策、感知、寻路、碰撞、动画和渲染几类，再按距离、可见性和玩法重要性分级：近处正在战斗的单位高频处理关键行动；远处单位降低感知和决策频率；部分静止单位可以暂缓无效工作。寻路和空间查询尤其要控制候选规模和请求频率。

避免集中在同一帧的办法是错开起始相位，让各个计时器不要同时到点。

降频也有底线：死亡、受击、目标失效等必要事件仍要及时响应，从低频恢复时积压的工作也要限量。线程化只能转移或并行一部分成本，不能消除总工作量；对象池同样消除不了仍在执行的 AI、碰撞和动画。[100][49]

第九册　高频基础补强

这一册补的是 C++、操作系统、计算机网络、设计模式和渲染里最常被单独问到的基础题。它们和游戏开发的关系不在于背定义，而在于能不能说清楚“这条规则在引擎里会以什么形式出现”。

C++ 高频入口

09-01　C++ 程序的代码段、只读数据段、全局/静态区、堆和栈分别保存什么？对象究竟“在哪”由什么决定？

常见的进程映像会区分几块区域：可执行代码、只读常量、已初始化和未初始化的全局/静态数据，以及运行时使用的堆和栈。这种划分是实现和平台层面的常见组织方式，C++ 标准并没有强制所有系统使用相同的段名和地址布局。

对象“在哪”，主要由它的存储期和创建方式决定：局部自动对象通常使用当前线程的栈式存储；动态分配的对象使用分配器取得的存储；全局和 static 对象具有静态存储期。对象内部再持有的资源，又可能位于别的地方。

UObject 还要遵守 UE 自己的创建和 GC 协议，不能简单归类成“new 出来就在普通 C++ 堆上，然后手动 delete”。[27][37][61]

09-02　class、struct、union、多继承、虚继承与菱形继承分别有哪些语义和布局影响？

class 和 struct 在语言上的主要差别，只是默认的访问权限和默认的继承权限不同；两者都能定义成员函数、构造函数，也都能继承。

union 让多个成员共享同一段存储，需要明确当前哪个成员是活动的。把任意字节重新解释成另一个成员，不能就此假定那个对象的生命周期是合法的。

多继承会在一个完整对象里形成多个基类子对象，基类指针之间的转换可能需要调整地址。菱形继承如果不用虚继承，共同基类会出现两份；虚继承可以让它们共享同一个虚基类子对象，代价是布局和访问机制变得更复杂。

面试回答时，最好先说明是否真的需要“is-a”关系。很多 Gameplay 的变化点，用组合和接口表达会更清楚。[57][59]

09-03　inline、宏、const 与 constexpr 分别解决什么问题？inline 不是强制内联时还意味着什么？

宏在预处理阶段做文本替换，不理解作用域和类型，参数还可能被重复求值；普通函数、模板和 constexpr 通常更安全。

const 表达“通过当前这个接口不会修改对象”的约束；constexpr 表达“这个对象或函数可以参与常量求值”，但到底是不是在编译期求值，仍取决于使用的上下文。

inline 不保证编译器一定把函数展开。它更重要的含义是：允许满足条件的同一个实体，在多个翻译单元里都有定义，所以常用于写在头文件里的函数或变量。至于是否真的内联，属于编译器的优化决策。

不要用宏或 inline 去掩盖模块边界和编译依赖的问题。[51][64]

操作系统与并发

09-04　进程间通信有哪些方式？管道、共享内存、消息队列和 Socket 各自适合什么场景？

- **管道**：适合顺序的字节流，用于有亲缘关系或已经明确连接的进程之间；
- **消息队列**：保留消息边界，方便异步投递；
- **共享内存**：避免重复拷贝，适合大量数据，但同步、生命周期和一致性必须另外设计；
- **Socket**：既能用于本机进程之间，也能扩展到网络通信。

选择时看数据量、是否需要消息边界、是否跨机器、是否需要持久排队、延迟要求和安全边界。

共享内存“快”不等于容易写对，仍然要用锁、信号量、事件或无锁协议协调访问。RPC 则是建在这些传输机制之上的一层，提供函数级的接口和数据编组。[101]

09-05　进程/线程上下文切换大致保存什么？调度、用户态与内核态为什么会带来成本？

上下文切换要保存和恢复足以让执行继续下去的处理器状态，比如程序计数器、栈指针、寄存器和调度状态；进程切换还可能涉及地址空间相关的状态。

真正的成本不只是保存几个寄存器，还来自缓存、TLB 和分支预测状态被打乱。线程发起系统调用、等待 I/O 或同步原语时，可能从用户态进入内核态，并触发调度。

所以更多线程只有在工作可以并行、而且粒度足够大时才可能提速。大量短任务、频繁唤醒和共享数据竞争，会让调度和同步的成本超过收益。[35][37][38]

09-06　协程、线程和任务有什么区别？为什么协程不等于并行?

线程是操作系统调度的执行流；任务是一段可被调度的工作描述；协程允许函数保存自己的执行状态并主动挂起，之后在调度器的控制下恢复。多个协程完全可以跑在同一个线程上，所以协程只提供并发的组织方式，不一定提供 CPU 并行。

协程特别适合把“等下一帧、等资源、等网络结果”写成一段连续的流程。但等待期间用到的对象仍可能失效，取消和关闭也仍然需要合作式的协议。

UE 的 Timer、AbilityTask、Async Task 和 Lua coroutine，生命周期和线程模型各不相同。yield 只是让出执行权，并不会“自动切到后台线程”。[15][35][72]

计算机网络与游戏传输

09-07　TCP 为什么需要三次握手和四次挥手？TIME_WAIT 与 CLOSE_WAIT 分别说明什么?

三次握手让双方确认彼此的发送和接收能力，并同步各自的初始序号。只交换两次的话，其中一方无法确认自己的应答已经被对方收到。

关闭时，两个方向的字节流可以各自独立结束：一方发出 FIN，对方的应用发完数据后再发自己的 FIN，于是通常形成四个报文阶段。

两个状态说明的是不同的事。主动关闭的一方通常进入 TIME_WAIT，用来处理迟到的报文，并保证最后一个 ACK 丢了还能重传；被动关闭的一方收到 FIN 后进入 CLOSE_WAIT，等待本地应用真正关闭连接。所以大量 CLOSE_WAIT 往往说明应用没有及时释放连接；大量 TIME_WAIT 则要结合“谁在主动关闭”和连接复用策略来分析，不能一律当成泄漏。[41]

09-08　TCP 如何通过序号、ACK、超时重传和滑动窗口提供可靠有序字节流?

TCP 给字节流编号，接收方用 ACK 确认已经连续收到的位置；发送方保留未被确认的数据，在超时或出现其他丢包信号时重传。接收方可以先缓存乱序到达的数据，但交给应用时仍然按字节顺序。滑动窗口让发送方不必每发一个包都等确认，可以同时保持多份在途数据。

重传计时有讲究：RTO 根据 RTT 的估计值计算，超时后还要退避，不能设成一个固定且过小的常量。SACK 等扩展还能更准确地告诉发送方哪些区间已经收到。

“可靠”的意思是协议会尽力重传并保持顺序，不意味着连接永远不会失败，也不意味着消息会立刻到达。[41][103]

09-09　流量控制与拥塞控制有什么区别？慢启动、拥塞避免、快速重传和快速恢复在解决什么?

两者保护的对象不同。流量控制保护接收方：接收窗口限制发送方，不要超过对端能缓存和处理的数据量。拥塞控制保护网络：拥塞窗口限制在途数据，不要把整条路径压垮。实际能发多少，同时受这两个窗口约束。

四个机制：慢启动从比较保守的窗口开始探测容量；拥塞避免让窗口更平缓地增长；收到重复 ACK 可以提示某个报文段可能丢了，从而触发快速重传；快速恢复避免每次都退回到最初的状态。

不同的 TCP 变体会调整具体算法。对 Gameplay 面试来说，首先要分清“接收方背压”和“网络拥塞”，而不是只会背窗口的名字。[102]

09-10　TCP 是字节流意味着什么？“粘包/拆包”为什么是应用层消息边界问题？

TCP 只给应用提供连续、有序的字节，不保留每次 send 调用的边界。一次发送的数据可能分几次读到，几次发送的数据也可能一次就读出来。

所以这不是 TCP 把两个“包”错误地粘在了一起，而是应用没有在字节流上定义自己的消息格式。常见的做法有固定长度、分隔符、长度前缀和自描述协议。

解析器要能处理半个头部、半条消息、连续多条消息、异常长度和版本兼容，不能假定一次 recv 就能拿到一个完整的结构体。对外部来的数据还必须限制长度，防止内存放大和越界。[41]

09-11　为什么很多实时游戏基于 UDP？可靠 UDP/KCP 通常要补哪些机制，哪些消息不该强制可靠有序？

UDP 保留数据报边界，不内置重传和全局有序交付，所以应用可以按每类消息的语义自己选择可靠性。实时的位置、朝向或输入快照一旦过期，新数据往往比旧数据更有价值；如果所有消息共用一个严格可靠有序的队列，早期的一次丢包就可能卡住后面的状态。

在 UDP 上做可靠传输，通常要补这些机制：连接标识、序号、ACK、重传、窗口、拥塞控制或节流、分片重组和超时管理。KCP 提供了 ARQ 这一层，但连接管理、加密、安全、消息分类和业务幂等，仍要项目自己补。

原则是：关键的提交可以走可靠通道；频繁更新的状态可以不可靠，或者只保留最新值。不要用一种通道语义覆盖全部消息。[42][104]

09-12　心跳、断线重连和快照恢复分别解决什么？怎样避免重放一次性副作用？

三者各管一件事：心跳在没有业务流量时判断连接是否还活着，并估计超时；断线重连重新建立会话和身份；快照恢复把客户端重建到服务器认可的当前状态。重新连上 Socket，不等于游戏状态已经一致。

恢复的数据要分成两类：可以重复应用的“现状”，和只能提交一次的“副作用”。比分、阶段和库存，可以通过带版本的快照重建；发奖、扣费和领取，则需要请求 ID、事务记录或幂等键，不能因为客户端重放了历史请求就再执行一次。

另外，旧会话的迟到结果要被丢弃，不能覆盖新会话的状态。[17][34]

09-13　网络序列化为什么不能直接 memcpy 一个 C++ 结构体？字节序、对齐和协议版本怎样处理？

结构体里可能有填充、指针、和虚表相关的状态，以及与平台相关的大小；不同机器的字节序和浮点表示约定也可能不同。直接 memcpy 会把本机的内存布局泄露出去，表达不了对象引用和可选字段，也很难保证版本兼容。

协议应该显式规定字段、顺序、长度、取值范围和字节序；对象用稳定的网络身份来表示，而不是地址。协议演进时，可以用版本号、可选字段、向后兼容的默认值，或者按消息类型分别升级。

反序列化必须验证长度和数值范围，不能因为数据来自“自己的客户端”就相信它的布局和内容。[3][17][41]

09-14　MTU、IP 分片和应用层分片有什么关系？为什么实时游戏应避免无界的大 UDP 数据报？

每条链路都有最大传输单元（MTU）。超过路径 MTU 的 IP 数据报可能被分片，或者在禁止分片时直接发送失败。任何一个分片丢了，整份原始数据报都可能无法重组；而且中间设备对分片的处理也并不总是稳定。

所以实时协议通常控制单个数据报的大小，在应用层把大消息拆成可识别的片段，并给出消息身份、片号、总片数、超时和内存上限。

更好的方案往往是从源头减少“必须一次发完”的数据：用增量状态、资源走 CDN，或者走独立的可靠通道，而不是把巨大的快照无限制地塞进 UDP。[42]

设计模式与工程架构

09-15　单例模式有什么问题？线程安全、生命周期、测试与多 World 场景下怎样避免进程级全局状态污染？

单例提供了一个全局访问点，但也会隐藏依赖、延长状态的寿命，并让测试难以隔离。线程安全的初始化只解决“是否只构造一次”，不解决对象内部的并发访问、关闭顺序和跨场景的状态污染。

UE 里这个问题更明显：编辑器、多人 PIE 和服务器进程中，可能同时存在多个 World，进程级的 static 不会自动变成“每局一份”。

更好的做法是选择寿命匹配的 Subsystem、GameState、PlayerState 或显式的服务对象，并通过接口注入依赖。只有真正属于进程级、而且不携带局内可变状态的设施，才适合做全局单例。[1][61]

09-16　观察者/事件总线如何处理订阅者销毁、重入、顺序和跨线程？

观察者模式把发布者和具体的订阅者解耦了，但“注册关系”本身也有生命周期。订阅者退出、Pawn 更换、Widget 移除时，都要解除绑定；弱绑定只能避免一部分悬空访问，证明不了回调仍然属于当前的业务对象。

广播期间，订阅者可能再次订阅、移除自己，或者触发嵌套广播，所以容器在遍历中被修改时怎么办、通知按什么顺序，都必须事先规定。跨线程发布时，还要规定数据的所有权，以及在哪里切回游戏线程。

事件总线适合通知“发生了什么事实”，不应该让任意订阅者都能修改核心状态，形成一条无法追踪的控制流。[31][63]

09-17　工厂、状态、策略和组件化分别适合解决什么变化点？什么时候组合优于继承？

四种模式应对四种不同的变化：工厂封装“创建哪一种具体对象”；状态模式把随当前状态变化的行为组织起来；策略模式在同一个接口下替换算法；组件化把可复用的能力挂到宿主上。用模式是为了应对变化，而不是为了用模式去叠类层次。

当多个能力需要自由组合、运行时替换，或者各自有独立的生命周期时，组合通常比深层继承更清楚。继承适合稳定的、真正的类型关系和引擎约定，比如 Character 是一种 Pawn；如果派生类只是为了开关几项功能，用组件、数据配置或策略通常更灵活。

无论用哪种模式，最后都要能说清：状态由谁持有、修改入口在哪、对象什么时候失效。[11][31]

渲染与性能常识

09-18　前向渲染与延迟渲染的核心差异是什么？GBuffer、透明物体和多光源分别带来什么权衡?

前向渲染通常在绘制几何体的同时计算相关的光照，材质和光源的组合会影响着色成本。延迟渲染先把几何和材质信息写进 GBuffer，再在后面的光照阶段只处理可见的像素，因此适合大量动态光源和屏幕空间效果。

延迟渲染的代价在 GBuffer：它占用显存和带宽，也不天然适合常规的半透明和 MSAA，透明物体通常仍走前向路径。前向路径的基线开销可以更低，对 MSAA 的支持也更直接，但面对复杂的多光源和材质特性，有它自己的权衡。

Gameplay 程序要知道的是：渲染路径会改变材质、灯光和效果的成本；“延迟渲染一定更高级”不是一个结论。[50][106]

09-19　深度测试、Early-Z、半透明排序和过度绘制怎样影响正确性与性能?

深度测试比较片元的深度，决定它能不能覆盖当前像素。如果 GPU 能在执行昂贵的像素着色之前就确认片元被挡住了，Early-Z 就能省掉这部分无效着色；但写深度、材质的副作用和某些透明路径，会影响提前测试的条件。

常规的半透明需要和背景混合，通常不能像不透明物体那样只保留最近的表面，因此要大致按从后往前的顺序排序，并且可能产生大量的过度绘制。复杂粒子、全屏半透明 UI 或多层特效，即使 Draw Call 不多，也可能成为像素瓶颈。

判断时结合 GPU 计时和可视化工具，而不是只看三角形数量。[49][50]

09-20　Shadow Map 的基本流程是什么？常见锯齿、偏移和漏光问题怎样缓解?

Shadow Map 分两步：先从光源的视角渲染一张深度图；在相机视角着色时，把当前点变换到光源空间，和深度图里的值比较，判断光线是否被挡住。方向光常用级联阴影覆盖不同的距离；点光源需要更多方向的深度信息。

三类常见问题和缓解方式：

- **锯齿**：来自有限的分辨率和采样，可以用 PCF 等滤波、提高有效分辨率或调整级联分配来缓解；
- **偏移**：深度偏移不够会出现自阴影，偏移过大又会让阴影“悬浮”；
- **漏光**：薄几何体和采样精度不足时可能出现。

这些参数只能在不同伪影之间权衡，必须结合场景尺度和相机范围来调。[50]

09-21　MSAA、FXAA、TAA/TSR 与 SSAA 分别解决什么，主要代价是什么?

- **MSAA**：对几何覆盖的边缘做多重采样，主要改善几何锯齿；成本随样本数增加，并不能完整解决材质和透明效果的锯齿。
- **FXAA**：空间上的后处理，速度快，但可能让细节变软。

- **TAA**：利用跨帧抖动的样本和历史信息，能处理更多种类的锯齿，还支持上采样；代价是可能出现拖影、闪烁和历史失效的问题。
- **SSAA**：用更高的分辨率渲染再缩小，质量最直接，代价也最高。

UE 中具体能用哪种方案，受渲染路径和平台影响，比如 MSAA 主要用于前向路径。

Gameplay 程序不需要实现这些算法，但应该能判断某个拖影或边缘问题属于哪一类，并理解它和帧时间的关系。[107]

第十册　Lua/UnLua 与 Gameplay 场景题

前半部分讲 Lua 语言本身和它与 UE 对象的边界；后半部分是几道综合场景题，把前面各册的知识串起来用。

Lua 与 UnLua

10-01　Lua table、metatable、__index / __newindex 怎样工作？如何据此模拟类、实例和继承？

table 是 Lua 的核心复合数据结构，既能当数组，也能当键值映射。metatable 给 table 或 userdata 提供“元行为”：读取一个不存在的键时，可能触发 `__index`；向一个不存在的键写入时，可能触发 `__newindex`。这两个元方法既可以是函数，也可以指向另一张表。

它们的常见用途：`__index` 链用于方法查找和原型式继承；`__newindex` 可以做校验、代理或只读表，但要避免它递归地再次触发自己，通常用 rawget/rawset 访问原始表。要记住，元表改变的只是访问语义，并不会带来 C++ 那样固定的对象布局和编译期类型检查。[15]

用它模拟类的常见做法是：让“类表”保存方法，把实例表的 `metatable.__index` 指向类表；派生类表再通过自己的元表向父类查找。冒号调用 `obj:Func(x)` 会把 obj 作为第一个参数传进去，本质上仍然是函数加表的组合。

Lua 没有强制唯一的类模型，也没有自动的构造、析构和访问控制，项目框架必须自己约定创建、销毁、继承和事件解绑的方式。混用多个 OOP 辅助库时，更要先确认 `__index` 链和对象身份，不能看到函数名相同，就假定它和 C++ 虚函数行为一致。[15]

10-02　Lua 闭包和 upvalue 怎样工作？为什么事件回调可能让旧状态长期可达？

内部函数引用了外层的局部变量，这些变量就成为 upvalue；函数和它捕获的环境一起构成闭包。外层函数返回之后，只要闭包还可达，对应的 upvalue 就一直存在，所以局部表不会因为离开了词法作用域就立刻被回收。

问题出在注册：闭包一旦被注册到全局事件、Timer 或 UObject 回调上，它就可能一直捕获着旧的 Pawn、旧的界面模型或大型配置表。只把某个外部变量设成 nil，不一定能断开所有引用。

正确做法是保存订阅句柄，在关系结束时解除订阅；回调执行时，还要核对当前的对象和请求身份。[15]

10-03　Lua 协程怎样挂起和恢复？它与操作系统线程、UE Timer 和异步任务有什么区别？

Lua coroutine 保存一个函数的执行状态：调用方通过 resume 推动它执行，它在 yield 处主动挂起。它通常在同一个操作系统线程里做协作式切换，因此不会自动并行执行，也绕不开“只能在主线程访问 UObject”的限制。

三者对比：Timer 按时间调度回调；异步任务可能在线程池里执行；协程主要改善的是流程的写法。三者在取消、错误传播和对象寿命上的规则都不一样。

协程挂起期间，角色可能已经死了，World 可能已经切换了，所以恢复时必须重新验证依赖。不能因为代码写起来是连续的，就以为中间状态没有变化。[15]

10-04　Lua GC 与 UObject GC 是两套系统时，绑定层如何防止悬空、泄漏和重复解绑？

Lua 对象是否可达，和 UObject 是否被 UE 的引用图保活，是两件独立的事。Lua 代理还在，底层的 Actor 可能已经 Destroy 了；反过来，UObject 还活着，也不代表旧的 Lua 逻辑仍然属于当前的 Pawn 或 World。

所以绑定层通常需要弱引用、有效性检查和销毁通知来协调两边。项目要规定清楚：谁拥有脚本实例；谁负责解绑委托和 Timer；UObject 结束时怎样让代理失效。

Lua 变量不是 nil，代替不了 `UGCObjectUtility.IsObjectValid` 或 UE 自己的对象检查。重复热载入和重生时，还要避免同一个回调被注册多次。[2][15][105]

10-05　UnLua 怎样暴露 UE 类型？脚本热更新能改什么，不能安全改变什么？

UnLua 借助 UE 的反射信息访问 UCLASS、UPROPERTY、UFUNCTION、USTRUCT 和 UENUM，还能让 Lua 替换或实现一部分蓝图事件和函数。它减少了手写胶水代码，但底层调用仍要匹配 UE 的类型、参数方向、对象身份和反射可见性。

脚本能读到一个属性，不代表这个属性可以在任意一端、任意线程、任意生命周期阶段被修改。RPC、复制、Authority 和 GC 的规则仍然属于 UE，Lua 只是又多了一层调用入口。排查问题时，要分清 Lua 表的状态、代理对象、真实的 UObject 和网络副本。[105]

热更新通常适合替换函数实现、配置和一部分脚本表的内容。已经存在的闭包、协程栈帧、事件订阅和 Lua 实例，未必会自动切换到新定义。C++ 类布局、UProperty 序列化结构、网络协议和已经加载的资产关系，也不会因为脚本文件变了就被安全地重建。

可靠的热更新方案要有版本和迁移边界：暂停或关闭受影响的流程，更新模块，重建或迁移实例，再恢复运行。旧回调到达时，要能识别它属于哪个代码版本和哪次业务；生产环境还要支持失败回滚。不要把“require 重新执行成功”当成所有状态都已经一致。[15][105]

Gameplay 场景设计

10-06　设计一个背包系统时，物品定义、物品实例、格子、堆叠与网络状态应该怎样分层？

分三层数据：物品定义描述静态数据，比如名称、图标、最大堆叠数和可用的行为；物品实例保存唯一 ID、数量、耐久和随机词条；格子只表达“在哪个位置、属于哪个容器”。Widget 不是物品状态，DataTable 的行也不能直接代表每一件运行时物品。

权威在服务器：服务器维护库存和事务入口，客户端只提交移动、使用或拆分的意图。一次操作要检查版本、来源格、目标格、数量和规则，然后原子地提交，再把结果复制出去。

UI 根据当前快照刷新。异步加载的图标和被复用的列表条目，都必须核对物品实例 ID，确保旧的加载结果不会覆盖新格子。

10-07　Buff 的叠加、刷新、覆盖和移除应该怎样设计，避免一个来源结束后把其他来源的效果一起清掉？

第一步是定义 Buff 的身份和叠加键：按类型合并，还是按施加者、技能实例或装备来源分别存在。第二步定义规则：叠层、刷新持续时间、取最大值、各自独立计时，还是互斥覆盖。只用一个 bool 表示“有这个 Buff”是不够的。

移除的关键是每个活动效果都保留来源和句柄，移除时只结束自己负责的那个实例。属性最好由“基础值 + 当前所有活动修饰”聚合得出，而不是在当前值上不断加加减减，再去猜怎么恢复。

GAS 已经提供了 Effect、Tag 和聚合协议，但具体怎么叠加、死亡后是否保留，仍然是项目自己的规则。[75][78]

10-08　从输入、技能、Montage/Notify 到伤害结算和网络表现，怎样定义一条不重复提交的攻击链？

每一层只做自己的事：输入只表达动作意图；技能或战斗状态机检查资格并建立“本次执行”；Montage 提供动作的时间线，Notify 可以开启命中窗口或触发表现；服务器在明确的提交点验证当前执行、目标、距离和资源，然后结算伤害。

防重复的核心是：同一次攻击有自己的执行身份，避免输入、Notify、碰撞回调和服务器确认各结算一次。

取消、被打断、死亡和换装备，都要关闭当前窗口并清理监听。远端客户端只消费权威状态或表现事件，动画回调不能成为最终伤害的真值。[77][87][89]

10-09　大量目标中寻找最近可见敌人时，怎样使用空间划分、粗筛、精确查询和每帧预算？

思路是逐步缩小候选，越往后越精确。先用不会改变结果语义的条件缩小范围，比如阵营、存活、距离范围和空间网格；再按距离下界或优先级处理候选；最后只对少量目标做视线、碰撞体或骨骼级的精确查询。

正确性有一个关键点：如果要求的是严格意义上最近的可见目标，停止条件必须能证明，没检查到的对象不可能更优。固定只检查最近的 K 个是一种近似策略，必须承认它可能漏掉“排在被遮挡目标之后”的合法目标。

成本方面，大量单位不必每帧全量查询，可以缓存注册表、错开更新时间、按重要性分级并设置预算。优化前后都要验证选出来的结果是否正确，以及最坏帧，而不只是平均耗时。[43][49][99]

10-10　大地图流送和异步加载中，怎样避免旧结果作用到已经离开的 World、区域或角色？

把每个加载请求和它的目标 World、区域坐标、资源版本、请求编号绑在一起。玩家离开区域、切换地图或目标被替换时，让旧请求的身份失效。完成回调仍然可能到达，但只能释放自己的句柄，不能把结果装到新场景里。

世界状态应该保存稳定 ID 和可以重建的数据，而不是旧 Actor 的地址。

还要分清三个阶段：资源加载、关卡流送、玩法激活。资源到了，不代表 Actor 已经生成；Actor 生成了，也不代表网络和依赖都已就绪。在每个阶段的边界记录状态和失败原因。[29][32][34]

10-11　一次性事件、持久状态和幂等提交分别应该怎样表达?

先给三类东西分好类：“门当前是开着的”“玩家有 3 把钥匙”是状态，应该能让晚加入或重新相关的客户端恢复；“播放一次爆炸声”是短暂的事件；“发放首通奖励”是一次性提交，需要服务器记录它是否已经完成。三者不能都塞进 Multicast，或者都用一个 bool 表示。

对应的表达方式是：事件可以携带操作 ID 用来去重；状态通过复制字段或快照发布；一次性副作用用事务记录和幂等键保护。

这样，客户端重建 UI、重连或重复发请求时，可以重新读取状态，但不会再次发奖、扣费或消耗道具。[17][34]

10-12　复杂 Gameplay Bug 应怎样沿调用链找到“第一个分歧点”，而不是同时修改多个系统?

先写出预期的链条：输入或事件来自哪一端，经过哪个对象和状态入口，产生了什么权威结果，怎样被复制或通知，最后由哪个表现层消费。

然后在每个边界记录：执行端、对象身份、请求 ID、旧值和新值、拒绝原因。复现问题后，从上游往下游比对实际的链条，找到第一个和预期不一致的位置：是请求没发、服务器拒绝了、状态没写进去、复制没到达、OnRep 没处理，还是 UI 绑着旧对象。

每次只改变一个假设，并保留一个能回归的最小用例。日志多不是目标，能还原时序和身份才是目标。[49]

参考资料

下列资料用于核对机制与接口。标注 UE5.5 的条目是本书的技术基线；标注“现行页面”的只用于对照稳定的概念，不把后续版本新增的行为当成 UE5.5 的事实。项目文章和开源仓库不构成 UE 固定实现的保证。

- [1] Gameplay Framework — Epic · UE5.5
- [2] Unreal Object Handling — Epic · UE5.5
- [3] Remote Procedure Calls — Epic · UE5.5
- [4] Actor Lifecycle — Epic · 现行页面，仅作稳定机制对照
- [5] Replicated Object Execution Order — Epic · 现行页面，仅作稳定机制对照
- [6] Components — Epic · UE5.5
- [7] Gameplay Timers — Epic · UE5.5
- [8] Enhanced Input — Epic · UE5.5
- [9] Networked Character Movement — Epic · 现行页面，仅作稳定机制对照
- [10] Cameras — Epic · UE5.5
- [11] Interfaces — Epic · UE5.5
- [12] Slate Overview — Epic · UE5.5
- [13] Optimization Guidelines for UMG — Epic · UE5.5

- [14] UListView — Epic · UE5.5
- [15] Lua 5.4 Reference Manual — Lua 官方 · 语言概念，不指定绿洲的绑定层或版本
- [16] Data Driven Gameplay Elements — Epic · UE5.5
- [17] Replicate Actor Properties — Epic · UE5.5
- [18] Detailed Actor Replication Flow — Epic · UE5.5
- [19] Actor Network Dormancy — Epic · UE5.5
- [20] FPredictionKey — Epic · UE5.5
- [21] Snapshot Interpolation — Gaffer On Games · 作者技术文章
- [22] Deterministic Lockstep — Gaffer On Games · 作者技术文章
- [23] UProjectileMovementComponent — Epic · 现行页面，仅作稳定机制对照
- [24] Peeking into VALORANT’s Netcode — Riot Games · 项目开发者解释，非 UE 通用保证
- [25] Unreal Header Tool — Epic · UE5.5
- [26] Unreal Engine Modules — Epic · UE5.5
- [27] 对象生命周期 [basic.life] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [28] Object Pointers — Epic · 现行页面，仅作稳定机制对照
- [29] Asynchronous Asset Loading — Epic · UE5.5
- [30] Unreal Smart Pointer Library — Epic · 现行页面，仅作稳定机制对照
- [31] C++ Core Guidelines — C++ 指南作者 · 工程建议，不是语言标准
- [32] Asset Management — Epic · UE5.5
- [33] Packaging Your Project — Epic · UE5.5
- [34] Travelling in Multiplayer — Epic · UE5.5
- [35] About Processes and Threads — Microsoft · Windows 进程与线程
- [36] 数据竞争与同步 [intro.races] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [37] Virtual Address Space — Microsoft · 虚拟地址空间
- [38] Working Set — Microsoft · 工作集与缺页
- [39] vector [vector] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [40] False Sharing — Linux 内核官方文档 · 伪共享
- [41] RFC 9293：Transmission Control Protocol — IETF · TCP 规范
- [42] RFC 768：User Datagram Protocol — IETF · UDP 规范
- [43] Vectors and Matrices — MIT 开放课程 · 向量与矩阵
- [44] Coordinate System and Spaces — Epic · 现行页面，仅作稳定机制对照
- [45] XMQuaternionSlerp — Microsoft · 四元数球面插值
- [46] Collision Overview — Epic · UE5.5
- [47] Physics Sub-Stepping — Epic · UE5.5
- [48] UPhysicsHandleComponent — Epic · UE5.5
- [49] Unreal Insights — Epic · UE5.5
- [50] Direct3D 11 Graphics Pipeline — Microsoft · 流水线概念，不代表 UE 固定渲染路径
- [51] const 与 volatile [dcl.type.cv] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [52] new 表达式 [expr.new] — C++ 在线工作草案 · 仅引用正文所述既有机制

- [53] unique_ptr [unique.ptr] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [54] shared_ptr [util.smartptr.shared] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [55] 拷贝与移动构造 [class.copy.ctor] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [56] move 与 forward [forward] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [57] 虚函数 [class.virtual] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [58] 构造与析构期间的对象 [class.cdtor] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [59] sizeof [expr.sizeof] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [60] 显式类型转换 [expr.cast] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [61] 存储期 [basic.stc] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [62] 复制省略 [class.copy.elision] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [63] Lambda 表达式 [expr.prim.lambda] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [64] 模板 [temp] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [65] 顺序容器 [sequences] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [66] 无序关联容器 [unord.req] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [67] 关联容器 [associative.reqmts] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [68] 标准库算法 [algorithms] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [69] Introduction to Algorithms：课程讲义 — MIT 开放课程 · 排序、图与动态规划
- [70] 互斥量 [thread.mutex] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [71] 原子操作与内存顺序 [atomics.order] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [72] std::thread [thread.thread.class] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [73] 条件变量 [thread.condition] — C++ 在线工作草案 · 仅引用正文所述既有机制
- [74] AddressSanitizer — Clang 官方 · 诊断工具
- [75] Gameplay Ability System — Epic · 现行页面，仅作稳定机制对照
- [76] Ability System Component and Attributes — Epic · 现行页面，仅作稳定机制对照
- [77] Using Gameplay Abilities — Epic · UE5.5
- [78] Gameplay Effects — Epic · UE5.5
- [79] FGameplayEffectSpec — Epic · 现行页面，仅作稳定机制对照
- [80] GASDocumentation：属性捕获与计算 — 作者一手示例 · 声明基线 UE5.3，非官方源码契约
- [81] Gameplay Ability Tasks — Epic · UE5.5
- [82] FGameplayAbilityTargetData — Epic · 现行页面，仅作稳定机制对照
- [83] AGameplayCueNotify_Actor — Epic · 现行页面，仅作稳定机制对照
- [84] Animation Blueprints — Epic · 现行页面，仅作稳定机制对照
- [85] Animation Optimization — Epic · UE5.5
- [86] State Machines — Epic · UE5.5
- [87] Animation Montage — Epic · UE5.5
- [88] Animation Slots — Epic · UE5.5
- [89] Animation Notifies — Epic · UE5.5
- [90] Aim Offset — Epic · UE5.5
- [91] Two Bone IK — Epic · UE5.5

- [92] Root Motion — Epic · UE5.5
- [93] Motion Warping — Epic · UE5.5
- [94] AAIController — Epic · 现行页面，仅作稳定机制对照
- [95] Behavior Tree Overview — Epic · UE5.5
- [96] StateTree — Epic · UE5.5
- [97] UBTTaskNode — Epic · UE5.5
- [98] AI Perception — Epic · UE5.5
- [99] Environment Query System — Epic · UE5.5
- [100] Navigation System — Epic · UE5.5
- [101] Interprocess Communications — Microsoft Learn · Windows IPC 概览
- [102] RFC 5681: TCP Congestion Control — IETF · TCP 拥塞控制规范
- [103] RFC 6298: Computing TCP's Retransmission Timer — IETF · TCP 重传计时规范
- [104] KCP - A Fast and Reliable ARQ Protocol — skywind3000 · 项目仓库与协议说明
- [105] UnLua — 腾讯开源 · UE Lua 脚本解决方案
- [106] Forward Shading Renderer — Epic · 现行页面，仅作稳定机制对照
- [107] Anti-Aliasing and Upscaling — Epic · 现行页面，仅作稳定机制对照

终局补全卷：场景题 • 项目追问 • 生产工程与趋势

补全卷的所有题都围绕同一组问题来答：涉及哪些对象、状态放在哪里、在哪一端执行、生命周期怎样、调用链怎么走、什么条件下会失效、拿什么证据验证。这组问题是思考的清单，不是答题的模板——每道题只挑真正相关的几项展开。

项目题不提供可以背诵的“标准答案”，只告诉你这道题在考什么、该从哪里想、要准备什么证据。答案必须来自你自己的代码和运行结果。

A. Gameplay 场景题库 • 40 题

场景题考的不是能不能说出一个架构名词，而是能不能从需求出发，搭出一个运行时模型。开口先给一句结论，然后沿着对象、状态、执行端、生命周期、失败条件和验证顺序往下讲，讲到面试官想追问的地方为止。

S01　[场景] 背包：物品定义、实例、格子与权威状态怎样分层?

核心是把“是什么物品”、“这一件物品现在怎样”和“它放在哪”拆成三层。静态定义保存名称、图标、最大堆叠和规则；运行时实例保存唯一 ID、数量、耐久和随机词条；格子只表达它在哪个容器的哪个位置。

权威状态在服务器上。服务器持有库存和事务入口，客户端只提交移动、拆分、使用这类意图；一次操作要校验版本、来源、目标、数量和规则，然后原子地提交。UI 只消费库存的快照或增量，不维护第二份库存真值；列表条目复用和异步图标都要核对实例 ID。持久化时保存稳定 ID 和实例状态，不保存 Widget 或 Actor 的地址。

常见失分：把 DataTable 的行直接当成运行时物品；让 Widget 自己加减数量；没有说明并发和重复请求时会发生什么。

S02　[场景] 两个客户端几乎同时移动/拾取同一件物品，怎样避免双重提交？

把“检查”和“修改”放进同一个权威事务里。服务器用物品或格子的版本号、操作 ID，或者资源锁定规则来裁决。

成功的路径只产生唯一的结果；失败的路径返回明确的原因，让客户端回滚预测或刷新快照。重复的包、重试和迟到的请求都必须幂等，同一个操作 ID 不能再发一次奖励或再扣一次。测试要覆盖同一帧内的竞争、延迟导致的乱序，以及断线重连后的重放。

常见失分：只在客户端禁用按钮；先检查、再异步修改，中间没有原子性；把 Reliable 当成事务。

S03　[场景] 角色重生时，装备、背包、血量和当前技能分别应该放在哪里？

先按寿命分类：跨身体存在的玩家身份和长期资源，与当前身体的状态分开；再按可见范围和权威端，决定它们归 PlayerState、Pawn、组件还是某个服务。

重生要有明确的迁移规则：哪些状态继承，哪些重置，哪些从持久状态重新派生。旧 Pawn 的计时器、委托、动画等待、武器引用和预测结果，只能清理它自己的，不能破坏新 Pawn。客户端重新绑定 UI、动画和输入时，要允许多次尝试，而且不能重复订阅。

常见失分：回答“都放 Character 里”；只 Spawn 了新 Pawn，没有处理旧关系；用固定 Delay 等数据到齐。

S04　[场景] 多个来源都施加同一种 Buff，叠层、刷新、覆盖和移除怎样设计？

先定义 Buff 的身份键：按类型合并，还是按施加者、技能实例、装备来源分别存在；再定义规则：叠层、刷新、取最大值、独立计时，还是互斥。

每个生效中的 Buff 都保留来源和句柄，移除时只撤销自己负责的那一份，而不是反过来去猜“原值”是多少。属性由“基础值 + 当前所有修饰”聚合得出，避免多次加减造成数值漂移。死亡、驱散、换装和重连时，Buff 活多久、怎样恢复，都要有规定。

常见失分：只用一个 bool；一个来源结束后，无条件把属性减回去；不区分来源。

S05　[场景] 技能有 3 层充能、共享冷却和单层恢复，状态怎样组织？

不要用一个“剩余时间”包办所有事情。把可用层数、每一层的恢复时间点、全局或共享冷却、资源消耗拆成各自明确的状态。

服务器做最终裁决；客户端可以预测 UI 和局部反馈，但服务器拒绝时要能收敛回来。同一个技能遇到重生、换装备或重新加载配置时，要明确计时器和效果是否跟着迁移。UI 从权威状态或预测状态派生，不能自己倒计时、变成真值。

常见失分：三个 Timer 互相覆盖；客户端冷却一到零就直接放技能；重生后旧 Timer 还在修改新技能。

S06　[场景] 门、钥匙卡、密码面板与任务条件怎样组合成可扩展交互系统?

玩家只发出“尝试交互”的意图；交互目标对外暴露契约；权限、凭证、任务规则交给独立的规则对象或服务判断。

门的权威状态和动画表现要分开：Open、Locked 这些是状态，播放声音和动画是表现。钥匙卡、密码、任务条件可以组合成规则，不应该硬编码进玩家类，或者写成每扇门里的一串分支。晚加入或重新加载时，直接根据门的当前状态恢复碰撞和动画进度。

常见失分：把所有条件写进 Character；把过去 Multicast 的开门动画当作唯一真值。

S07　[场景] 一个任务有“未开始 → 进行中 → 完成 → 领奖”，晚加入和重连怎样恢复?

先把三样东西分开：任务阶段是持久状态；“弹一次完成提示”是短暂事件；“发奖励”是只能执行一次、必须幂等的提交。

服务器保存阶段、进度、版本和奖励提交记录，客户端从快照恢复 UI。阶段切换要有唯一的入口和合法的前置条件，避免多个触发器同时把任务“完成”。重连只能重建现状，不能把发奖这样的副作用再执行一遍。

常见失分：阶段、弹窗和奖励全用同一个 RPC；客户端在本地完成任务。

S08　[场景] 存档结构升级后，旧版本存档怎样兼容?

存档只保存稳定 ID 和最小必要的状态，并带上明确的版本号。加载时先做格式和取值范围的校验，再迁移到当前的结构。

迁移函数按版本一步一步升级，处理好默认值和废弃字段；迁移失败时要给出能诊断的原因。运行时的 Actor 或组件地址、临时句柄、委托都不进存档。对关键资产的 ID，要有缺失或重命名时的处理策略，并用一组历史存档做回归测试。

常见失分：直接反序列化到最新结构；把对象指针写进存档；迁移失败后悄悄重置玩家数据。

S09　[场景] 异步加载 A 尚未完成，玩家已切换到 B；A 的回调晚到怎么办?

每次请求都绑定目标的身份、请求编号，以及所在 World 和角色的代次；发起新请求时，让旧身份失效。

晚到的回调只释放它自己的句柄，不把结果安装到当前对象上。路径相同不代表是同一次请求，所以不能只比较资源路径。切换地图、重生、关闭界面时，要取消或作废请求，并记录第一个出现分歧的地方。

常见失分：只检查 UObject 还活着；旧结果直接覆盖了当前资源。

S10　[场景] 对象池复用 Actor 时怎样防止“旧回调命中新租户”?

池里的对象每次被租出去，都要有一个新的租约或业务身份；回调带着这个身份，执行前先核对。

归还时要重置速度、碰撞、可见性、计时器、委托、命中标记，以及网络和业务状态。还要清楚对象池的边界：它解决的是创建和注册的热点，不会自动消除 Tick、渲染、碰撞和复制的成本。对网络 Actor，要先确认复用同一个网络身份在业务语义上是否成立。

常见失分：只调用 SetHiddenInGame；旧 Timer 继续执行；把对象池当成万能的性能优化。

S11　[场景] 本地预测弹体与服务器权威弹体怎样匹配和收敛?

客户端立刻生成一个纯表现的预测弹体；服务器独立验证这次射击意图，生成或结算权威结果。

两边靠项目自定义的 ShotId 或 FireId 关联同一发射击，要讲清楚它的作用域、重试规则和多弹丸规则；不要把它和 GAS 的 PredictionKey 混为一谈。权威结果到达时，决定是替换、吸附还是平滑；被拒绝或超时，就清理预测对象。还要处理切换武器、重生，以及权威对象还没复制过来就已经命中这些晚到的路径。

常见失分：客户端命中后直接扣血；把 PredictionKey 当作全局事务号；只处理正常到达的情况。

S12 [场景] Hitscan 在高延迟下怎样兼顾手感与服务器权威?

手感交给客户端，判定交给服务器。客户端可以立刻播放枪口火光、后坐力和本地命中特效；伤害由服务器裁决。

如果要做服务器回溯，就要保存有限长度的历史碰撞状态，用服务器认可的射击时间或序号重建当时的判定，并限制能回溯的窗口。同时要防止客户端随意提交一个过去的时间，或者伪造起点和方向；服务器还要检查武器状态、射速、视角和位置的边界。

回溯是一个专项方案，不是每个游戏都需要。如果用了，要能讲清它的成本、内存和公平性策略。

常见失分：认为“客户端看到就算命中”；把回溯当成 UE 的默认功能；忽略作弊的边界。

S13 [场景] 近战连击如何避免 AnimNotify、碰撞和技能回调重复结算伤害?

一次攻击有自己的执行 ID；玩法状态机或能力是唯一的提交者，Montage 和 Notify 只负责驱动命中窗口或发出请求。

命中窗口内要对目标去重，并明确是“同一个目标只打一次”还是“多段伤害”。服务器最后再验证动作仍然有效、目标仍然合法，然后提交伤害。被打断、死亡或换武器时，关闭窗口，并让晚到的回调失效。

常见失分：每个 Notify 都直接 ApplyDamage；靠“动画只播一次”来保证不重复。

S14 [场景] 锁定最近“可见且可攻击”的目标，怎样既正确又能扩展到大量目标?

逐层缩小候选：先用阵营、存活、范围和空间分区粗筛，再按距离下界或优先级处理少量候选，最后做视线或精确碰撞检测。

正确性要单独说清：如果要求的是严格意义上最近的可见目标，停止条件必须能证明没检查的对象不可能更优；固定取 Top-K 只是近似。目标失效、被遮挡或离开范围时，要有明确的解锁或切换策略。单位很多时错开更新频率，并验证最坏帧和选取结果是否正确。

常见失分：每帧全量 GetAllActorsOfClass 再加 Trace；把 Top-K 当作严格正确的答案。

S15 [场景] “点下一波后隔很久才出怪”，怎样按调用链定位而不是直接缩短 Delay?

先把链条写出来：UI 发请求 → 服务器或玩法状态接受 → 上一波结算完成 → 资源和出生点准备好 → 调度 → Spawn 成功。

在每个边界记录阶段、波次 ID、计划时间、阻塞条件和失败原因，找到第一个和预期不一致的位置。要区分几种原因：本来就设计了等待、计时器没清掉、还有存活的敌人、异步资源没到、出生点失败。修好之后，用连续多波、快速连点、暂停与恢复，以及打包版做回归。

常见失分：一律把计时器改成 0；没有波次身份，旧计时器能启动新的一波。

S16 [场景] 比赛阶段与倒计时怎样让服务器、客户端和晚加入者一致?

服务器保存阶段枚举和权威的时间基准（或结束时间）；客户端据此计算显示，而不是各自启动一个当作真值的 Timer。

阶段转换只有服务器的规则入口能做，UI 只负责显示。晚加入的玩家直接拿到当前阶段和时间基准，恢复当前的显示和交互权限。还要处理暂停、时间同步误差、重新相关和阶段快速连续切换。

常见失分：用 Multicast 让各端开始一个 30 秒的本地倒计时；由客户端 UI 决定阶段结束。

S17 [场景] 两个玩家同时拾取同一 Actor，怎样保证只有一个人获得奖励?

客户端只发请求；服务器在同一个权威入口里检查“现在还能不能拾取”，并立刻标记或移除这个可被争抢的状态。

奖励和物体失效要组成一次不可重复的提交，第二个请求得到失败原因。表现上可以预测，但失败时要回滚，不能只依赖 Actor 在客户端先消失。用并发测试模拟同一帧、不同延迟和重复 RPC。

常见失分：客户端 Destroy 之后就认为拾取成功；把奖励和销毁拆成两个中间能被插入的异步步骤。

S18 [场景] 角色死亡到重生，哪些数据应该迁移，哪些应该重置?

先把数据按寿命列出来：玩家身份、比赛状态、库存与装备、能力系统、身体属性、动画与任务。跨 Pawn 的状态由寿命更长的宿主持有，或者从权威数据重建；只属于身体的局部状态明确重置。

重生过程要分阶段：旧身体终止 → 新 Pawn 创建并被控制 → 依赖到齐 → 重新绑定 → 恢复表现。旧的回调、预测、Timer 和委托都要按旧身份隔离开。

常见失分：把旧 Pawn 的字段全部复制到新 Pawn；忘了解绑旧的 UI、动画或能力监听。

S19 [场景] 打开地图再关闭后，货架附近一批 Actor 突然可拾取，优先怀疑什么?

先怀疑地图 UI 的打开和关闭改了什么：输入模式、交互扫描开关、碰撞通道、可见性，或者“当前候选”缓存。

原则上，能不能拾取应该来自目标自身的规则和权威状态，与 UI 是否打开无关。所以重点检查关闭地图时，是不是误恢复了一个过宽的碰撞或交互过滤器，或者复用了旧的候选集合。用 Actor 类别、组件、碰撞响应、交互标记，以及打开地图前后的差异日志，找到第一个发生变化的地方。

常见失分：把货架当成单个 Actor 来查；只给出问题的武器加黑名单；没有弄清旁边的 Actor 为什么一起变了。

S20 [场景] 隐身技能被中断后永久隐身，怎样定位和设计避免?

把“是否处于隐身状态”和“材质与可见性的表现”分开；状态只能由能力或规则的唯一入口拥有。

然后把所有结束路径列全：正常结束、取消、被打断、死亡、更换 Avatar、服务器拒绝。每一条路径都必须撤销它自己施加的状态。如果有多个来源都能让角色隐身，要用来源句柄、引用计数或 Tag 聚合，不能一个来源结束就简单地 SetVisible(true/false)。测试时，在每个时间点强制打断，并记录 Tag、Effect、Cue 或材质以及能力实例的身份。

常见失分：只在“正常完成”的回调里恢复；表现层的 bool 成了唯一真值；用延迟补一个 SetVisibility。

S21　[场景] 背包有几千条记录，UMG 长列表怎样避免卡顿和错条目?

用虚拟化列表，让屏幕外的条目不再持续创建和布局。条目是可复用的视图，不是数据对象。

每个条目绑定稳定的 ItemId 或数据模型；复用时完整刷新，并取消旧的异步图标请求。更新靠事件驱动，而不是每帧对整个列表做 Bind 或 Tick；批量变化合并成一次刷新。用 Slate/UMG 的性能工具确认瓶颈到底在构建、布局、绘制还是资源加载。

常见失分：用 ScrollBox 手动塞几千个 Widget；在条目里保存一份库存真值；异步图标不校验 ID。

S22　[场景] UI、Lua/蓝图和玩法系统都保存“当前阶段”，为什么容易出错?

因为出现了好几份可写的真值。正确做法是指定唯一的权威状态宿主，其他层只派生或缓存可以随时丢弃的表现数据。

状态变化通过事件或数据绑定往外传；UI 只发意图，不反过来决定玩法。切换地图、重生、热更新之后，要重建订阅关系，并且能从真值重新生成界面。出现不同步时，沿“真值 → 通知 → 模型 → Widget”找第一个分歧点。

常见失分：三层都能写；UI 本地先改，再去“通知”玩法，形成循环。

S23　[场景] 列表条目复用时旧异步图标覆盖了新物品，怎样修?

每次给条目绑定数据，都生成并记录当前的 ItemId 和请求 ID；异步结果回来时先核对这两个值。

条目被复用或销毁时，取消或作废旧请求，并清理临时占位。资源缓存和条目身份要分开：缓存可以复用资源，但不能复用业务请求。测试时覆盖快速滚动、重复加载同一路径、资源缺失和界面关闭。

常见失分：只比较资源路径；以为取消之后回调就一定不会来。

S24　[场景] 小地图/大地图打开后，怎样避免输入、交互和世界状态互相污染?

UI 的输入模式、角色移动和视角的禁用、世界交互扫描，各自要有明确的拥有者和恢复路径。

打开地图前保存需要恢复的状态；恢复时核对当前上下文，避免旧 UI 把后来其他系统的修改覆盖掉。地图只负责显示世界数据，不修改“能否拾取”“能否攻击”这类权威资格。关闭、切换地图、死亡、被强制弹窗等异常路径，都走同一个可以重入的清理逻辑。

常见失分：打开地图时全局修改碰撞或 Actor 的启用状态；关闭时无条件恢复成默认值。

S25　[场景] AI 看见目标后丢失视线，应该保存什么“记忆”?

要分开保存当前可感知的目标、最后已知位置、刺激发生的时间和记忆的过期时间。手里持有那个 Actor，并不等于可以一直读取它的最新位置。

感知更新只提供事实，搜索、追击还是放弃由行为规则决定。目标死亡、阵营改变、被销毁或刺激过期，都能让这段记忆失效。多人游戏中关键的决策在服务器上做，客户端只消费表现。

常见失分：丢失视线后仍然每帧读取目标的当前位置；Blackboard 里一个 Target 指针包办了所有事实。

S26　[场景] MoveTo 显示成功寻路，但 AI 还是走不到，排查顺序是什么?

先确认路径本身：导航代理的尺寸、起点和终点的投射，以及这条路径是否真的可行。然后看 PathFollowing 的状态和移动组件。

接着检查动态障碍、角色碰撞、移动模式、速度和加速度，以及接受半径。要分清“规划成功”和“执行成功”是两件事；目标移动时按策略重新规划，不要每帧重发。记录请求 ID、失败码和当前路径，而不是只看角色动没动。

常见失分：一味放大 AcceptableRadius；同时重建 NavMesh、行为树和动画。

S27　[场景] 500 个 NPC 怎样分配 AI、动画和感知预算?

先把成本拆开：决策、感知、寻路、物理、动画、渲染；再按距离、可见性和玩法重要性分层。

远处的 AI 降低决策和感知频率；寻路请求要限流，并错开相位；关键事件仍然即时响应。动画可以用预算、降频、简化骨骼等机制，但伤害结算不能依赖可能被跳过的动画帧。验证时用最坏帧、大量单位同时被唤醒、以及从低频恢复到高频这几个场景。

常见失分：回答“全部放进线程池”；用了对象池，却仍然让每个 NPC 全频率 Tick；只看平均 FPS。

S28　[场景] Boss 从阶段 1 切到阶段 2 时，旧阶段的异步攻击怎样彻底退出?

每个阶段有唯一的代次或身份；所有任务、Timer、Montage、移动和投射物都知道自己属于哪个阶段。

切换顺序是：先让旧阶段停止产生新工作，再取消或作废已有的旧工作，最后建立新阶段。晚到的回调必须核对阶段身份，只能清理自己的东西，不能修改新阶段的状态。可以用故障注入验证：在切阶段的那一瞬间触发命中、动画 Notify、异步查询返回，或者让 Boss 死亡。

常见失分：只把 Phase 改成 2；旧 Timer 或 Notify 还能 Spawn 东西；清理函数把新阶段也一起清空了。

S29　[场景] Montage 被打断后旧 Notify 仍到达，怎样避免污染新动作?

玩法动作有自己的执行 ID 或能力实例；Notify 只有在当前执行仍然匹配时才生效。

正常完成、BlendOut、Interrupted、Ability 被取消，这些路径都进入同一个终止逻辑。外部委托、碰撞窗口和 Timer 由动作的拥有者注册，并成对地清理。旧动作只能撤销自己施加的限制，不能把新动作的状态设回 Idle。

常见失分：只监听 MontageEnded 的正常分支；回调里只检查 Character 是否有效。

S30　[场景] Root Motion 动作在客户端看起来正确，却持续被服务器拉回，为什么?

先确认位移最终是由 CharacterMovement 和它支持的根运动路径统一处理的，没有出现 Mesh 和 Actor 双重移动。

被拉回的根本原因是两端认知不一致：服务器和客户端必须对动作的身份、开始时间点、取消和运动模式达成一致；只在本地播放动画，不会产生权威的位移。依次检查根运动的提取模式、Montage、移动模式、网络角色和校正。观察时把服务器的胶囊轨迹和客户端的 Mesh 轨迹分开看。

常见失分：SetActorLocation 和 Root Motion 同时驱动；只去调网络平滑参数。

S31　[场景] 从输入到镜头、移动、瞄准、开火，一条 3C 链应该怎样分工?

每一层只负责一件事：Enhanced Input 只产生意图；Controller 和角色状态决定控制方向；CharacterMovement 负责会发生碰撞的移动；Camera 和 PlayerCameraManager 负责视图。

再往下，动画读取速度、朝向差这些事实生成姿态；武器和战斗系统按相机或枪口，以及服务器的规则，定义实际的射击。要分清 ControlRotation、ActorRotation、Mesh 朝向和枪口方向，明确谁跟着谁。联网时，拥有者得到即时反馈，服务器裁决，旁观者消费复制和插值之后的结果。

常见失分：输入值直接驱动所有层；认为相机朝哪儿，服务器上的枪口就一定朝哪儿。

S32 [场景] 策划说“手感差”，程序应该怎样把它变成可测问题?

第一步是把“手感”拆成维度：输入延迟、加速度和减速度、转向、镜头、后坐力、动画延迟、命中反馈、网络校正等。

然后建立基线：录像、输入时间戳、帧时间和关键参数，每次只改一个变量做 A/B 对比。把主观的说法转成具体场景：起步、急停、180° 转向、跳跃落地、开镜、受击。同时记录策划的评价和客观数据，不要一次改十个参数，再凭感觉判断是哪个起了作用。

常见失分：把“手感”归结为某个插值系数；没有对照实验，也没有固定的测试场景。

S33 [场景] 偶发一帧 120ms 卡顿，怎样定位?

先确认卡在哪一段：游戏线程、渲染线程、RHI 或 GPU、IO、GC，还是任务同步。用 Unreal Insights 或平台的 profiler 抓一段完整的时间线。

然后在尖峰前后找来源：同步点、资源加载、批量生成 Actor、日志或文件 IO、导航或物理、shader 或 PSO 等。建立能稳定复现的触发条件，比较 P95、P99 和最坏帧，而不是平均帧率。优化之后，用同一个场景、同一种采样方法复测，防止只是把成本挪到了别处。

常见失分：只看 FPS；把所有 Tick 都改成 Timer；没有基线就声称优化了多少百分比。

S34 [场景] UObject 偶尔被 GC / 旧对象偶尔还“有效”，应该怎么查?

两个现象，两条线索。对象被 GC 掉：检查引用是否处在 GC 能遍历的路径上、持有者本身是否可达，以及是不是只用了裸指针或普通 C++ 容器。旧对象还“有效”：要区分“对象还活着”和“它的业务身份仍是当前的”——旧 Pawn、旧 World 即使 IsValid，也可能已经不能再用。

外部回调、Timer 和异步请求要记录对象和业务身份，并在生命周期结束时解绑。不要用 AddToRoot 长期掩盖引用设计上的问题。

常见失分：一看到 GC 问题就全部 AddToRoot；只做 nullptr 检查。

S35 [场景] Dormancy/Relevancy 优化后客户端永久停在旧状态，为什么?

相关性决定一个连接是否需要这个对象，休眠减少长期不变对象的复制工作；但优化的前提是状态仍然能恢复。

修改休眠对象的状态时，要按所用的复制路径先正确唤醒或刷新，再改需要传播的数据。重新相关、晚加入和休眠期间发生的变化，都要验证能恢复到最终状态。定位时分段看：服务器上的值、连接的相关性、Actor Channel 和复制日志、客户端的 OnRep。

常见失分：做了“少发包”之后只看带宽；用一次 Multicast 补发状态，而不去解决恢复问题。

S36 [场景] 玩家断线 20 秒后重连，怎样恢复而不重复奖励/动画?

重新认证会话和玩家身份，然后从服务器快照重建持久状态和当前阶段。

一次性的副作用要有事务 ID 或提交记录，这样客户端重放请求也不会再执行一次。短暂的表现事件不需要完整重播，由当前状态决定该显示什么。旧连接迟到的结果，按会话或操作的代次丢弃。

常见失分：把断线前的 RPC 队列全部重放；新连接沿用旧对象的地址和回调。

S37 [场景] World Partition 流送后任务、AI 和交互对象怎样恢复?

把“这块区域是否加载在世界里”和“玩法状态是否存在”分开。持久状态用稳定 ID 或区域键，保存在寿命更长的层里。

区域加载时，根据当前的权威状态或存档状态，重新实例化或应用，而不是依赖上一次 EndPlay 之前留在内存里的东西。区域卸载时，AI 和任务的异步工作要终止或作废，旧结果不能作用到新的实例上。Data Layer 或流送策略变化时，要验证引用、任务和网络相关性是否仍然正确。

常见失分：跨卸载保存 Actor 指针；认为 EndPlay 之后对象一定会立即销毁。

S38 [场景] 编辑器能运行，Cook/Package 后资源缺失，怎样查?

先判断失败发生在哪个阶段：Build、Cook、Stage、Package，还是 Deploy/Run，然后看对应的日志和产物。

最常见的原因是：资产没有被引用、也没被 Asset Manager 的规则收集；用动态字符串拼出的路径；EditorOnly 的内容；平台条件。不要为了“先跑起来”把整个 Content 目录无条件打进包，而要先证明依赖链是完整的。在目标平台和配置上做最小的打包回归，并保留日志。

常见失分：把“打包”当成一个步骤；只在 PIE 里验证。

S39 [场景] 怎样把 Data Validation 与自动化测试接进玩法开发?

两类工具分工不同。验证器负责静态的、资产级别的不变量：DataAsset 的字段、引用、命名、关卡里对象的配置，并且尽量给出能定位的错误。测试分层进行：纯计算用单元级测试；涉及世界的玩法链路用自动化地图或功能测试；多人状态和打包版用更高层的回归。

每一条检查都要对应一个明确的失败定义，避免“测试存在，但什么都拦不住”。本地开发、提交门禁和 CI 分层运行，控制耗时和误报。

常见失分：所有问题都写成端到端测试；验证器偷偷修改资产；失败时只输出“测试失败”。

S40 [场景] AI 生成了一批 UE 代码/蓝图并编译通过，怎样证明功能真的完成?

把验证拆成多层：静态检查 → 编译和链接 → 资产与蓝图结构读回 → 运行 → 断言 → 日志、截图和各端证据 → 人工审查。

给 AI 明确它能修改的范围、项目约束和禁止事项；生成的结果必须能回滚、能 diff。网络行为、生命周期、资产连接和运行时语义，都不能用“编译成功”来代替。面试中要说清楚，哪些是自己设计的，哪些是 AI 生成的，哪些借鉴了社区，哪些经过了实际验证。

常见失分：把编译通过当作验收；没有结构读回和运行断言；把 AI 生成的结果夸大成个人手写。

B. AS（ArcaneSandbox）项目追问 • 45 题

AS 的追问只围绕已经确认的主线展开：GAS、多人网络、重生、预测与权威，以及 ShotId。下面每道题都先说明面试官真正想确认什么，再给出思考的抓手，最后说明需要准备什么证据。凡是涉及 AS 里还没确认的具体实现，答案都要回到你的代码和运行结果；宁可说“这部分还没验证”，也不要为了答得满而编造。

能力系统的归属与重生

AS01　[AS] 为什么把 ASC 放在 PlayerState，而不是 Character?

这道题考的是你会不会用“寿命”来做架构决策。回答的主线是：希望能力系统的状态跨 Pawn、跨重生持续存在，而身体是可以替换的。

接着要主动讲代价，这才显得你真的权衡过：Owner 和 Avatar 分离之后，初始化时序变复杂了，Avatar 更换后要重新绑定，复制和可见范围也要重新考虑。最后加一句：这是项目的选择，不是 GAS 唯一正确的架构。

准备的证据：能指出 PlayerState、ASC、AttributeSet 的实际类名，以及初始化发生在哪里。

AS02　[AS] OwnerActor 与 AvatarActor 在 AS 中分别承担什么?

考点是你是否分得清“谁拥有能力”和“谁在执行能力”。Owner 是长期的能力系统拥有者；Avatar 是当前执行动作、播放动画、负责移动的身体。

顺着这个区分往下讲：身体更换后要刷新 ActorInfo，并重建依赖当前 Character 的动画、装备和委托。再补一个容易被追问的点：旧 Avatar 仍然有效，不代表它还能参与当前的能力。

准备的证据：重生前后 ActorInfo 和对象身份的日志。

AS03　[AS] 哪些状态跨重生保留，哪些必须清空?

这题考的是分类能力，而不是记住一张清单。把状态按类别摆出来：玩家身份和长期资源、能力授予、临时的 GameplayEffect 和 Tag、身体属性、动画任务、武器状态。

每一类都回答三个问题：它的宿主是谁，在哪里结束，要不要迁移。还要强调，旧身体上的任务和表现，必须按旧的执行身份结束，不能误伤新身体。

准备的证据：一份重生状态表，或者对应的代码路径。

AS04　[AS] 服务器 Possess、客户端 PlayerState 到达和 ASC 初始化如何收敛?

考点是你是否理解“两端拿到依赖的时间不同”。先说明服务器和拥有者客户端各自在什么时候具备初始化条件。

解决思路是：Possess、OnRep_PlayerState 等相关回调都去尝试“依赖齐全才初始化”，并保证重复调用是幂等的。明确表态不用固定 Delay。

准备的证据：一段日志，包含执行端、Pawn、PlayerState、ASC、Owner 和 Avatar。

AS05　[AS] 怎样防止重生后能力重复授予或委托重复绑定?

这题检验你有没有被“重复初始化”坑过。能力授予要限定在权威端、限定在明确的阶段，可以用句柄或“已授予”状态来避免重复；绑定前先解除旧对象的绑定，或者把初始化写成可以重复调用的函数。

光说思路不够，要讲你怎么测：连续重生、多次触发 OnRep、加入网络延迟。

准备的证据：AbilitySpec 数量或委托调用次数的回归结果。

AS06　[AS] 血量、队伍、装备、弹药分别为什么放在不同宿主?

考点是你能否给每个字段的归属一个理由，而不是“习惯这么放”。从寿命、权威端、可见范围和复制成本四个角度解释。

公开的玩家信息和私有的、身体局部的状态要分开；换 Pawn、换武器时，各自怎么迁移也要讲清。

准备的证据：能指到每个字段实际所在的宿主，以及它的复制条件。

AS07　[AS] 死亡链怎样保证只有一个“最终提交点”?

这题考的是你会不会防重复提交。把死亡链分层：伤害产生 → 属性变化 → 死亡条件 → 比赛或重生规则。

要避免动画、UI 或 GameplayCue 再触发一次死亡奖励；重复伤害、同一帧多次命中、晚到的结果，都应该是幂等的。

准备的证据：死亡事件的 ID 或状态，或者对应的测试。

AS08　[AS] 旧 Pawn 销毁前后，哪些清理放 EndPlay，哪些放重生流程?

考点是“清理时机跟着关系走”。玩法关系结束时，就要解除 Timer、委托、输入和任务，不要等 GC。

同时注意两条边界：跨 Pawn 的持久状态不能跟着旧 Pawn 被清掉；EndPlay 的原因不同，也要防止误清了新对象的东西。

准备的证据：讲一处真实的清理代码，以及为什么放在那里。

射击、预测与权威

AS09　[AS] 客户端开火请求到底提交什么，服务器验证什么?

这题考信任边界。客户端只提交意图和最少必要的参数；服务器验证武器、射速、资源、起点和方向的边界，以及角色状态。

客户端算出来的命中和伤害不能直接当真。被拒绝时，要有可以诊断的原因，并让客户端收敛回来。

准备的证据：Server RPC 的参数，以及服务器上的验证入口。

AS10　[AS] LocalPredicted Ability 在 AS 中预测了什么，没有预测什么?

考点是你是否清楚 GAS 预测的边界。先列出真正受 GAS 预测支持的东西：状态、消耗、表现。再说明哪些不会因为配置了 LocalPredicted 就自动可回滚：Actor 生成、物理、背包、伤害等自定义副作用。

最后强调：服务器仍然独立执行 CanActivate、Commit 和命中裁决。

准备的证据：一条成功的日志和一条服务器拒绝的日志。

AS11 [AS] PredictionKey 与 ShotId 为什么不能合并?

考点是你是否理解两个编号的作用域不同。PredictionKey 属于 GAS 的预测作用域；ShotId 是项目里“一次射击”的业务身份。

一次能力可以开火多次，一次射击可以有多个弹丸，所以两者对不上。ShotId 还需要自己的生成、去重、超时和跨重生规则。

准备的证据：打印同一个 Ability 内的多个 ShotId，以及各自对应的 PredictionKey。

AS12 [AS] 本地预测弹体与权威弹体怎样配对?

这题考的是对账逻辑。预测对象只负责即时反馈；权威对象或权威结果由服务器产生。

配对的方式是：按角色或武器的作用域加上 ShotId 找到预测对象，权威结果到达时平滑替换或清理。匹配失败、重复到达和超时，都要有明确的处理路径。

准备的证据：录下拥有者、服务器和旁观者三端的画面，并标出 ShotId。

AS13 [AS] 服务器拒绝这次预测射击时客户端如何回滚?

考点是失败路径。先区分拒绝原因：能力被拒、资源不足、射速、目标或姿态不合法。

回滚时，清理预测弹体和表现；预测扣掉的消耗，由 GAS 或项目自己的协议恢复。还要保证旧的拒绝结果不会影响后来同一个 ID 空间里的新射击。

准备的证据：故障注入一次服务器拒绝，并展示最终状态。

AS14 [AS] 权威弹体很晚才复制到客户端，表现怎么处理?

这题考高延迟下的表现策略。预测弹体继续表现，但要设寿命或超时；权威弹体到达后，根据误差大小决定吸附、平滑，还是隐藏后替换。

两个底线：不能让两个弹体都产生伤害；旁观者没有本地预测，应该直接从权威状态得到稳定的表现。

准备的证据：高 RTT 条件下的录像。

AS15 [AS] 权威弹体还没复制就已经命中/销毁，客户端怎样结束预测对象?

考点是“不要依赖权威 Actor 一定会出现”。权威的命中结果或射击结果消息，必须能独立标识 ShotId。

客户端收到最终结果后，结束预测对象并播放必要的表现；如果那个迟到的 Actor 后来又出现了，要能识别出它对应的身份已经结束。

准备的证据：构造一次近距离、高延迟的命中。

AS16 [AS] 预测对象超时后怎么做，为什么不能无限等?

这题考你对“超时”的理解是否完整。为每个 ShotId 记录开始时间和终止状态，超时就清理表现并记录诊断信息。

关键的一点：超时不等于服务器一定失败了，迟到的结果仍然要幂等处理。超时阈值来自网络目标和玩法体验，不能拍脑袋定。

准备的证据：超时日志，以及迟到包的处理过程。

AS17　[AS] 开火后立刻换武器，迟到结果如何归属?

考点是结果归属。ShotId 要同时绑定武器、执行身份，或者其他能验证的上下文。

旧武器的结果只能结束旧的那次射击，不能修改当前武器的弹药或动画。服务器上的权威伤害可以继续有效，但表现归谁，按规则处理。

准备的证据：快速切枪的回归测试。

AS18　[AS] 开火后立刻死亡重生，旧结果怎样隔离?

这题和上一题同一个思路，换成了“身体”。重生会产生新的身体代次，旧的射击和任务保留旧 Avatar 和旧代次。

旧命中可以按服务器规则结算原来那次射击，但不能修改新 Pawn 的临时状态；客户端旧的预测表现要清理掉，不能误匹配到新身体上。

准备的证据：死亡瞬间开火，再加上延迟包的测试。

AS19　[AS] 霰弹枪一发多弹丸时 ShotId 怎样设计?

考点是编号的层级设计。一个 FireId 下可以挂 PelletIndex 或子结果，避免把每颗弹丸当成一把独立的武器来扣费。

伤害去重和网络消息可以按玩法聚合；还要说明服务器怎样验证散布和总弹数。

准备的证据：如果项目没有实现霰弹枪，就明确说这是设计题，不是项目事实。

AS20　[AS] Owner、Server、Observer 三端各自应该看到什么?

这题考你脑子里有没有“三端”的画面。拥有者的输入和预测反馈最快；服务器负责权威状态和命中裁决；旁观者看到的是复制和插值之后的结果。

还要分清三类东西：本地预测对象、权威 Actor、纯表现的 Cue。用同一个 ShotId 贯穿三端的日志，方便对账。

准备的证据：三端同步的录像。

伤害、效果与表现

AS21　[AS] 伤害从命中到 Attribute 再到死亡，完整链是什么?

这题要求你把一条链从头讲到尾。依次说明：命中判定由谁负责，EffectSpec 和 TargetData，Execution、MMC 或 Modifier，Attribute 的变化，死亡规则。

最终伤害只由服务器提交，客户端的表现不能成为真值。每一层都要说出可以观察的点和失败条件。

准备的证据：拿一发真实的射击，逐段给出日志和代码。

AS22 [AS] 技能消耗应该在 Activate 还是 Commit，为什么?

考点是你能否把技术选择和玩法规则联系起来。先说项目的规则：是起手就支付，还是真正发射或确认之后才支付。

然后讲约束：Commit 失败后，必须终止后续的提交；要避免输入、Notify 和服务器确认各扣一次；预测的消耗和权威的确认要能收敛。

准备的证据：展示一次取消或被拒绝时，资源的最终数值。

AS23 [AS] 爆头/暴击/武器参数怎样从命中进入伤害计算?

这题考数据分层。把静态的武器配置、命中的事实、可变的上下文分开，分别用 TargetData、EffectContext、SetByCaller 等合适的载体承载。

可能被伪造的数据，服务器要验证；计算层不要从随便某个 UI 或动画对象反查真值。

准备的证据：指出每个参数的来源，以及服务器在哪里验证。

AS24 [AS] TargetData 到服务器后为什么仍不能直接信?

考点还是信任边界。TargetData 是客户端提交的目标信息，不等于合法的命中。

服务器要检查范围、视线、射速、时间和姿态，以及目标是否有资格被命中；高价值的结果要重新计算或验证。预测只是改善体验，不改变信任边界。

准备的证据：给出至少三项服务器端的验证。

AS25 [AS] 怎样防止同一命中结果重复扣血?

这题考幂等。射击或命中要有业务身份，服务器做短期去重，或者让提交入口天然只能执行一次。

客户端重试、网络重复、碰撞回调多次触发，都不能重复产生副作用。记录第一次提交，以及之后被拒绝的重复请求。

准备的证据：重复发送同一个结果的测试。

AS26 [AS] GameplayCue 为什么可能重复，怎样做幂等表现?

考点是表现层的重复从哪来。预测和权威确认可能先后各触发一次表现；持续型的 Cue 还涉及重新相关和晚加入。

Cue 不负责提交伤害；表现对象按 Cue 或执行身份去重，并成对地移除。还要讲清 OnActive、WhileActive、OnRemove 各自的语义。

准备的证据：录像证明“预测 + 确认”不会出现两份特效。

AS27 [AS] 持续效果对晚加入/重新相关客户端怎样恢复?

这题考“状态 vs 事件”。恢复的依据是持久的状态、GameplayEffect 和 Tag，而不是过去某一次瞬时的 Multicast。

客户端根据当前状态建立持续的表现；一次性的起手音效不需要重播。

准备的证据：让第二个客户端晚加入来验证。

AS28 [AS] AbilityTask 被取消时哪些外部工作不会自动消失?

考点是你是否清楚 AbilityTask 清理的边界。手动注册的委托、自己创建的 Actor、Timer、后台任务，都需要项目代码自己清理。

成功、失败、取消、打断、Avatar 更换，都应该进入同一个结束路径；晚到的回调要核对执行身份。

准备的证据：在能力执行到一半时 CancelAbility 的回归测试。

AS29 [AS] 旧 Montage Notify 为什么可能在新技能开始后仍造成问题?

这题考时序信号和状态修改的边界。Notify 只负责发出时序信号，处理的入口要核对当前的能力或动作身份。

旧技能结束时，关闭碰撞窗口、解除订阅；Notify 不能无条件地 SetState(Idle)，也不能直接扣费或造成伤害。

准备的证据：连续快速切换技能的测试。

AS30 [AS] 委托和 Timer 由谁解绑?

考点是“谁注册谁负责”。注册者，或者关系的拥有者，保存句柄，并在关系结束时解除。

弱绑定只解决一部分悬空问题，解决不了旧业务身份的问题。清理函数要能重复调用，而且只清理自己的工作。

准备的证据：列出 AS 中至少两类句柄。

AS31 [AS] Avatar 更换后动画/GAS 的连接怎样重建?

这题考重绑定的完整性。刷新 ActorInfo，让新的 AnimInstance、装备、输入和属性 UI 重新绑定到当前 Avatar，同时解除旧 Avatar 上的全部绑定。

服务器和客户端初始化的时间不同，必须允许依赖晚到。

准备的证据：重生前后对象名、指针或 NetGUID 的日志。

AS32 [AS] 武器 Actor 的 Owner、Attachment 和网络拥有权为什么要分开?

考点是三种关系的区分。Attachment 只决定空间层级；Owner 和拥有连接影响 RPC 以及相关的网络语义。

换武器或掉落时，空间关系和逻辑拥有关系要分别更新。服务器是武器最终状态的拥有者。

准备的证据：展示一把武器的挂接和设置 Owner 的路径。

调试、测试与验证

AS33 [AS] AS 的结构化日志最少应该打印哪些字段?

这题考你会不会让日志真正有用。最少的字段是：执行端或 Role、World、玩家、Pawn 和武器的身份、ASC 的 Owner 和 Avatar、Ability 或 ShotId、阶段或状态、旧值到新值、拒绝原因。

每一条业务链用一个一致的 ID 串起来。日志的目的是证明“第一个分歧点在哪”，不是越多越好。

准备的证据：一条完整射击链的日志样例。

AS34 [AS] 怎样模拟 RTT、抖动、丢包并验证预测收敛?

考点是你有没有在坏网络下测过。用可以控制的网络模拟参数，固定测试动作，以及种子或场景。

观察三端的最终结果：位置、资源、射击匹配和 Cue。记录在什么阈值下开始失败，以及哪些体验不可接受。

准备的证据：至少覆盖低、中、高三档 RTT，再加上丢包。

AS35 [AS] 服务器拒绝测试应该覆盖哪些原因?

这题考失败路径是否被认真对待。要覆盖的原因包括：技能资格、冷却和资源、射速、目标不合法、死亡或眩晕、武器已经切换、请求已经过期。

每一种拒绝，客户端都要有清理，也要有可以诊断的原因；多次被拒绝之后，后续合法的射击仍然要正常。

准备的证据：最好是自动化或脚本化的故障注入。

AS36 [AS] 怎样证明不会重复授予能力/重复绑定?

考点是“用计数说话”。每次初始化时，打印或断言 AbilitySpec 和委托的数量。

在连续重生、重新 Possess、客户端重连之后，数量仍然保持预期；必要时提供一个幂等的初始化函数。

准备的证据：计数型的证据。

AS37 [AS] 为什么要做连续 50/100 次重生压力回归?

这题考你是否理解生命周期错误的特点：它们往往在多次绑定、旧 Timer、残留的弱引用之后才累积出来。

压力回归时监控能力数、委托数、Actor 数、内存、重复的日志，以及功能是否仍然正确；一旦失败，定位到第一次偏离的那一代。

准备的证据：一个脚本，或者可以手动复现的操作序列。

AS38 [AS] 多人同时开火时要验证什么?

考点是并发和隔离。要验证：ShotId 的作用域不会冲突；服务器上提交逻辑的线程或游戏线程语义是明确的；不同玩家的结果不会互相清理。

还要看相关性和带宽压力下，旁观者的表现能否恢复；死亡、重生和同时命中，不会产生重复的死亡。

准备的证据：至少 2 到 4 个客户端并发的录像。

取舍与表达

AS39 [AS] AS 网络优化优先看什么，而不是先上 ReplicationGraph/Iris?

这题考优化的顺序感。先量化：Actor 数量、属性更新频率、RPC、带宽、相关性和休眠，确认瓶颈在哪。

先减少不必要的状态和频率、正确设置相关性，之后再考虑更复杂的后端。Iris 和 ReplicationGraph 是工具，替代不了一个错误的状态模型。

准备的证据：Insights 或网络统计数据。

AS40 [AS] 打包版与 PIE 行为不一致时怎样查?

考点是环境差异的拆分。区分配置、Cook 的资源、网络模式、初始化时序和 EditorOnly 依赖。

保留目标配置下的日志，复现同一条射击或重生链。不能用“PIE 里正常”证明发布版本是对的。

准备的证据：一次 Development 或接近 Shipping 配置的回归。

AS41 [AS] 为什么 AS 用 GAS，而不是完全自研技能框架?

这题考的是技术选型的理由是否站得住。从标签、效果和属性体系、网络预测、AbilityTask、可扩展性，以及项目的学习目标几个方面讲收益。

同时承认 GAS 的复杂度、初始化和复制的成本，以及它不适合的简单场景；再说明哪些部分仍由项目自己定制。

准备的证据：用一个具体的能力，说明 GAS 实际提供了什么。

AS42 [AS] ASC 放 PlayerState 的复制与生命周期代价是什么?

考点是“代价意识”。PlayerState 寿命长，而且通常被广泛复制，所以 ASC 和属性的配置要考虑复制模式和更新频率。

重生变简单了，但 Avatar 的时序、持久效果和临时效果之间的边界，变得更复杂。要能说明项目为什么接受这个权衡。

准备的证据：结合实际的 NetUpdate 和复制设置来讲。

AS43 [AS] 如果不用预测弹体，还有哪些方案？为什么没选?

这题考你是否了解其他方案。可选的有：只做本地特效加权威 Hitscan、纯权威的投射物、客户端预测轨迹加服务器判定命中、服务器回溯等。

从武器速度、RTT、作弊、公平性和实现成本几个维度比较，不要把当前方案说成唯一正确的选择。

准备的证据：说出项目的目标，以及做出的取舍。

AS44 [AS] AS 目前哪些结论没有真正验证?

这题考诚实。主动列出还没覆盖的部分：高 RTT、丢包、多人规模、打包版、边界武器和重生等。

讲清“未验证”和“已知错误”不是一回事，并给出下一步的验证方案。面试中宁可把边界说明白，也不要冒充生产环境的实测结果。

准备的证据：一张“已验证 / 未验证”的表。

AS45 [AS] 60 秒介绍 AS，怎样避免堆技术名词?

考点是表达。先说一个能玩的主循环或要解决的问题，再说你负责的两个核心技术难点。

用“对象、状态、执行端、失败”讲一条有代表性的链，例如“ASC 跨重生 + 预测弹体对账”。最后给出运行证据，以及还没验证的边界。

准备的证据：能在 60 秒内自然地把面试官引到下一层问题上。

C. ISD 项目追问 • 30 题

ISD 的价值不在功能有多少，而在交互、物理、Gloo 和关卡规则能否组合成一个可玩的系统，以及你能否讲清楚其中的生命周期和失败恢复。下面的题目只告诉你该想什么、该准备什么；ISD 是否联机、具体用了哪些 API，以你的代码为准。

交互与规则

ISD01　[ISD] 交互系统的“玩家、目标、权限、世界状态”怎样解耦?

这题考你能不能把一个交互拆成四个角色：玩家只发出交互意图；目标对外暴露可以交互的能力；权限和规则单独判断；世界状态由目标本身或规则的宿主持有。

一个容易被追问的细节是：交互提示只是表现，不是“能不能交互”的真值。用门、物品、密码面板三种对象举例，说明怎样避免 Character 里的分支越写越多。

准备的证据：展示接口、组件和规则对象之间的关系。

ISD02　[ISD] 接口和 ActorComponent 各适合放什么?

考点是两种复用方式的区别。接口表达“能做什么”的契约；组件承载可以复用的状态和行为。并不是所有目标都必须挂同一个组件。

组件的寿命跟着宿主走；通过接口调用时，仍然要处理目标已经失效的情况。避免做出一个“万能交互组件”，什么子系统都知道。

准备的证据：指出至少两类交互对象在实现上的差别。

ISD03　[ISD] 凭证、密码和任务条件怎样组合?

这题考规则的组合能力。把每个条件建模成可以组合的规则，判断结果要带上“允许”或“拒绝的原因”。

凭证数据和 UI 输入要分开，由服务器或权威的玩法层判断。规则变化之后，门的状态和提示应该从当前状态重新推导出来。

准备的证据：举一扇门的规则配置作为例子。

ISD04　[ISD] 门的开关状态、动画和碰撞为什么要分开?

考点是“状态 vs 表现”。Open、Locked 这些是可以恢复的状态；动画只是从一个状态过渡到另一个状态的表现。

碰撞必须和当前的权威状态一致，晚加入的玩家不能靠过去播放过的动画来恢复。门在运动中途被反向切换或打断时，要定义好进度或目标状态。

准备的证据：切换地图或重新进入后，验证门的状态、动画和碰撞一致。

物理抓取与碰撞恢复

ISD05　[ISD] PhysicsHandle 抓取时真正控制的是哪个对象?

这题检验你是否真正搞清楚了控制目标。先定位实际在模拟物理的 PrimitiveComponent、抓取点和目标姿态。

PhysicsHandle 用约束式的方式把物体拉向目标，不等于每帧 Teleport 一个 Actor；质量、阻尼、刚度和碰撞共同决定手感。

准备的证据：展示 Updated/Grabbed Component。

ISD06　[ISD] 不同质量物体为什么抓起来手感差异很大?

考点是调手感的方法论。物理参数和质量会影响响应。先保证控制源是唯一的，再去调刚度、阻尼和插值。

必要时可以按质量分档，或者限制能抓起的范围，但要说明这背后的玩法取舍。用固定的测试轨迹，比较超调、滞后和抖动。

准备的证据：一组可以重复的参数对照。

ISD07　[ISD] 持有物与玩家/墙碰撞怎样处理?

这题考你有没有想清楚持有期间的碰撞策略：忽略玩家、保持世界碰撞、限制穿墙，或者调整目标位置。

修改碰撞前保存原来的响应，恢复时只还原自己改过的部分。目标被销毁或换到另一只手时，要能安全结束。

准备的证据：碰撞配置修改前后的截图或日志。

ISD08　[ISD] 为什么“放手”和“恢复碰撞”是两个状态?

考点是两件事的完成时间不同。玩家应该立刻退出持有状态；恢复碰撞则可以等物体离开角色的安全区域。

如果混在一起，旧物体被卡住时，会挡住下一次拾取。所以两个过程各有自己的身份和结束条件。

准备的证据：复现“贴着玩家放手”的场景。

ISD09　[ISD] 旧碰撞恢复任务怎样避免破坏新拾取?

这题考旧任务和新状态的隔离。恢复任务要记录具体的组件、原来的碰撞响应，以及持有或恢复的代次。

物体被再次拾起时，旧任务要被作废，或者执行前检查身份；旧任务只能恢复它自己修改过的状态。

准备的证据：“放下后立刻再拾起”的回归测试。

ISD10　[ISD] 待恢复物体被销毁或关卡卸载怎么办?

考点是等待中的对象消失时怎么处理。回调要使用弱引用或有效性检查，并绑定 World 和任务的身份。

对象销毁或 EndPlay 时，取消等待，或者让结果自然失效。不能为了等待恢复而强行保活世界中的对象。

准备的证据：在等待中途销毁对象的测试。

Gloo 泡沫

ISD11　[ISD] Gloo 的附着、反弹、回收规则如何组织?

这题考规则的组织方式。把材质、Receiver、目标的运动状态等输入，转换成明确的规则结果。

视觉和音效只消费这个结果，是否附着由规则决定。重复命中、动态物体和无效目标，都要有确定的分支。

准备的证据：列出规则表，再举一个失败的例子。

ISD12　[ISD] 为什么泡沫生长不能只改 Mesh Scale?

考点是“看起来”和“实际上”的一致。显示尺寸、碰撞体、导航和可行走性、附着关系，在语义上要保持一致。

生长过程中可能夹住玩家，或者穿插进场景，所以要做空间检查。不同阶段的碰撞可以分段启用。

准备的证据：一段生长前后的碰撞可视化。

ISD13　[ISD] 怎样防止 Gloo 生长把玩家卡死?

这题考生长的安全检查。在生长之前或过程中，用 Sweep、Overlap 或空间可用性判断检查空间。

检查失败时，可以停止、缩小、偏移或回收，但规则要稳定。如果角色已经在泡沫内部，不能把碰撞瞬间扩大到最大。

准备的证据：故意在狭窄的空间里测试。

ISD14　[ISD] 泡沫附着动态物体时坐标和寿命怎么处理?

考点是附着关系的维护。要明确使用世界坐标还是相对坐标，以及 Attachment 关系；宿主移动或销毁时，泡沫要跟着移动或被清理。

物理模拟的对象和附着约束不能互相冲突；放回对象池复用之前，要解除旧的 Attachment。

准备的证据：在动态平台上测试。

ISD15　[ISD] 泡沫对象池归还时必须重置什么?

这题考重置清单是否完整：可见性、碰撞、物理速度、材质和阶段、Timer、委托、Attachment、Receiver 和命中身份。

每次租出都产生新的业务身份，旧回调核对身份时就会被拒绝。池的容量由真实的峰值和创建成本决定。

准备的证据：连续复用同一个实例的测试。

世界服务与异步

ISD16　[ISD] 为什么使用 WorldSubsystem？什么时候不该用?

考点是寿命匹配。如果一项服务只属于当前 World——比如交互、对象池、调度——WorldSubsystem 的寿命正好合适。

跨地图的玩家数据不应该塞进去，进程级的 static 更不合适。PIE 中有多个 World 时，每个 World 都有自己独立的实例。

准备的证据：指出它实际承担的职责。

ISD17　[ISD] World 切换时 Subsystem 中的待处理任务怎么办?

这题考服务销毁时的收尾。WorldSubsystem 销毁时，要停止或作废任务和回调。

异步结果要带着 World 和请求的身份，晚到时只释放自己的东西；不能把旧 World 的 Actor 带到下一张地图。

准备的证据：在切换地图的过程中打断的测试。

ISD18　[ISD] 异步资源或查询晚到如何避免修改新关卡对象?

考点是晚到结果的归属。请求要记录目标 Actor 或稳定 ID、World 和代次。

完成时重新解析目标，并核对当前的关系；不能因为对象 IsValid，就认定旧结果自动合法。

准备的证据：切换地图或重建目标的测试。

关卡设计与扩展

ISD19　[ISD] 门禁、搬运、Gloo 和任务怎样组合成“关卡问题”而不是功能展柜?

这题考的是你有没有设计思维。先定义玩家的目标和约束，再让多个系统产生互相依赖的解法。

每个系统至少要改变一个能被观察到的世界状态，并且这个状态能被其他系统读取。关卡要有失败和替代方案，而不是按顺序按按钮。

准备的证据：用一段完整的关卡流程来说明。

ISD20　[ISD] 怎样设计一个能体现 ISD 技术亮点的垂直切片?

考点是怎样展示技术。把流程控制在 5 到 10 分钟：门禁挡路 → 寻找并搬运物体 → 用 Gloo 改变空间 → 任务状态解锁。

技术要嵌在玩法选择里，而不是弹出一个调试菜单介绍 API；同时录下失败和恢复的路径。

准备的证据：一个陌生人不需要讲解就能玩通。

ISD21　[ISD] 任务状态和世界交互怎样避免互相硬引用?

这题考系统之间的连接方式。通过稳定 ID、事件或查询接口、规则服务来连接，不要让所有 Actor 互相持有裸引用。

任务消费的是“门已解锁”“物体已放置”这样的事实；目标被流送或销毁之后，状态仍然能恢复。

准备的证据：指出一次跨系统的通信。

ISD22　[ISD] ISD 如果需要存档，应保存什么?

考点是存什么、不存什么。保存任务阶段、门和关键交互的状态、物品的稳定 ID 和位置等必要的数据。

物理的瞬时速度要不要存，取决于玩法，不能无脑把所有组件都序列化。加载时按顺序：重建对象 → 应用状态 → 恢复表现。

准备的证据：设计一个最小的存档结构。

ISD23　[ISD] 如果以后扩成多人，第一批要改的边界是什么?

这题考你对网络边界的预判。要明确哪些交互由服务器裁决：拾取、门、Gloo 的生成和附着、任务状态。

PhysicsHandle 的本地手感和权威物理之间，会产生预测和同步的问题。这是一道扩展设计题；如果当前项目是单机，必须明确说还没实现。

准备的证据：面试时先声明当前实际的网络范围。

调试、测试与表达

ISD24　[ISD] 大量 Gloo/可交互物导致性能问题，先量什么?

考点是先测量再优化。分别测量 Actor 和 Tick、物理碰撞、组件注册、材质和半透明、导航更新与查询。

对象池只解决创建和销毁的热点，不会自动降低每帧的物理成本；非关键的更新可以按距离和可见性降低频率。

准备的证据：Insights 或统计数据的截图。

ISD25　[ISD] 交互反馈如何连接输入、状态和动画/音效?

这题考一条完整的交互链：输入发出意图 → 规则给出结果 → 提交状态 → 表现消费结果。

失败时也要给出明确的反馈，但不修改权威状态；表现被中断，也不能反过来破坏已经完成的交互提交。

准备的证据：一条开门或拾取的完整链路。

ISD26　[ISD] 最容易做的故障注入有哪些?

考点是你会不会主动制造故障。几种容易做的：抓取过程中销毁目标；放手后立刻再拾起；Gloo 生长时移动宿主；门的动画播放中反向切换状态；切换地图时异步回调返回。

每次注入前先定义验收标准：“不崩溃、不污染新状态”，并记录第一个分歧点。

准备的证据：至少做三项，并保留证据。

ISD27　[ISD] 调试物理交互要打印什么?

这题考日志设计。要打印：World、Actor 和 Component，物理模拟状态，碰撞响应，抓取和释放的代次，原始和目标的 transform，Handle 的目标。

只在关键状态变化时记录，避免每帧刷屏，并结合碰撞可视化一起看。

准备的证据：一条异常释放的日志。

ISD28　[ISD] 如何证明对象复用没有“脏状态”？

考点是用测试证明重置是完整的。每次租出前，断言关键字段处于默认值；归还后，统计 Timer、委托和 Attachment 的数量。

随机地反复租出给不同用途，检测旧状态是否泄漏；必要时给 reset 函数写自动化测试。

准备的证据：100 次复用的回归测试。

ISD29　[ISD] ISD 当前最值得补的是更多系统还是更强组合?

这题考你对项目方向的判断。优先完成一个可玩的垂直切片、做好反馈和失败恢复；让已有的系统能连续交互，比再加几个浅功能更有价值。

技术上，优先补可观察的证据、性能数据和边界情况，而不是增加名词的数量。还没实现的计划，要和当前的事实分开讲。

准备的证据：一张“已完成 / 下一步”的两栏表。

ISD30　[ISD] 60 秒介绍 ISD 怎样讲?

考点是表达。先用一句话说明目标：沉浸式的交互和物理关卡。

再挑两个难点：PhysicsHandle 的释放和碰撞恢复；Gloo 的规则、对象池和空间检查。最后说明这些系统怎样组合成关卡，以及现在还缺什么。

准备的证据：能自然地引出一个真实 Bug 的修复故事。

D. 绿洲 / Lua 实习追问 • 25 题

这一部分的核心是两件事：生产经历要真实，边界要克制。不要把平台现成的功能说成自己研发的 UE C++ 系统。真正值得展示的是 Lua、UI、多端联调、生命周期、问题定位和交付意识。平台的绑定层、Lua 版本和有效性接口，以平台文档和你的实际经历为准。

职责与边界

OZ01　[绿洲] 怎样一句话界定你在绿洲的真实职责?

这题考诚实和清晰。先说你实际负责的范围：哪些功能、哪些脚本、哪些 UI、哪些联调；再明确哪些能力是平台、策划或现成资源提供的。

只对你自己实际修改、调试和验证过的那条链做技术上的主张。生产流程的经验和 UE C++ 的深度要分开讲。

准备的证据：三个“我负责 / 平台负责”的对照例子。

Lua 语言与生命周期

OZ02　[绿洲] Lua table / metatable / __index 在项目里最容易出什么问题?

考点是 Lua 对象模型在实际项目中的坑。先讲清原型查找、共享表、实例表和写入代理各自的语义。

多层 `__index`，或者框架自带的 OOP 写法，容易让状态写到错误的表上；调试时要确认 rawget/rawset 访问的是哪个实际对象。不要把 Lua 的类模型等同于 C++ 的虚表。

准备的证据：用一段实际的脚本或最小示例来解释。

OZ03　[绿洲] 闭包/upvalue 为什么会让旧界面或旧角色长期存活?

这题考引用链。事件或 Timer 保存着闭包，闭包又捕获了表或对象代理，所以 Lua 的 GC 仍然认为它们可达。

关系结束时要解除订阅，回调执行前核对它是否仍属于当前对象。把一个局部变量设成 nil，不代表所有引用都断开了。

准备的证据：展示一次解绑的写法。

OZ04　[绿洲] Lua coroutine 与 UE Timer/异步任务的差别?

考点是三种异步机制的区别。coroutine 是协作式地保存执行状态，通常不会自动切换线程；Timer 是按时间调度的回调；异步任务可能在线程池上执行。三者的取消和对象访问规则都不同。

yield 期间，World 和对象都可能变化，resume 之前要重新验证。

准备的证据：能讲一个“协程醒来时，对象已经换了”的例子。

OZ05　[绿洲] Lua GC 与 UObject GC 两套系统怎样产生悬空?

这题考两套回收系统的边界。Lua 代理还可达，不代表底层的 UObject 仍然可用；UObject 还活着，也不代表旧脚本里的关系仍然是当前的。

绑定层需要有效性检查、销毁通知和解绑协议。不要把“Lua 变量不是 nil”当成对象有效的证明。

准备的证据：说清楚平台实际提供的有效性 API。

OZ06　[绿洲] 事件订阅怎样避免重复绑定和旧回调?

考点是订阅的生命周期。保存句柄或注册身份；初始化可以重复执行，但不能重复订阅。

在销毁、关闭界面、重生或更换对象时解除订阅；晚到的回调核对当前对象和请求。

准备的证据：一条真实的事件链。

状态、配置与多端

OZ07　[绿洲] UMG、Lua/蓝图与玩法状态怎样分工?

这题考状态归属。UI 只负责显示状态和发送意图；脚本或玩法层维护运行时的真值；DataTable 和配置保存静态规则。

避免 UI 和 Lua 各自保存一份可写的阶段或库存。重建 UI 时，要能从真值恢复。

准备的证据：用一个按钮或一个面板来说明。

OZ08　[绿洲] DataTable/配置错误怎样尽早发现?

考点是“尽早失败”。读取时校验行名、引用、取值范围和必填字段，错误信息要输出具体的资产和字段。

编辑器里的验证和运行时的保护要分层；服务端或打包部署的配置，要和编辑器预览区分开。

准备的证据：列出一个你实际遇到过的配置类问题。

OZ09　[绿洲] DS、Client、PIE 状态不一致，第一处看哪里?

这题考排查的起点。先明确谁是权威、对象在哪个实例里、请求从哪一端发出。

然后沿着“输入或请求 → 权威状态 → 同步或事件 → UI”找第一个分歧点。日志里要带上执行端、对象身份、阶段和请求 ID。

准备的证据：一个多端日志的格式。

OZ10　[绿洲] 为什么 UI 显示正确不等于服务器状态正确?

考点是表现层和真值的区别。UI 可能有本地缓存、预测或旧的绑定，它只能证明“表现层现在显示的是什么”。

关键状态要回到权威宿主或同步消息上核对。重连和重生，是最容易暴露旧 UI 绑定的场景。

准备的证据：展示一个“状态对、UI 错”的排查，或者反过来的例子。

OZ11　[绿洲] 重生后 UI/脚本残留旧状态怎样处理?

这题考重绑定。先解绑旧 Pawn 和旧数据源，等新对象到齐后再重新绑定，并保证初始化是幂等的。

脚本里的闭包、Timer 和 UI 模型都要有生命周期。不要用固定的 Delay 等 PlayerState。

准备的证据：多次重生的验证。

OZ12　[绿洲] 跨端/多环境上线差异怎样定位?

考点是差异的拆分。区分资源、配置、网络模式、平台接口、构建版本和权限。

保存版本号和配置摘要，把问题缩小到“编辑器与目标环境之间第一个不同的地方”。本地成功不等于部署成功。

准备的证据：一次实际的联调或部署案例。

OZ13　[绿洲] 热更新能改什么，不能改什么?

这题考热更新的边界。脚本函数和配置比较适合热更新；已经在运行的闭包、协程、订阅和实例状态，不一定会自动迁移。

C++ 的类布局、反射和网络协议、已经加载的资产关系，都不能靠重新加载 Lua 安全地重建。热更新需要版本、迁移和失败回滚的边界。

准备的证据：说清楚绿洲实际支持的热更新范围。

定位、回归与交付

OZ14　[绿洲] 复杂 Bug 为什么先找“第一个分歧点”？

考点是排查方法。同时修改多个系统，就丢失了因果关系。

写出预期的链条，在每个边界记录输入、执行端、对象和状态变化，找到第一次出现不同的地方；每次只验证一个假设，并保留回归用例。

准备的证据：准备一个真实 Bug 的复盘。

OZ15　[绿洲] 生产日志怎样既有用又不刷屏?

这题考日志的取舍。状态变化和关键边界用结构化字段记录；高频的 Tick 不逐帧打印。

同一个请求或任务用一致的 ID 串起来；错误日志要包含配置、对象、执行端和版本。

准备的证据：展示日志的字段。

OZ16　[绿洲] 修一个小 Bug 时如何控制回归范围?

考点是修复的克制。先做最小复现、找到根因，只在最小的责任边界内修改。

列出可能受影响的相邻系统、执行端和生命周期路径；修完后，覆盖原来的复现、正常路径，再加一到两个边界情况。

准备的证据：讲一个你实际的修复流程。

OZ17　[绿洲] 长列表/UI 卡顿时先查什么?

这题考性能排查的顺序。把条目数量和虚拟化、每帧的绑定或 Tick、布局失效、资源加载、绘制批次分开测量。

条目复用时要完整刷新，并取消旧的异步请求。不要一上来就重写 Slate。

准备的证据：Profiler 或统计数据。

OZ18　[绿洲] 脚本状态和引擎对象寿命如何对应?

考点是两边寿命的关系。脚本层持有代理，并不会改变底层 Actor 的 Destroy 或 World 生命周期。

回调进入时，先验证对象和业务身份；脚本对象本身也要定义好谁创建、谁注销、什么时候回收。

准备的证据：画出一条对象关系。

OZ19　[绿洲] 与策划联调时，怎样把“需求不清”转成技术契约?

这题考沟通和建模。确认输入、状态、失败条件、能看到的反馈和验收场景。

把可配置的部分和程序必须保证的不变量分开；时序或多端上有歧义的需求，先做一个最小的状态机。

准备的证据：一个小需求的拆解例子。

表达与诚信边界

OZ20　[绿洲] 绿洲经历怎样迁移到 UE C++ 岗而不夸大?

考点是“可迁移”和“不可冒充”的分界。能迁移的是：生命周期意识、多端联调、状态单一真值、日志定位、回归和交付的习惯。

不要声称平台上的 Lua 功能等同于自己写过 UE 底层；真正的 UE C++ 机制证据，由 AS 和 ISD 来补。

准备的证据：一张“可迁移 / 不可冒充”的两栏表。

OZ21　[绿洲] 如果面试官追问“这是不是平台自带的”，怎么答?

这题考诚实。直接区分平台的能力，和你自己实现的逻辑、修复与联调。

讲你做出的技术决策、定位过程和验证，而不是抢功；如果只是调用了平台接口，也可以讲调用的边界和失败处理。

准备的证据：至少两个诚实的边界例子。

OZ22　[绿洲] 你在生产项目里怎样处理版本和回滚?

考点是工程流程。说明实际的版本控制和提交流程、最小改动、回归和问题复现。

脚本和配置的变更，要知道对应的是哪个资源版本。如果不确定公司内部的流程，就只讲你自己实际参与的部分。

准备的证据：引用真实的提交或任务记录（可以脱敏）。

OZ23　[绿洲] 一个线上/联调问题如何写成高质量复盘?

这题考复盘的结构：现象 → 影响 → 复现 → 第一个分歧点 → 根因 → 修复 → 回归 → 怎样预防。

区分事实和猜测，给出时间线和日志。避免只写一句“加了判空就解决了”。

准备的证据：准备一页复盘。

OZ24　[绿洲] 哪些技术问题你应该明确说“不知道/没做过”?

考点是诚信的边界。平台的底层实现、你没参与过的网络或引擎模块、没有实际验证过的新技术，都应该明确说不知道或没做过。

可以给出推理和验证方案，但要标明哪些是假设。守住诚信的边界，后面的深挖才有可信度。

准备的证据：列出三个常见的越界点。

OZ25　[绿洲] 60 秒介绍绿洲实习怎样讲?

这题考表达。先说真实的生产环境，以及你负责的功能和联调范围。

挑一个有代表性的问题——多端状态、Lua 生命周期，或者 UI、配置的 Bug——讲清你怎样定位、怎样回归。最后说这段经历带给你的工程习惯，以及为什么下一步要补 UE C++ 的核心生产能力。

准备的证据：讲得克制、具体，经得起追问。

E. 生产工程与趋势 • 15 题

这 15 题补的是面经里很少出现、但在实际生产中很重要的那部分职责。除了 P07–P10 这几道趋势题，其余大多可以作为目标岗位的工程深化来准备。

读这一部分时不要把它当 API 百科。每道题都抓住四个问题：它解决什么问题？什么时候值得用？什么时候不该优先学或优先用？出了问题看什么证据？新技术题还要能说出成熟度和迁移风险。

P01　[生产] Data Validation 应该验证什么，为什么不应该偷偷修资产?

Data Validation 解决的是“错误的配置要在编辑器里就暴露出来，而不是等到运行时”。它验证 DataAsset、DataTable、Blueprint 和关卡对象的结构性不变量：引用、取值范围和命名。

它值得用的前提，是错误能被定位到具体的资产和字段。自动修复只能做确定的、可以撤销的安全操作，否则资产会在没人察觉的情况下被改掉。

还要记住它的边界：验证器的作用是让问题提前暴露，并不能替代运行时的防御代码。

P02　[生产] UE 自动化测试怎样分层?

自动化测试解决的是“改了代码之后，怎样确定没把别的东西弄坏”。按需要的环境分层：纯计算和容器规则，用快速的单元级测试；需要 World 的玩法链，用自动化地图或功能测试；多人和打包版，用更高层的回归。

每一层都要定义失败条件、耗时和运行频率。不要把所有测试都写成慢速的端到端测试——那样会因为太慢而没人去跑。

P03　[生产] UAT / BuildCookRun 在流水线里解决什么?

它解决的是“构建发布能不能被稳定地重复”。UAT 和 BuildCookRun 把 Build、Cook、Stage、Package、Deploy/Run 这些步骤自动化，并产出可以审计的日志和产物。

失败时先按阶段定位，不要把“打包失败”当成一个黑盒。本地和 CI 使用相同的关键参数，减少环境之间的差异。

P04　[方向] BuildGraph/CI 值得实习生掌握到什么程度?

知道怎样把构建、Cook、测试、产物归档和门禁拆成可以复用的节点或阶段，就够了。

重点是可重复、缓存、失败定位和权限，不必背所有的 XML 和节点。除非简历上写了工具链相关的经历，否则不要让它占用主干的复习时间。

P05　[生产] 怎样给项目建立性能预算而不是“感觉还行”?

性能预算解决的是“性能好不好”没有标准可依的问题。先选定预算的维度和目标设备：帧时间、内存、网络、加载和卡顿等。

看的是 P95、P99 和最坏帧，并记录场景和版本。每一次优化都要有前后对照，还要有防止性能退化的门禁。

P06 [方向] World Partition / Data Layers 需要掌握什么?

要理解的核心只有一点：流送会让 Actor 和区域进进出出 World，所以玩法的持久状态不能依赖“对象一直存在”。

任务、AI 和交互，都要用稳定 ID 和更长寿命的状态来恢复。只有在方向对口的岗位上，才需要深入 HLOD、Data Layers、OFPA 等具体工作流。

P07 [趋势] Motion Matching / Pose Search 为什么是动画方向的进阶内容，而不是公共核心?

Motion Matching 解决的是“怎样让角色动作自然地衔接”：它根据查询特征，从 Pose Database 里选出最合适的姿态，适合高质量的移动动画和 3C。

但它并没有取代基础：它仍然依赖传统的玩法状态、动画数据、轨迹和特征，以及性能预算。UE5.5 已经有 PoseSearch 和 MotionMatch 的 API，但部分接口仍带有 Experimental 标记，所以它不能替代动画基础。做动画方向的同学值得深入，其他 Gameplay 岗位知道它解决什么就够了。

P08 [趋势] StateTree 与行为树怎么比较?

两者擅长的表达方式不同：行为树擅长按优先级和组合来做决策；StateTree 更显式地表达层次状态、状态转换和任务。

它们面临同样的难题：异步任务怎样结束、外部工作怎样清理、状态怎样失效。做 AI 方向的同学能比较两者就够了；不要因为 StateTree 新，就把现有的行为树全部重写。

P09 [趋势] Iris 在 UE5.5 为什么只做进阶认知?

Iris 的目标是改进复制系统的过滤、优先级、并行处理和扩展方式。

但在 UE5.5 中，它的 API 位于 Runtime/Experimental/Iris 路径下，迁移仍有差异和风险。所以应该先掌握传统的 Actor 复制、RPC、相关性、休眠、预测和调试，再来看 Iris。

P10 [趋势] Mover 为什么不能替代 CharacterMovement 作为实习主干?

Mover 面向模块化运动和 rollback 网络这两个目标，但目前仍然属于 Experimental 插件路线。CharacterMovement 才是成熟、广泛使用的现有主线。

所以知道 Mover 解决什么、迁移有什么风险就够了；等目标岗位明确采用时，再深入学习。

P11 [生产] 工具模块怎样避免污染 Runtime 和打包依赖?

这题解决的是“编辑器工具把运行时拖下水”的问题。编辑器工具放在 Editor Module，或者只作为 Editor-only 依赖；运行时的核心放在 Runtime Module。

依赖方向是从工具指向核心，运行时代码不引用 UnrealEd。验证的方法是做打包回归，确认没有 EditorOnly 的类或资源泄漏进去。

P12 [生产] 一个“可交付功能”的技术方案至少包含什么?

至少包含：对象与状态模型、接口与数据、执行端与权限、生命周期与失败恢复、性能和资源预算、调试和可观测性、测试、降级和回滚。还要说明和策划、美术之间的输入输出，以及哪些是可配置的。

判断方案好坏的标准，不是 UML 画得多，而是失败时能不能定位、需求变化时能不能演进。

P13　[生产] AI 辅助 UE 开发怎样设置验证门禁?

AI 生成的代码，最大的风险是“看起来能用”。所以门禁按顺序设置：约束修改范围 → 生成 → 静态检查 → 编译 → 资产和结构读回 → 运行断言 → 日志和截图 → 人工审查。

重要的状态和网络行为必须真实运行来验证，而不是让模型自己评价自己。每一步都要有可以重复执行的命令和明确的失败定义。

P14　[生产] 为什么“读回”是修改 Blueprint/资产后的关键步骤?

因为资产文件存在，不代表图结构、默认值、引用和编译状态都是对的。结构化的读回，可以确认节点、属性和依赖是否正确，然后再进入运行测试。

读回和运行是互补的：前者确认结构，后者确认语义。

P15　[生产] 面试中怎样描述 AI 使用而不削弱个人能力可信度?

讲清楚分工：你负责需求拆解、架构和约束、审查、测试和最终的责任；AI 负责哪些草稿或机械性的工作。把借鉴社区、AI 生成的部分，和你自己验证过的部分分开说。

最有说服力的，是讲一次 AI 生成的错误怎样被你的测试发现并修正，而不是说“AI 让效率提高了很多”。

F. 最终模拟面试清单

- 从底座层和 UE 核心层的主教材题里随机抽 20 道，每题 60–90 秒，必须讲到“结论 → 机制 → 边界或反例 → 项目联系”。
- 随机抽 5 道场景题，每题在 8 分钟内讲出对象、状态、执行端、生命周期、失败情况和验证方案。
- AS 至少连续追问 5 层：从一条简历描述一路追到代码和运行证据；遇到没验证过的边界，主动说明。
- ISD 用一段垂直切片，说明至少 3 个系统怎样组合；不能只罗列 PhysicsHandle、Gloo、Subsystem 这些名词。
- 绿洲的回答必须区分“平台提供的”和“自己实现、定位、验证的”，不要把 Lua/UGC 的生产经验包装成自研 UE C++ 底层。
- 至少做一次 C++ 现场编码，再做一次网络或生命周期 Bug 的口述定位；算法训练继续按 Hot100 独立推进。
- 任何“我优化了 X%”“我解决了多人问题”“已经稳定”之类的说法，都要准备好测量方法、基线、环境和可复现的证据。

终局停止条件

- 主教材不再因为单篇面经临时加题；只有在目标角色明显变化、或者发现生产能力缺口时，才修改核心内容。

- 182 题核心中，底座层和 UE 核心层的题能够闭卷稳定输出。
- AS、ISD、绿洲的每一条简历描述，都能被连续追问五层，并能指向代码、日志或运行结果。
- 至少有一个项目同时具备：可运行的视频、源码或结构证据、故障注入，以及性能、网络或生命周期日志。
- 趋势题只在目标岗位需要时才加深，不追逐所有 UE 新名词。